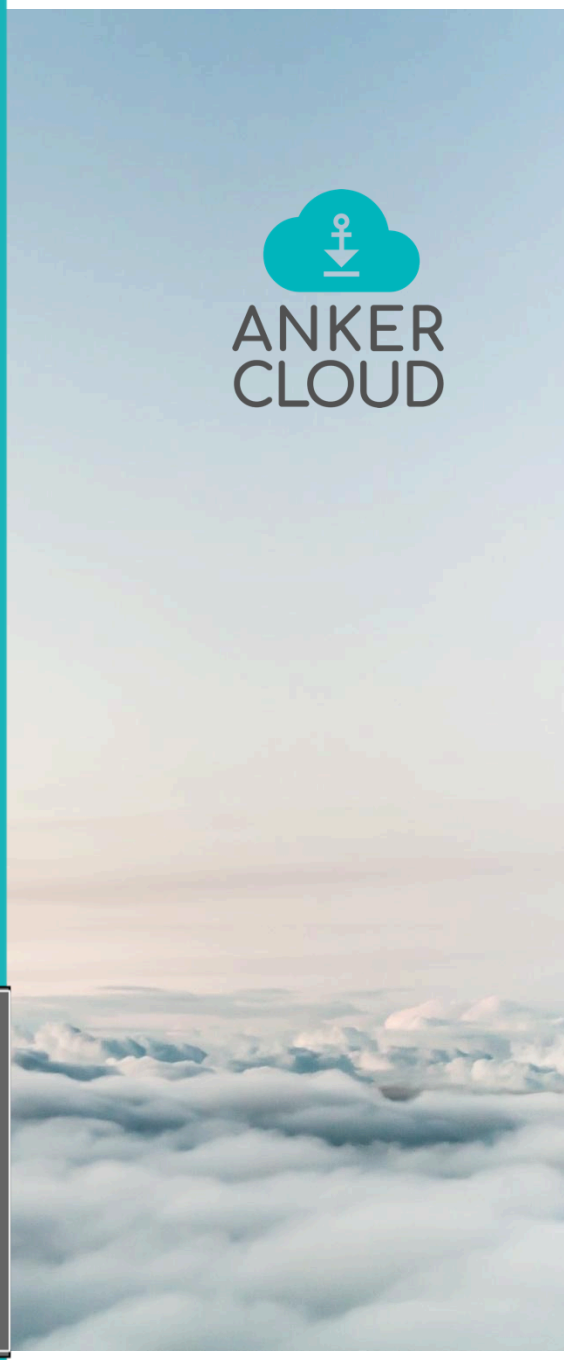
Knowledge Transfer Document

Bitech_Documentation

Bitech



Prepared by: Sakshi Mahendra Naik
(Data Engineer)
Reviewed by:
Abhishek Anand

TABLE OF CONTENTS

1. Introduction **4**
1.1 Project Overview 4
1.2 Goals and Objectives 4
2. Architecture **5**
2.1 Architecture Overview 5
2.2 Core Architectural Components 5
2.3 Core Design Principles 6
3. Data Extraction Pipeline **7**
3.1 Data Extraction Orchestration 7
3.2 Lambda Implementation for Data Extraction 11
3.3 Data Landing in S3 13
4. Data Lake Architecture **19**
4.1 Data Sources 19
4.2 S3 Medallion Structure 20
4.3 Medallion Layer Responsibilities 21
Silver Layer 22
Gold Layer 24
5. IAM Role and Design **24**
6. Glue Data Catalog & Metadata Management **32**
6.1 Glue ETL Jobs (Bronze → Silver Processing) 32
Steps to Create Glue ETL Job 33
6.2 Glue Crawlers (Schema Inference & Catalog Registration) 37
Steps to Create Glue Crawler 37
6.3 Glue Databases & Table Organization 44
7. Lake Formation Authorization Model **44**
7.1 LF-Tag Creation and Table Classification 44
7.2 Data Access Permissions Using LF-Tags 47
7.3 Column Level Access Control Using Data Filters 52
8. Lambda Implementation **55**
8.1 Athena Based Data Access Approach 57
8.1.1 Lambda 1 Code Link : bitech-prod-StartAthenaQuery-rate-limit 57
8.1.2 Lambda 2 Code : bitech-prod-GetAthenaResults-rate-limit 59
8.2 S3 Based Data Access Approach (S3 presigned URL : Data Download) 62
8.2.1 Lambda 3 Code : bitech-prod-s3-url 62
9. API Gateway and AWS cognito **63**
9.1 Amazon API Gateway Set up 63
9.1.1 Steps to Configure API Gateway with Rate Limiting 63
9.2 AWS Cognito Set up 71

9.2.1 Configure Amazon Cognito Authentication for a REST API 71
10. Step Function - Orchestration 74
10.1 Steps to Create State Machine 74
10.2 Steps to Eventbridge 78
10.2.1 EventBridge Rule Set Up 79
10.3 Step Function State Machine CODE : 79
11. End-to-End Data Access Demo (UI Walkthrough) 83

1. Introduction

1.1 Project Overview

The Purpose Based Access Control (PBAC) framework provides secure, scalable, and role-aware access to multiple datasets within a centralized Data Lake. It ensures users can access only the data aligned with their assigned roles or purposes. The system supports multiple data consumption methods, including querying via Amazon Athena and data downloads using S3 pre-signed URLs, with primary emphasis on Athena based data access to enable fine grained and controlled access.

The datasets include:

- **Traffic Data** - real time road conditions, congestion, and incidents.
- **Crime Data** - disturbances, safety alerts, and aggregated crime reports.
- **Events Data** - concerts, sports matches that are football, and crowd movement information.
- **Weather Data** - rainfall, temperature, storms, precipitation.
- **Insurance Data** - This contains Policy and claims data for insurance analytics.

Data is organized using a medallion architecture (Bronze, Silver, Gold) for structured storage, cleaning, enrichment, and aggregation. Combined with **Lake Formation LF-Tags and IAM roles**, PBAC ensures that sensitive or role-specific datasets are accessible only to authorized users, while maintaining security and governance across all data sources.

1.2 Goals and Objectives

The main goals of this project are:

1. **Secure Data Access** - Enforce role based access using PBAC so that each user can access only the datasets permitted for their purpose.

2. **Scalable Data Consumption** - Support multiple access methods including query-based analytics and file downloads using pre-signed URLs.

3. **Dynamic Data Filtering** - Enable dynamic filters (e.g., city_name, postal_code) to allow users to retrieve relevant subsets of data.

4. **Data Governance** - Maintain consistent schema, transformations, and controlled exposure through medallion layers and Lake Formation policies.

5. **Audit & Observability** - Track access patterns, query executions, and role usage for operational transparency and security.

6. **Fail-Safe Security** - Block unauthorized access entirely, ensuring no partial data leakage or accidental exposure.

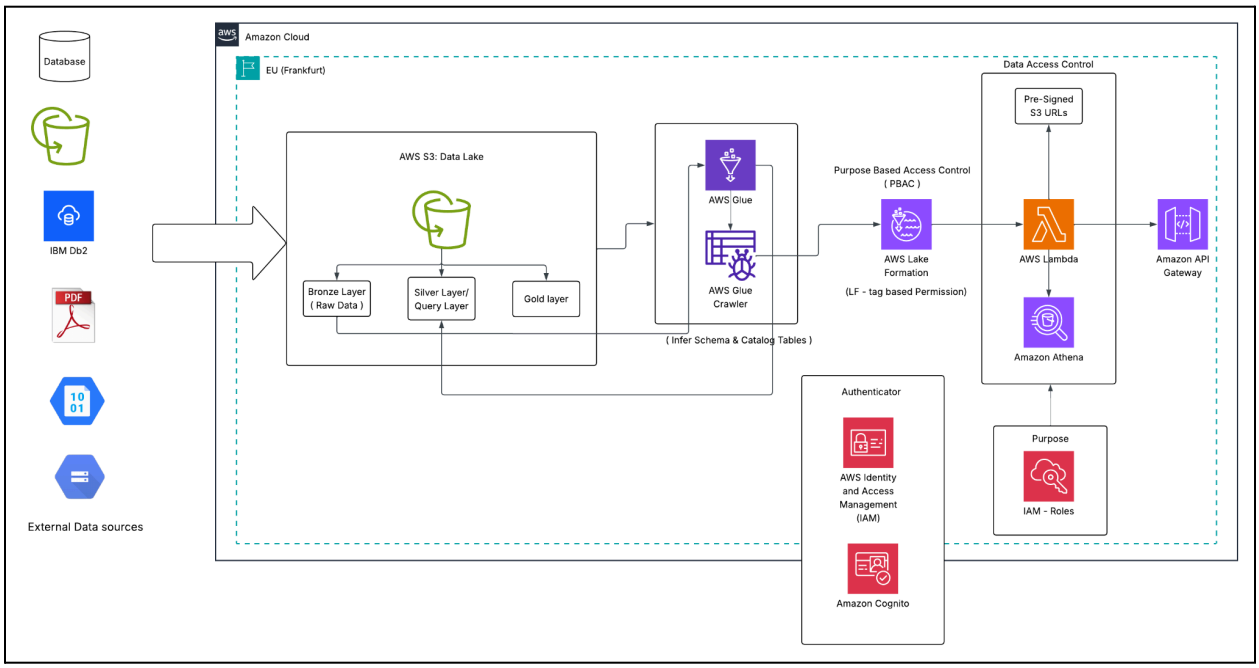
2. Architecture

2.1 Architecture Overview

The Purpose Based Access Control (PBAC) platform is built using AWS managed, serverless services and follows a purpose driven access model.

All data access is centrally governed using AWS Lake Formation while Lambda functions act as the only controlled entry points to the Data Lake.

Users never interact directly with Athena or S3.Instead, every request is authenticated, authorized, audited, and enforced through APIs.



2.2 Core Architectural Components

- **Amazon S3**
    - Acts as the centralized Data Lake storing all datasets in Bronze, Silver, and Gold layers.
- **AWS Glue (Jobs + Catalog + Crawlers)**

    - ETL jobs transform data from Bronze to Silver using best practices such as date partitioning.
    - Glue Crawlers infer schemas and register tables in the Glue Data Catalog.

- **AWS Lake Formation (PBAC Engine)**
    - All Glue tables are registered with Lake Formation.
    - LF-Tags are applied to tables and mapped to IAM roles.
    - Lake Formation implicitly enforces permissions during Athena and S3 access.

- **AWS IAM**
    - Defines purpose-based roles (Analyst, Marketing) that are assumed dynamically by Lambda.

- **AWS Lambda**
    - Acts as a **policy enforcement orchestrator**, assuming the correct role and executing Athena or S3 operations.

- **API Gateway + Cognito**
    - Cognito handles user authentication and issues JWT tokens.
    - API Gateway validates tokens and forwards identity context to Lambda.

2.3 Core Design Principles

- No Direct Data Access
    - End users:
        - Do not access S3 buckets
        - Do not access Athena directly
        - Do not receive IAM credentials

This prevents bypassing PBAC controls and ensures all access is governed.

- API Only Access
    - All data access flows through API Gateway and Lambda.
    - This enables:

        - Centralized authorization
        - Rate limiting
        - Auditing
        - Consistent enforcement across all access types

- Implicit Authorization via Lake Formation
    - Lambda does not decide whether a user can access a table.
    - Instead:
        - Lambda assumes a role based on user purpose
        - Athena/Glue calls are made using that role
        - Lake Formation evaluates LF-Tags on tables
        - Access is allowed or denied automatically

Lake Formation is the final authority for access decisions.

- Fail Closed Security Model
    - If a query references
        - Any unauthorized table
        - Any restricted dataset in a JOIN
    - Then:
        - The entire request fails
        - No partial results are returned
        - No data leakage occurs

This ensures strict enforcement of least privilege.

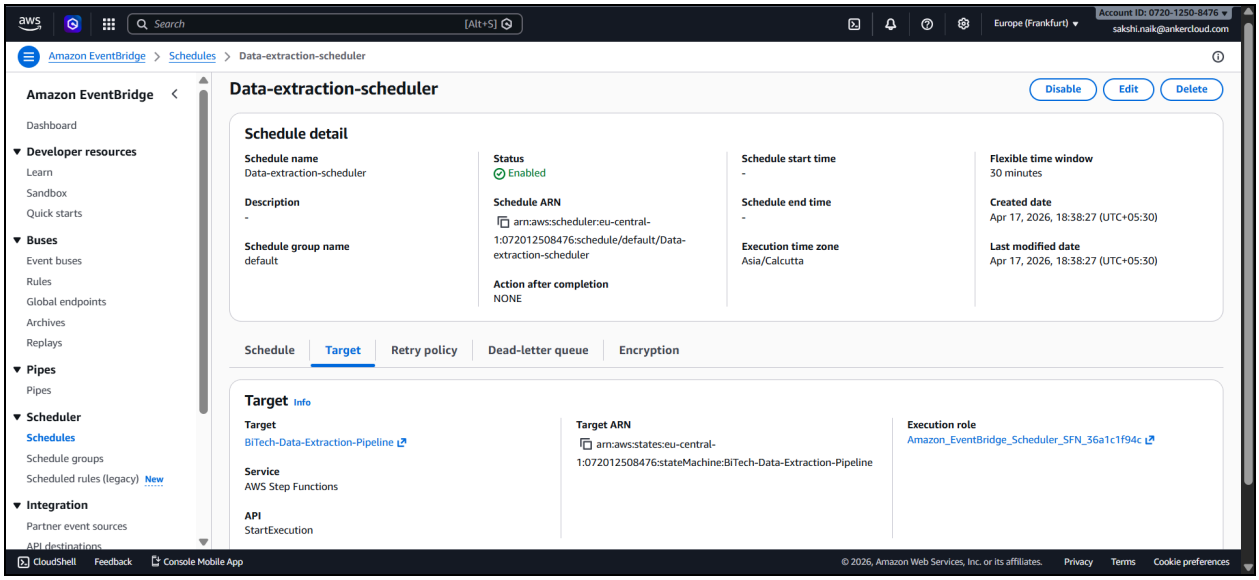
3. Data Extraction Pipeline

3.1 Data Extraction Orchestration

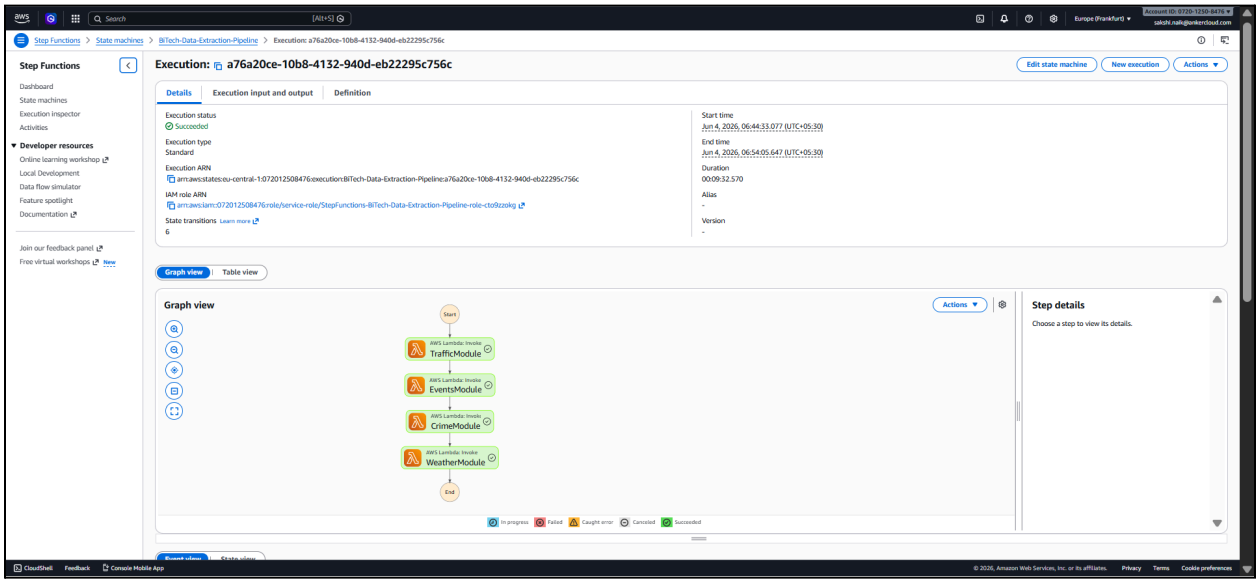
The orchestration of the extraction pipeline is handled by **AWS Step Functions** and scheduled via **Amazon EventBridge**. Rather than running all data modules sequentially in a single execution which risks timing out—the Step Function isolates each module (Traffic, Events, Crime, Weather) into its own 15-minute Lambda window.

- **Amazon EventBridge Scheduling**

    - EventBridge Scheduler is configured to trigger the state machine automatically every day at 12:00 PM IST (06:30 AM UTC) to ensure the data is fresh and available for downstream Glue jobs by 1:00 PM.



- **Step Functions State Machine**
    - The Standard Step Function passes a specific JSON payload (e.g., `{"step": "traffic"}`) to a single, centralized Lambda function. If a module fails due to temporary external API limits, native retry blocks automatically re-trigger the task before failing the workflow.



- **Step Functions State machine Code:**

```
None
{
```

```
  "Comment": "BiTech Data Extraction Pipeline - Sequential Execution",
  "StartAt": "TrafficModule",
  "States": {
    "TrafficModule": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName":
"arn:aws:lambda:eu-central-1:072012508476:function:bitech-pipeline",
        "Payload": { "step": "traffic" }
      },
      "Next": "EventsModule",
      "Retry": [
        {
          "ErrorEquals": ["States.TaskFailed"],
          "IntervalSeconds": 30,
          "MaxAttempts": 2,
          "BackoffRate": 2.0
        }
      ]
    },
    "EventsModule": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName":
"arn:aws:lambda:eu-central-1:072012508476:function:bitech-pipeline",
```

```
        "Payload": { "step": "events" }
      },
      "Next": "CrimeModule"
    },
    "CrimeModule": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName":
"arn:aws:lambda:eu-central-1:072012508476:function:bitech-pipeline",
        "Payload": { "step": "crime" }
      },
      "Next": "WeatherModule"
    },
    "WeatherModule": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName":
"arn:aws:lambda:eu-central-1:072012508476:function:bitech-pipeline",
        "Payload": { "step": "weather" }
      },
      "End": true
    }
  }
}
```

3.2 Lambda Implementation for Data Extraction

The compute layer is powered by a custom **AWS Lambda Docker Container** hosted on Amazon Elastic Container Registry (ECR).

- **Containerization & Headless Scraping**
    - Because the Crime module requires web scraping, the Docker image is built on the official AWS Python 3.12 base image and heavily modified to include system dependencies, Google Chrome for Testing, and ChromeDriver for headless Selenium execution.
- **Dockerfile Snippet:**

```
Shell


FROM public.ecr.aws/lambda/python:3.12


RUN dnf install -y atk cups-libs gtk3 libXcomposite alsa-lib \

    libXcursor libXdamage libXext libXi libXrandr libXScrnSaver \

    libXtst pango at-spi2-atk libXt xorg-x11-server-Xvfb \

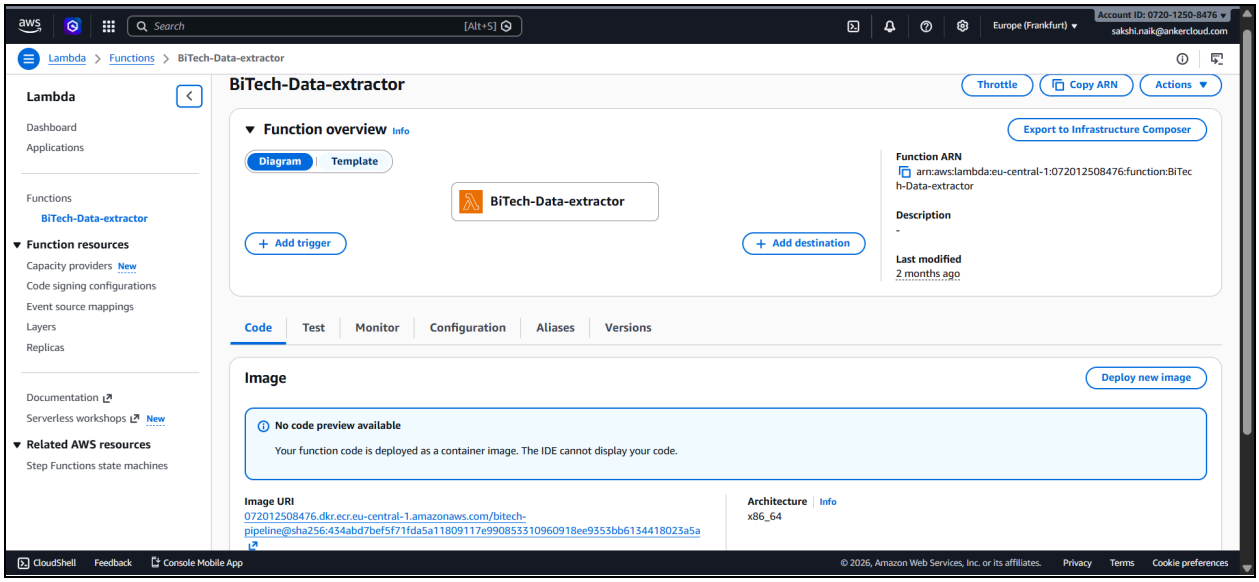
    xorg-x11-xauth dbus-glib nss mesa-libgbm jq unzip && \

    dnf clean all


COPY . ${LAMBDA_TASK_ROOT}

RUN pip install --no-cache-dir -r requirements.txt

CMD [ "run_pipeline.handler" ]
```



- **Payload Routing (run_pipeline.py)**
    - The entry point for the Lambda container is `run_pipeline.py`. This script acts as a router.
    - It ingests the `step` parameter from the Step Function payload and executes the corresponding module dynamically using `subprocess.run()`.
    - If a sub-process fails, the router explicitly raises an exception so the Step Function accurately registers the failure.
- **Router Implementation Code (main snippet):**

Python

```
def handler(event, context):
    """
    AWS Lambda entry point for Step Functions.
    Expects payload format: {"step": "traffic"}
    """
    setup_directories()
    step = event.get('step', 'all').lower()
    log(f"=== STEP FUNCTION TRIGGERED: {step.upper()} ===")
```

```
    try:
        if step == 'traffic':
            run_module("Traffic_Module", "main.py")
        elif step == 'events':
            run_module("Events_Module", "main.py")
        elif step == 'crime':
            run_module("Crime_Module", "main.py")
        elif step == 'weather':
            run_module("Weather_Module", "main.py")
        else:
            raise ValueError(f"Unknown step provided in payload: {step}")

        return {'statusCode': 200, 'body': f'{step.upper()} completed
successfully.'}

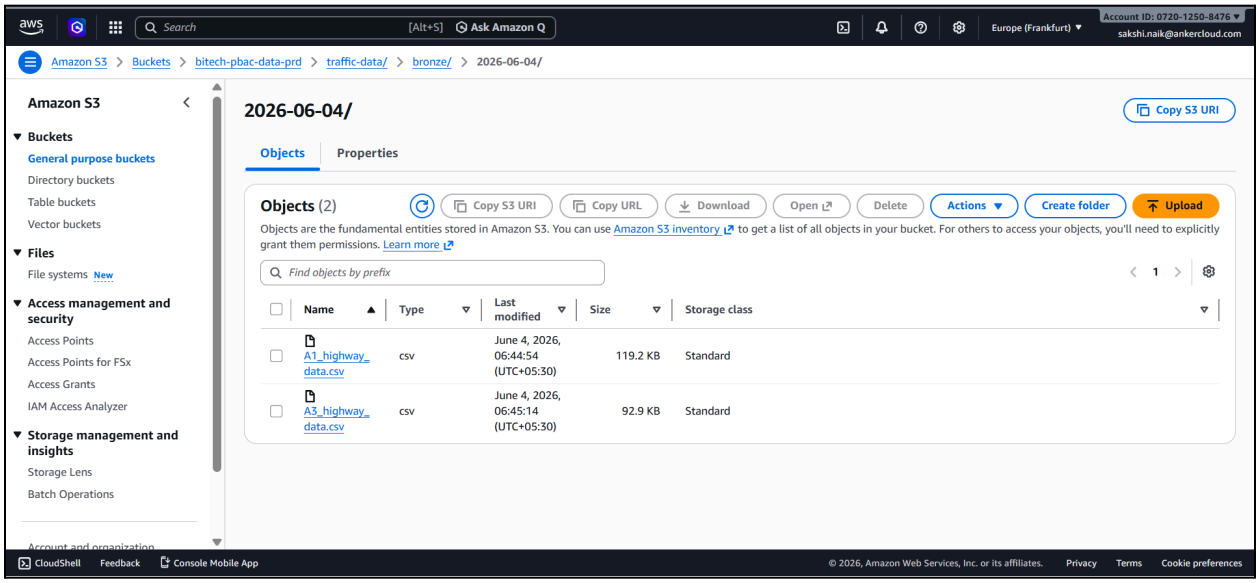
    except Exception as e:
        log(f"FATAL ERROR in {step}: {str(e)}")
        # CRITICAL: Raise the error to trigger the Step Function's "Retry" state
        raise Exception(f"Pipeline failed at step {step}: {str(e)}")
```

3.3 Data Landing in S3

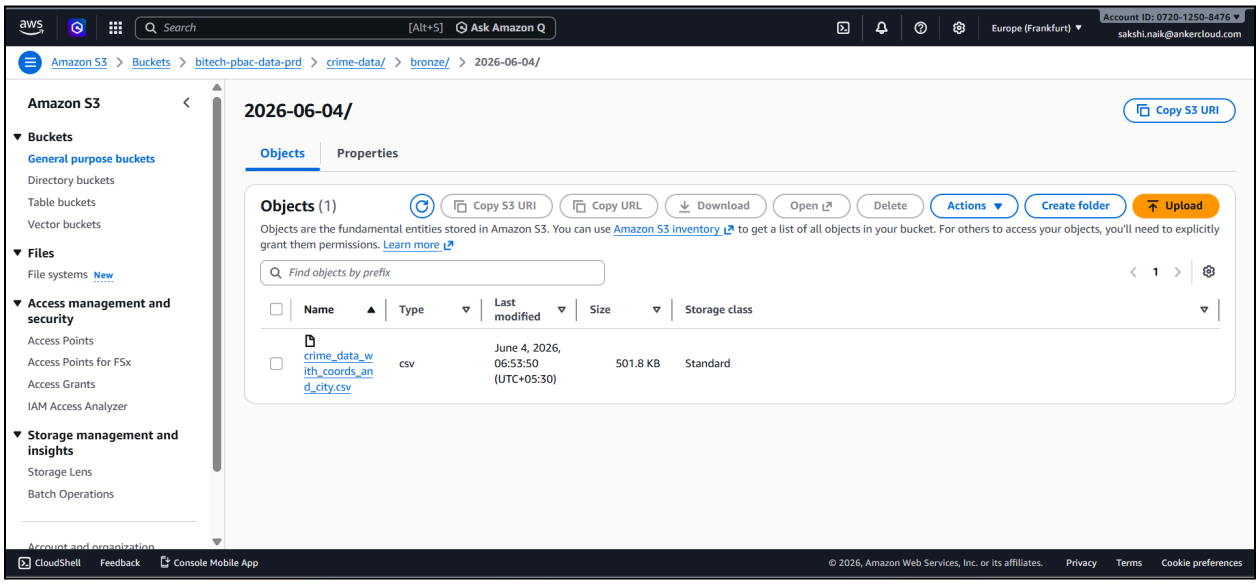
Extracted data is pushed directly into the `bitech-pbac-data-prd/bronze` S3 bucket. The data landing architecture separates raw/temporary data from analytics-ready data to maintain strict governance.

- **The Bronze Layer** (`bitech-pbac-data-prd/{module_name}/bronze/`)
    - The Bronze layer is logically separated by data domain, with each module producing specific, analytics-ready datasets:

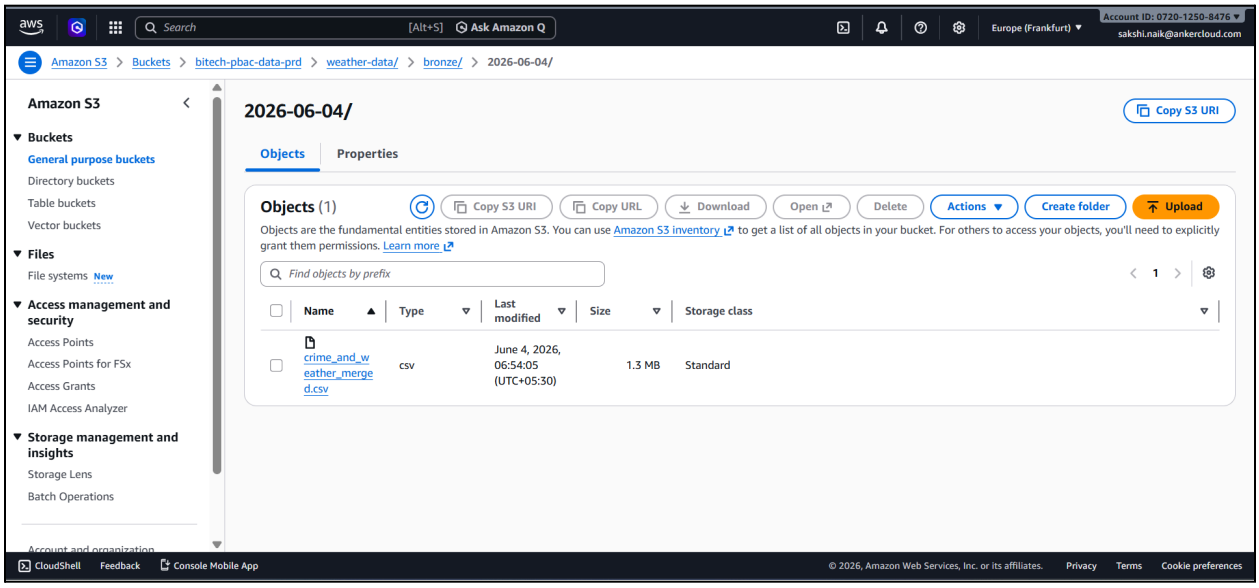
- Traffic Data (`/traffic-data/bronze/<PARTITION_DATE>/`)
  - `A1_highway_data.csv` & `A3_highway_data.csv`: Contains processed warning and roadworks data specific to major German Autobahns. The data includes exact geocoded coordinates, timestamps, and severity levels ready for traffic congestion analysis.



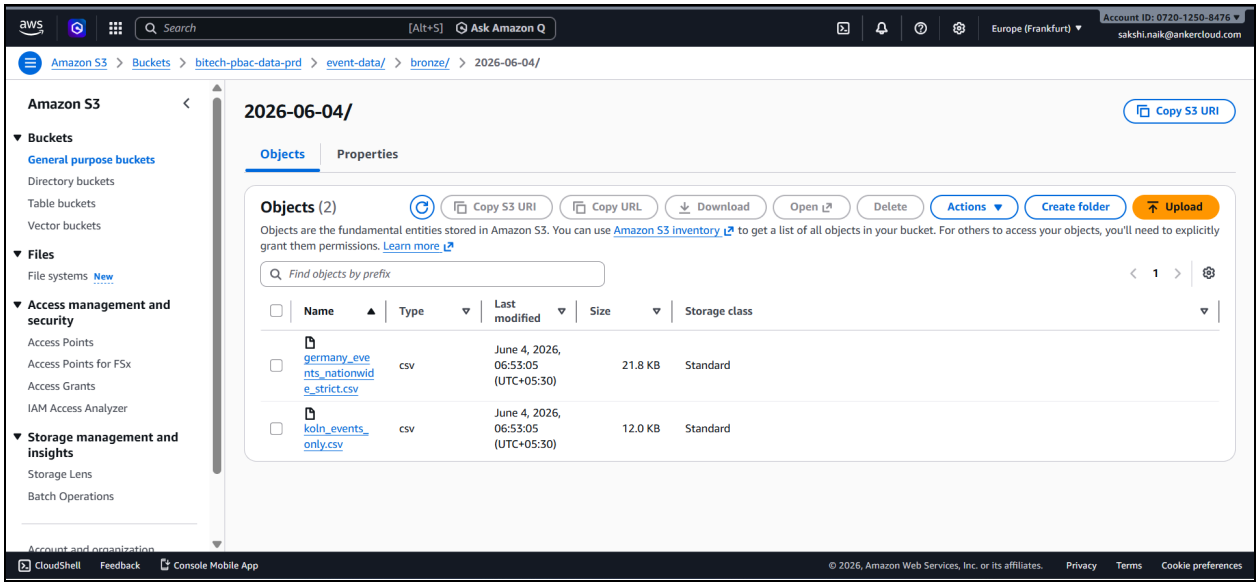
- Crime Data (`/crime-data/bronze/<PARTITION_DATE>/`)
  - `crime_data_with_coords_and_city.csv`: The finalized dataset containing records scraped from the Polizei NRW portal, enriched with precise latitude/longitude coordinates and standardized city names.



- Weather Data (`/weather-data/bronze/<PARTITION_DATE>/`)
  - `crime_and_weather_merged.csv`: Contains highly localized weather parameters (temperature, precipitation, storm warnings) fetched via Open-Meteo, merged directly with the relevant crime data for the identified active cities.



- Event Data (`/event-data/bronze/<PARTITION_DATE>/`)
  - `germany_events_nationwide_strict.csv`: Contains standardized, nationwide event details, including event names, venues, start/end dates, and crowd capacities.
  - `koln_events_only.csv`: Contains localized event data specifically filtered for Cologne (Köln) to support targeted regional analytics.



- Path Configuration

None

```
from datetime import datetime

PARTITION_DATE = datetime.now().strftime('%Y-%m-%d')


S3_CRIME_BRONZE = f"s3://bitech-pbac-data-prd/crime-data/bronze/{PARTITION_DATE}"

S3_TRAFFIC_BRONZE =
f"s3://bitech-pbac-data-prd/traffic-data/bronze/{PARTITION_DATE}"

S3_WEATHER_BRONZE =
f"s3://bitech-pbac-data-prd/weather-data/bronze/{PARTITION_DATE}"

S3_EVENTS_BRONZE = f"s3://bitech-pbac-data-prd/event-data/bronze/{PARTITION_DATE}"


# --- Crime Data ---

CRIME_OUTPUT_FILE = f"{S3_CRIME_BRONZE}/crime_data_with_coords_and_city.csv"


# --- Traffic Data ---

TRAFFIC_A1_OUTPUT = f"{S3_TRAFFIC_BRONZE}/A1_highway_data.csv"
```

```
TRAFFIC_A3_OUTPUT = f"{S3_TRAFFIC_BRONZE}/A3_highway_data.csv"


# --- Weather Data ---

WEATHER_OUTPUT_FILE = f"{S3_WEATHER_BRONZE}/crime_and_weather_merged.csv"


# --- Events Data ---

EVENTS_NATIONWIDE_OUTPUT =
f"{S3_EVENTS_BRONZE}/germany_events_nationwide_strict.csv"

EVENTS_KOLN_OUTPUT = f"{S3_EVENTS_BRONZE}/koln_events_only.csv"
```

- **The Helper Data Layer** (`bitech-pbac-data-prd/helper-data/`)
    - This layer acts as the cache, temporary staging area, and cross-module communication bridge. By decoupling raw extraction from final formatting, it keeps the production Bronze layer pristine. Crucially, it stores intermediate files and API memory banks to prevent the Lambda container from breaching its 15-minute execution timeout. The helper layer is logically separated by data domain:
    - Traffic Data (`/helper-data/traffic-data/`)
        - `traffic_geocode_cache.csv`: A persistent API memory bank for Nominatim reverse geocoding. Because the API strictly rate-limits requests to 1 per second, executing hundreds of coordinate lookups would force Lambda to timeout. This cache drops highway processing time from over 7 minutes to under 10 seconds.
        - `cologne_traffic_data_exploded.csv`: A temporary staging file used to flatten and explode complex, nested JSON payloads from the initial API fetch before the structured data is promoted to the Bronze layer.
    - Crime Data (`/helper-data/crime-data/`)
        - `crime_data_raw_with_plz.csv`: Acts as a temporary landing zone for raw data dumps generated by the headless Chrome (Selenium) scraper directly from the Polizei NRW portal.

        - `plz_coords_cache.csv`: A geocoding dictionary that stores previously resolved latitude and longitude coordinates for German postal codes, bypassing repetitive and slow API calls during subsequent pipeline runs.
    - Weather Data (`/helper-data/weather-data/`)
        - `postal_code_coordinates.csv`: A cross-module dependency file. Because the architecture is decoupled, the Crime module extracts unique locations from its run and drops this CSV into the helper bucket. The Weather module then reads this file to dynamically determine exactly which cities require weather forecasts.
        - `crime_weather_data_forecast.csv`: Holds the raw, unformatted payload fetched from the Open-Meteo API. It sits in the Helper layer as an intermediate stepping stone until the pipeline securely merges it with the crime dataset.
    - Path Configuration

```
None


S3_HELPER_BASE = "s3://bitech-pbac-data-prd/helper-data"


S3_CRIME_HELPER = f"{S3_HELPER_BASE}/crime-data"

S3_TRAFFIC_HELPER = f"{S3_HELPER_BASE}/traffic-data"

S3_WEATHER_HELPER = f"{S3_HELPER_BASE}/weather-data"


# --- Crime Data Helper Files ---

CRIME_RAW_STAGING = f"{S3_CRIME_HELPER}/crime_data_raw_with_plz.csv"

CRIME_GEOCODE_CACHE = f"{S3_CRIME_HELPER}/plz_coords_cache.csv"


# --- Traffic Data Helper Files ---

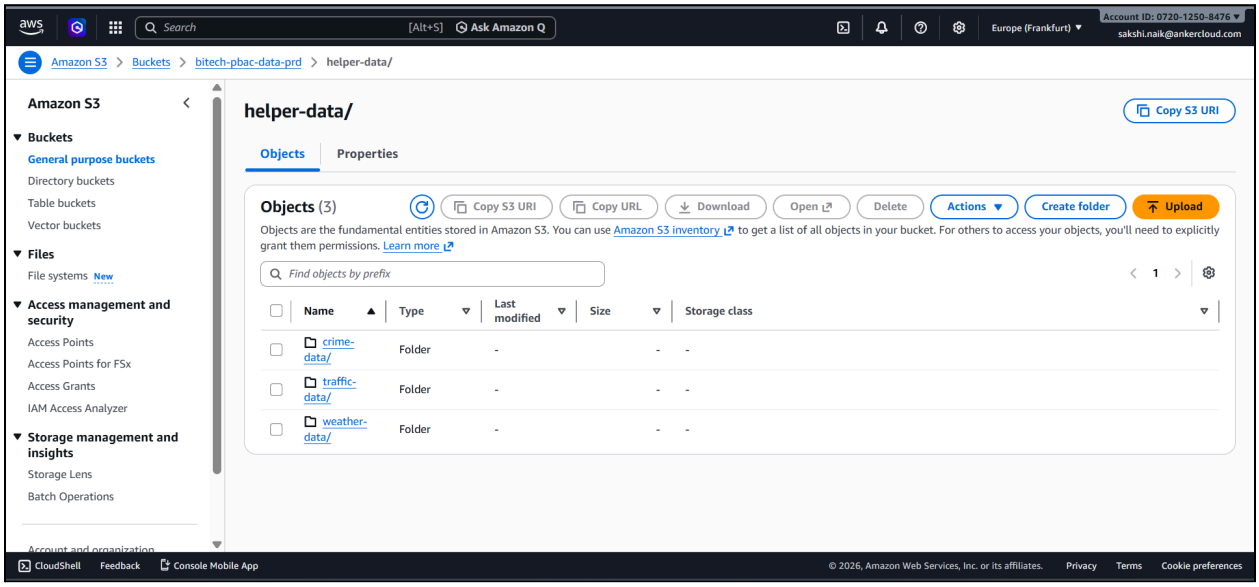
TRAFFIC_GEOCODE_CACHE = f"{S3_TRAFFIC_HELPER}/traffic_geocode_cache.csv"

TRAFFIC_COLOGNE_STAGING = f"{S3_TRAFFIC_HELPER}/cologne_traffic_data_exploded.csv"
```

```
# --- Weather Data Helper Files ---

SHARED_LOCATION_DEPENDENCY = f"{S3_WEATHER_HELPER}/postal_code_coordinates.csv"

WEATHER_RAW_STAGING = f"{S3_WEATHER_HELPER}/crime_weather_data_forecast.csv"
```

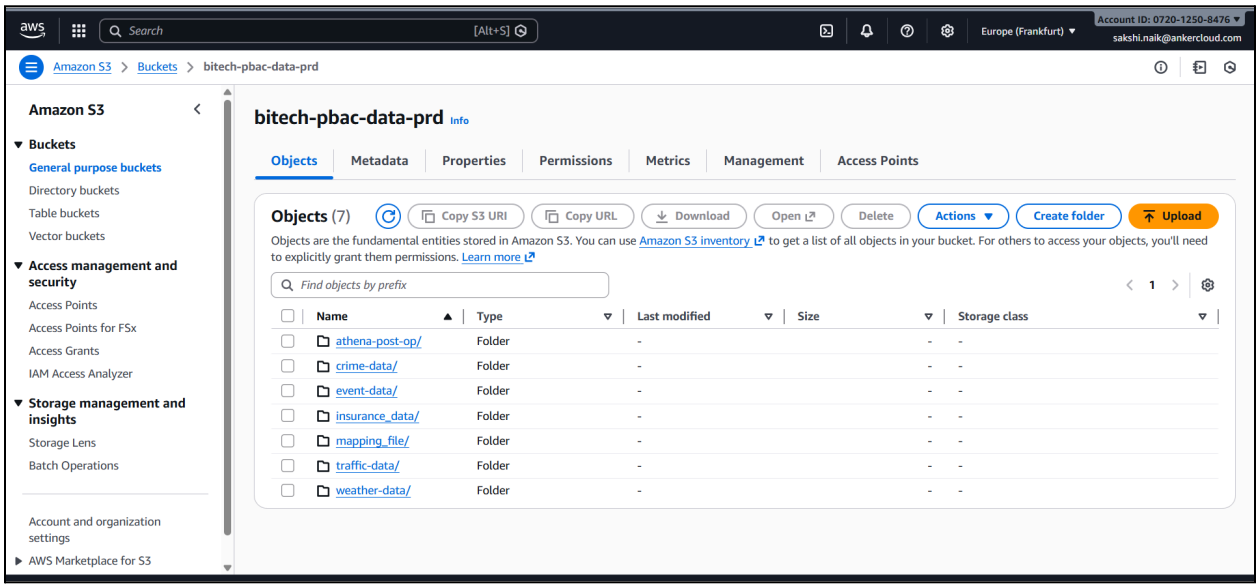


4. Data Lake Architecture

4.1 Data Sources

Data Source	Description
Traffic Data	Road closures, congestion, incidents
Crime Data	Disturbances, safety alerts
Events Data	Concerts, football matches, crowd movement
Weather Data	Rainfall, temperature, storms
Insurance Data	Policy and claims data for insurance analytics.

4.2 S3 Medallion Structure

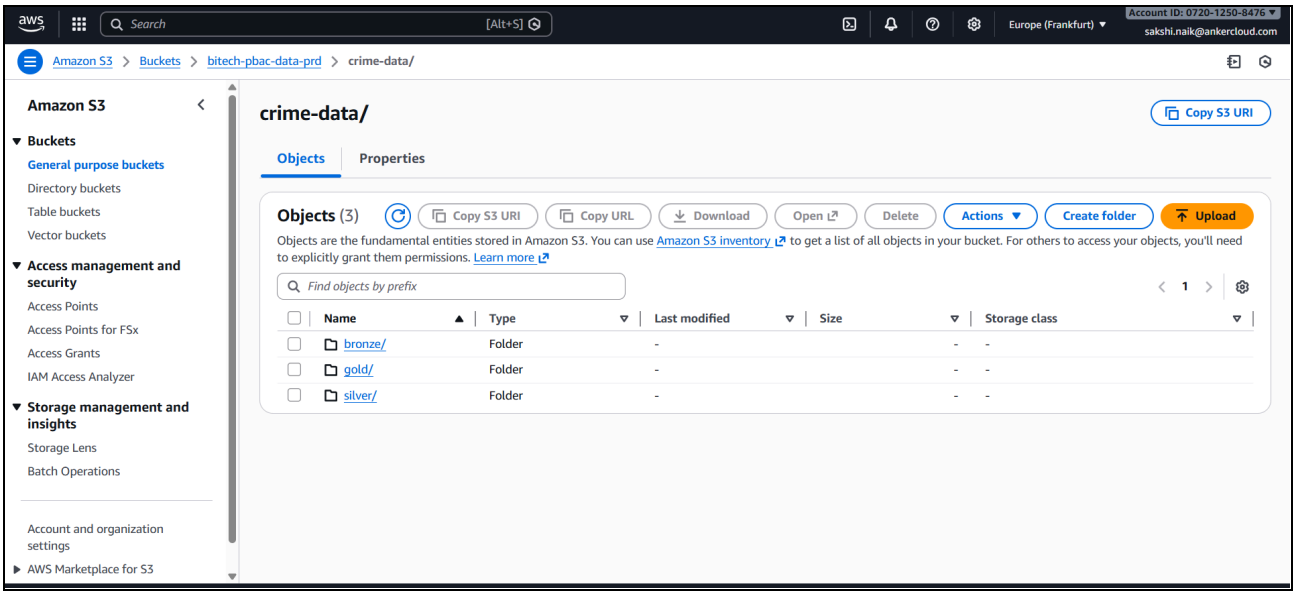


None

```
s3://bitech-pbac-prd/

  ├── traffic-data/
  │   ├── bronze/
  │   ├── silver/
  │   └── gold/
  ├── crime-data/
  │   ├── bronze/
  │   ├── silver/
  │   └── gold/
  ├── events-data/
  │   ├── bronze/
  │   ├── silver/
  │   └── gold/
  └── weather-data/
      ├── bronze/
      ├── silver/
      └── gold/


So on for Insurance Data
```



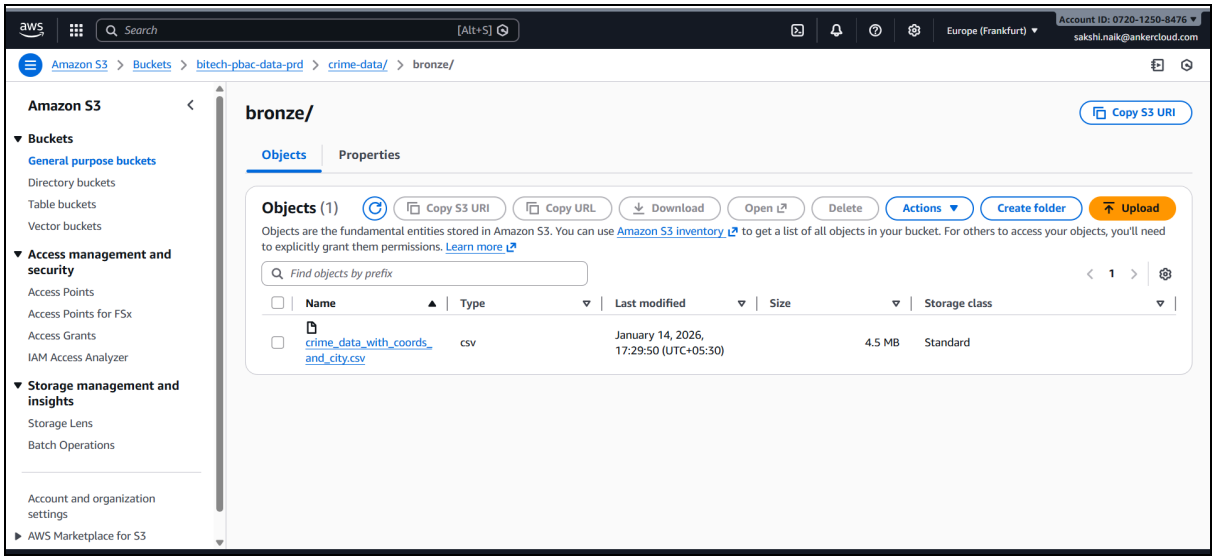
4.3 Medallion Layer Responsibilities

The Bitech Data Lake is designed using a **medallion architecture (Bronze, Silver, Gold)** to ensure clear separation of ingestion, transformation, governance, and consumption responsibilities.

Bronze Layer

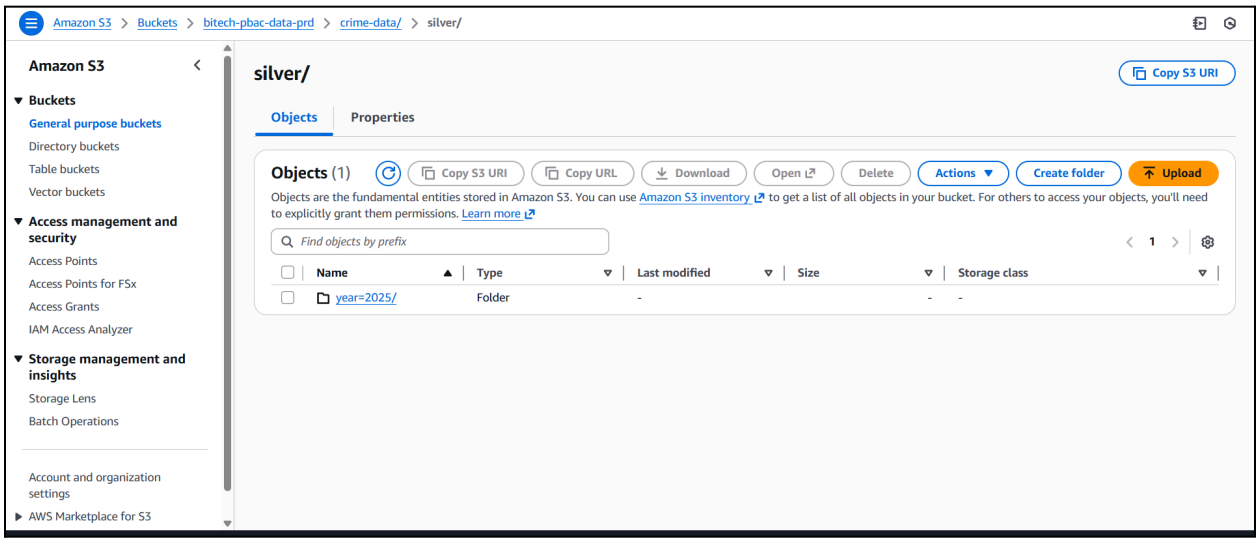
The Bronze layer is the raw ingestion layer of the Bitech Data Lake, where data from all source systems is ingested and stored exactly as it arrives, without applying any transformation or cleansing. Once the data lands in the S3 Bronze layer, it preserves complete data granularity, ensuring that no detail from the source is lost.

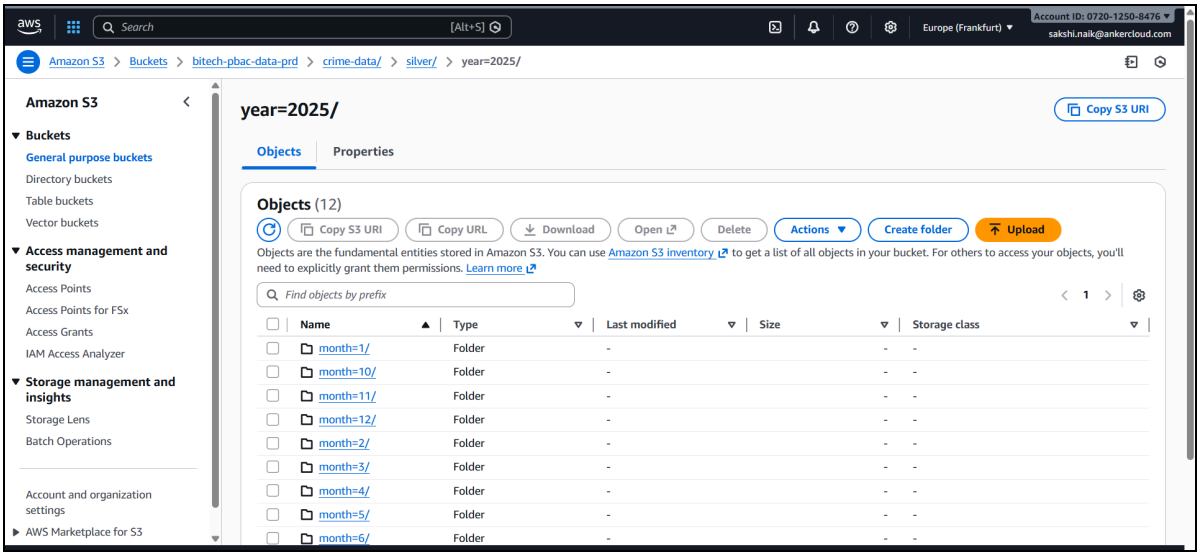
This layer serves as the foundational system of record for the entire data platform, supporting critical use cases such as reprocessing, historical backfills, and end-to-end data lineage tracking. Because it contains raw, unprocessed, and potentially sensitive attributes, access is strictly limited to internal data engineering pipelines and operational processes only, and it is not exposed to end users or analytical workloads to ensure data security and governance compliance.

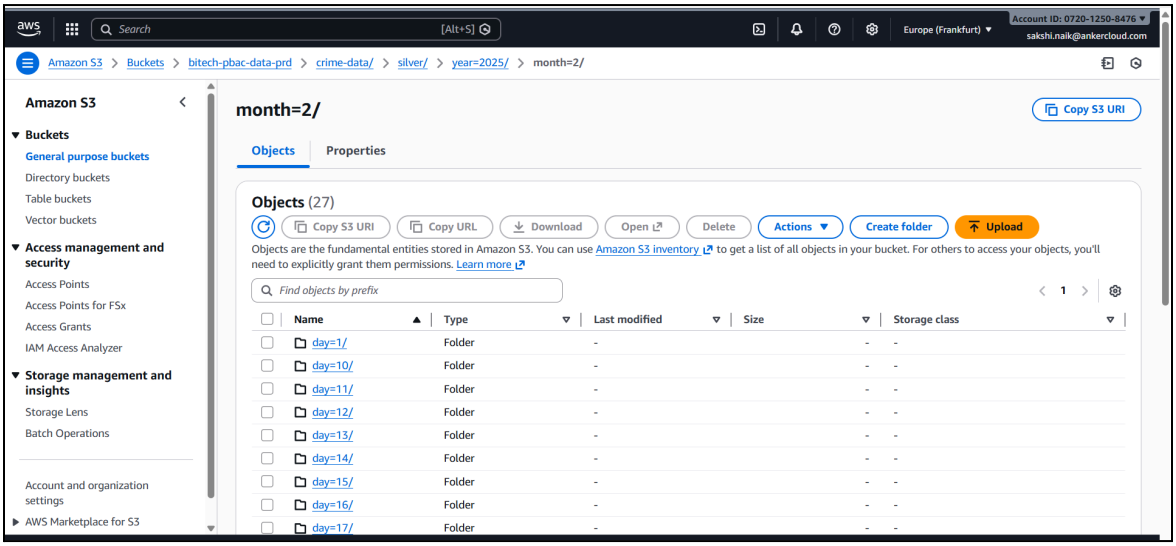


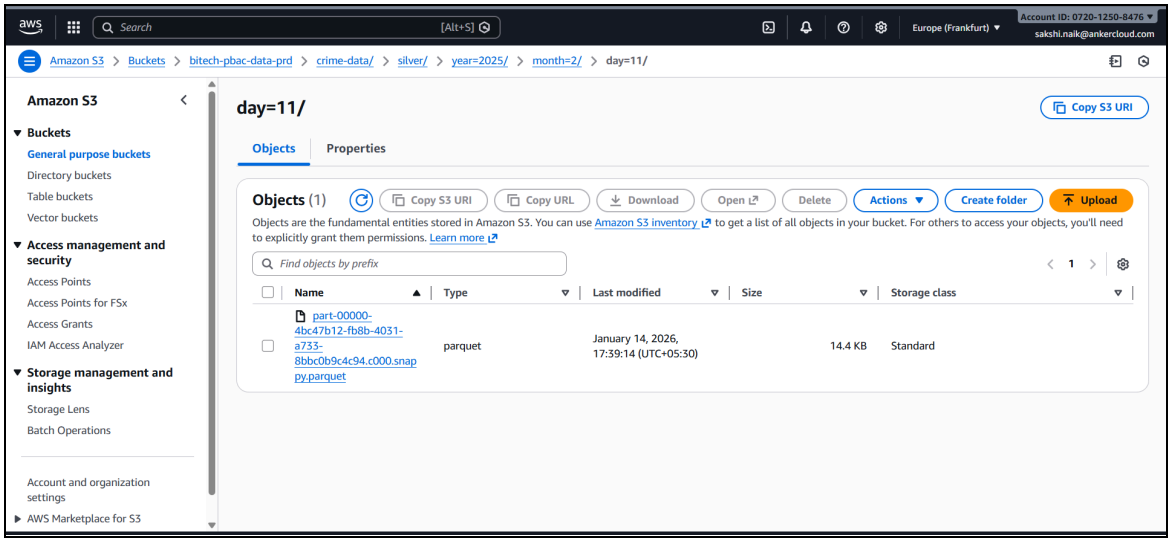
Silver Layer

The Silver layer is the curated and standardized layer derived from the Bronze data, designed for analytics and controlled access. Data is cleaned, enriched, and conformed to consistent schemas, with quality rules applied and datasets optimized using date based partitions. All Silver datasets are registered in the Lake Formation Data Catalog and governed using LF-Tags, making this layer the primary enforcement point for PBAC across Athena queries and API based access.




Search [Alt+S]
Europe (Frankfurt)
Account ID: 0720-1250-8476
sakshi.naik@ankercloud.com
Amazon S3 > Buckets > bitech-pbac-data-prd > crime-data/ > silver/ > year=2025/
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Account and organization settings
AWS Marketplace for S3
year=2025/
Copy S3 URI
Objects
Properties
Objects (12)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
month=1/ Folder
month=10/ Folder
month=11/ Folder
month=12/ Folder
month=2/ Folder
month=3/ Folder
month=4/ Folder
month=5/ Folder
month=6/ Folder


Search [Alt+S]
Europe (Frankfurt)
Account ID: 0720-1250-8476
sakshi.naik@ankercloud.com
Amazon S3 > Buckets > bitech-pbac-data-prd > crime-data/ > silver/ > year=2025/ > month=2/
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Account and organization settings
AWS Marketplace for S3
month=2/
Copy S3 URI
Objects
Properties
Objects (27)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
day=1/ Folder
day=10/ Folder
day=11/ Folder
day=12/ Folder
day=13/ Folder
day=14/ Folder
day=15/ Folder
day=16/ Folder
day=17/ Folder



Gold Layer

The Gold layer contains aggregated and business friendly datasets derived from the Silver layer. These datasets are optimized for reporting, dashboards, and cross-domain analysis. Typical use cases include trend analysis, operational metrics, and summary level insights.

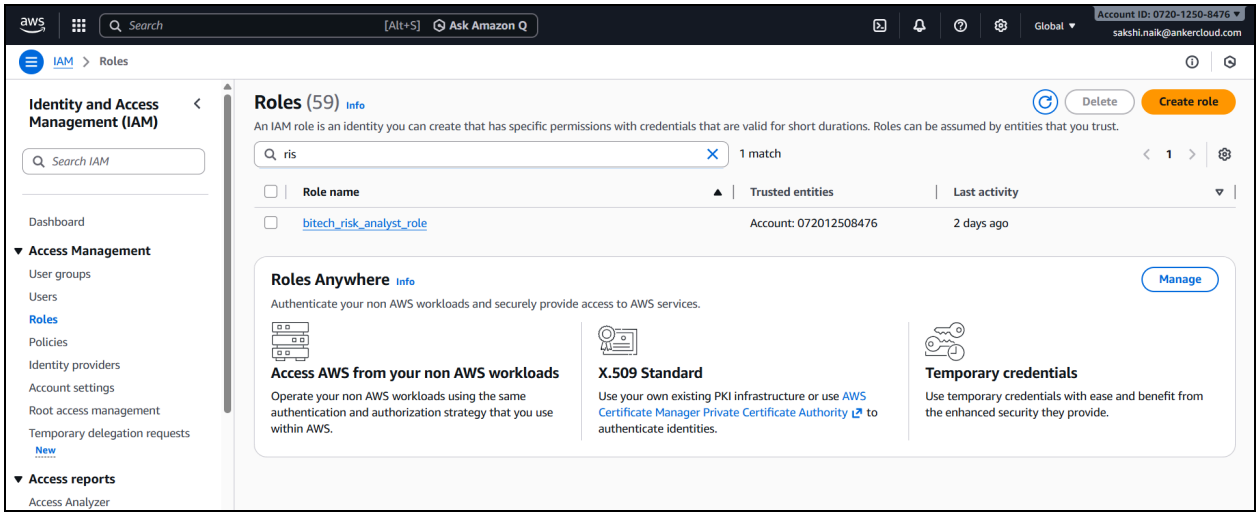
Note : In the current scope of the project, only the **Bronze and Silver layers** are implemented. The Gold layer is not yet part of the live platform but is included in the architecture for future extensibility.

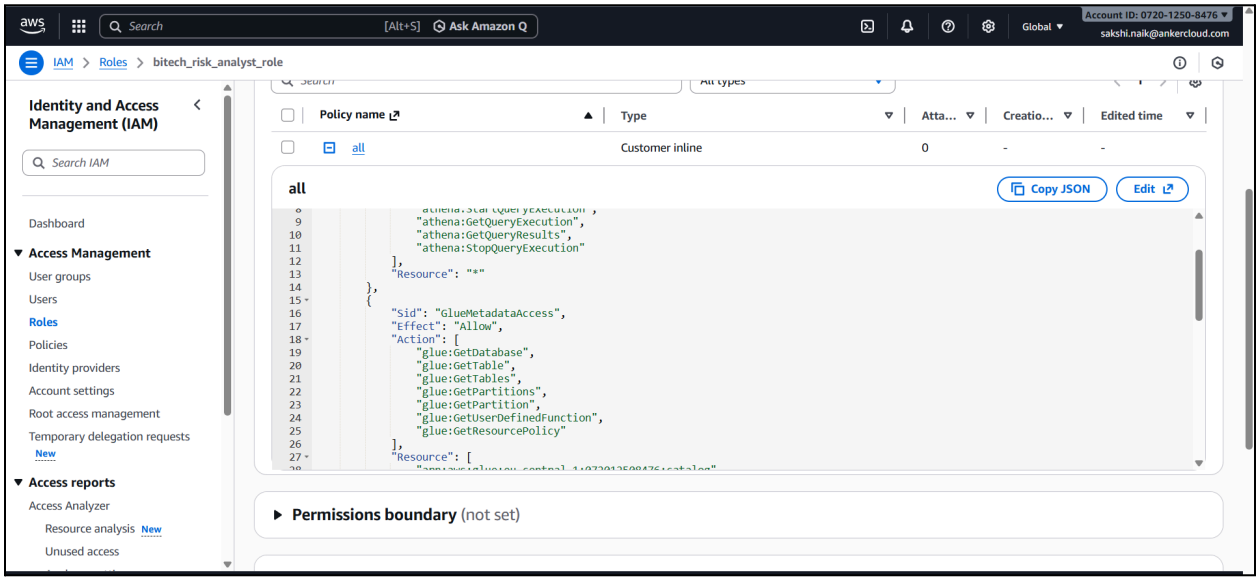
5. IAM Role and Design

In the Bitech PBAC system, AWS Identity and Access Management (IAM) plays a central role in controlling who can access which data and through which service.

For this project, we designed purpose specific IAM roles aligned with the principle of least privilege, ensuring users and services only have access to what they need. We created two main PBAC roles for end user access that are **bitech_risk_analyst_role** and **bitech_marketing_role**.

The Risk Analyst role provides broad access for deep analytics, investigations, and operational intelligence, including the ability to start Athena queries, retrieve results, and access Glue metadata across all project databases, including insurance, traffic, weather, crime, and event datasets.





JSON

bitech_risk_analyst_role - INLINE POLICY

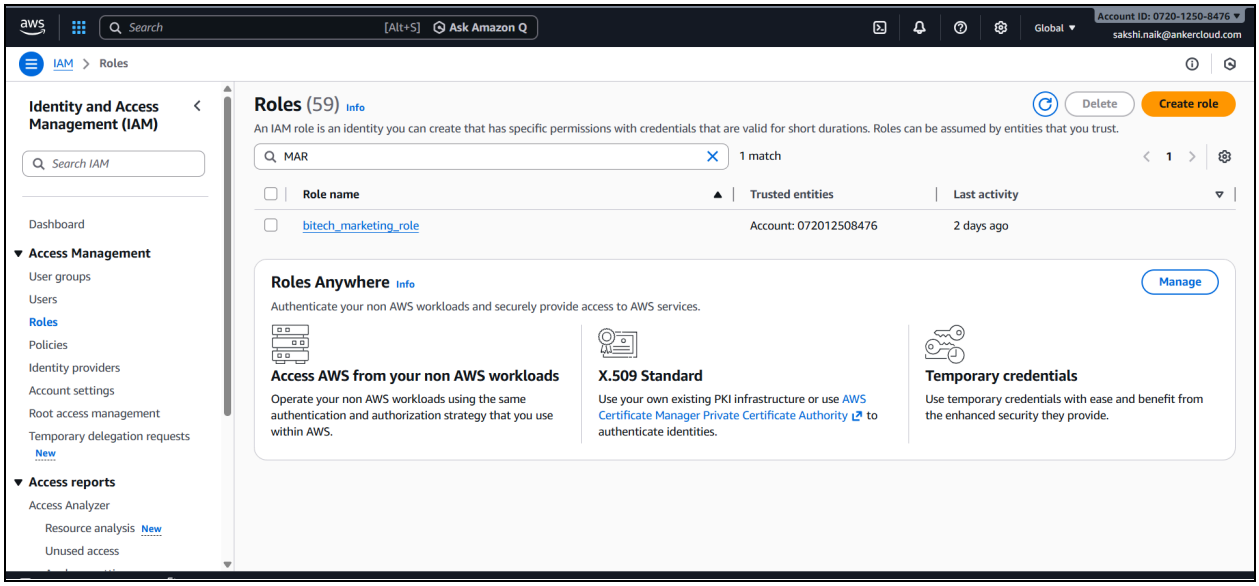
```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AthenaActions",
            "Effect": "Allow",
            "Action": [
                "athena:StartQueryExecution",
```

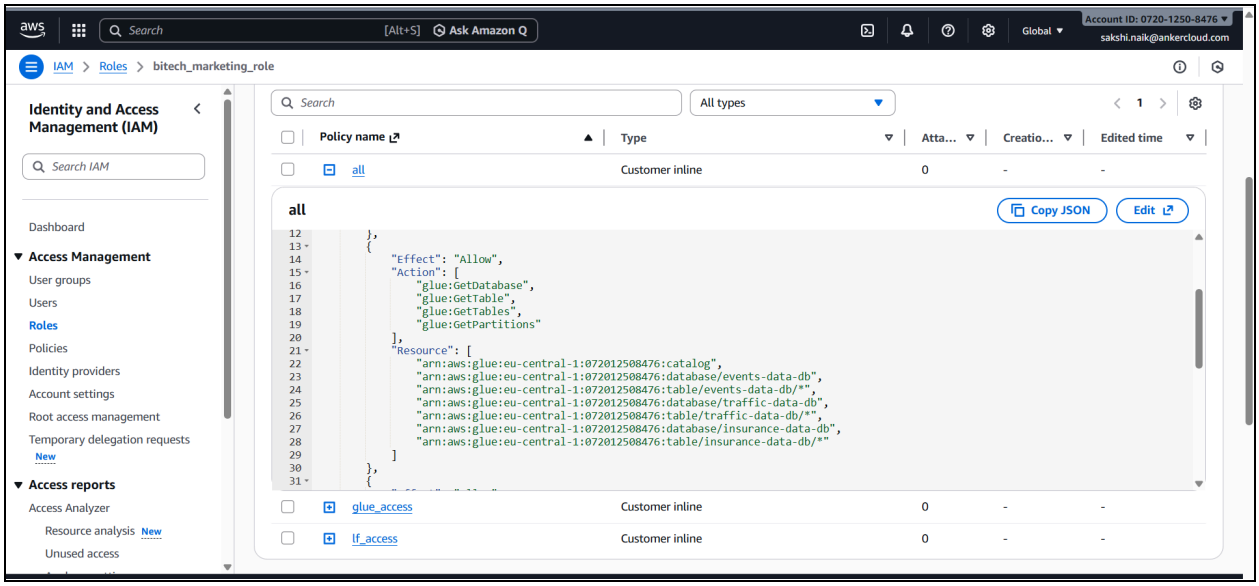
```
                "athena:GetQueryExecution",
                "athena:GetQueryResults",
                "athena:StopQueryExecution"
            ],
            "Resource": "*"
        },
        {
            "Sid": "GlueMetadataAccess",
            "Effect": "Allow",
            "Action": [
                "glue:GetDatabase",
                "glue:GetTable",
                "glue:GetTables",
                "glue:GetPartitions",
                "glue:GetPartition",
                "glue:GetUserDefinedFunction",
                "glue:GetResourcePolicy"
            ],
            "Resource": [
                "arn:aws:glue:eu-central-1:072012508476:catalog",

"arn:aws:glue:eu-central-1:072012508476:database/events-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/events-data-db/*",
                "arn:aws:glue:eu-central-1:072012508476:database/weather-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/weather-data-db/*",
                "arn:aws:glue:eu-central-1:072012508476:database/traffic-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/traffic-data-db/*",
                "arn:aws:glue:eu-central-1:072012508476:database/crime-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/crime-data-db/*",

"arn:aws:glue:eu-central-1:072012508476:database/insurance-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/insurance-data-db/*"
            ]
        },
        {
            "Sid": "S3DataReadAndOutputWrite",
            "Effect": "Allow",
            "Action": [
                "s3:GetBucketLocation",
                "s3:ListBucket",
                "s3:GetObject",
                "s3:PutObject"
            ],
            "Resource": [
                "arn:aws:s3:::bitech-pbac-data-prd",
```

```
                "arn:aws:s3:::bitech-pbac-data-prd/*",
                "arn:aws:s3:::bitech-pbac-data-prd"
            ]
        },
        {
            "Sid": "LakeFormationAuth",
            "Effect": "Allow",
            "Action": "lakeformation:GetDataAccess",
            "Resource": "*"
        }
    ]
}
```





JSON

bitech_marketing_role - Inline Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "athena:StartQueryExecution",
                "athena:GetQueryExecution",
                "athena:GetQueryResults"
            ],
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "glue:GetDatabase",
                "glue:GetTable",
                "glue:GetTables",
                "glue:GetPartitions"
            ],
            "Resource": [
```

```
                "arn:aws:glue:eu-central-1:072012508476:catalog",
                "arn:aws:glue:eu-central-1:072012508476:database/events-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/events-data-db/*",
                "arn:aws:glue:eu-central-1:072012508476:database/traffic-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/traffic-data-db/*",

"arn:aws:glue:eu-central-1:072012508476:database/insurance-data-db",
                "arn:aws:glue:eu-central-1:072012508476:table/insurance-data-db/*"
            ]
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetBucketLocation",
                "s3:ListBucket",
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::bitech-pbac-data-prd",
                "arn:aws:s3:::bitech-pbac-data-prd/*"
            ]
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutObject"
            ],
            "Resource": [
                "arn:aws:s3:::bitech-pbac-data-prd/athena-post-op/*"
            ]
        },
        {
            "Effect": "Allow",
            "Action": "lakeformation:GetDataAccess",
            "Resource": "*"
        }
    ]
}
```

Both roles are mapped to **Lake Formation (LF) tags** to enforce fine grained, column level access control.

For instance, the Risk Analyst role corresponds to the LF tag **AccessRiskAnalytics=Analyst**, while the Marketing role corresponds to **AccessMarketing=Marketer**.

This mapping ensures that data access is mediated by Lake Formation, so users never interact directly with S3 or Athena; instead, they access data through controlled entry points like Lambda and API Gateway.

Supporting these PBAC roles, we also created service-specific IAM roles. The **bitech_lambda_role** is assumed by Lambda functions to orchestrate Athena queries and S3 operations. It includes permissions to assume the PBAC roles dynamically via STS, execute Athena queries, access S3 data, interact with Glue metadata, and write logs to CloudWatch.

JSON

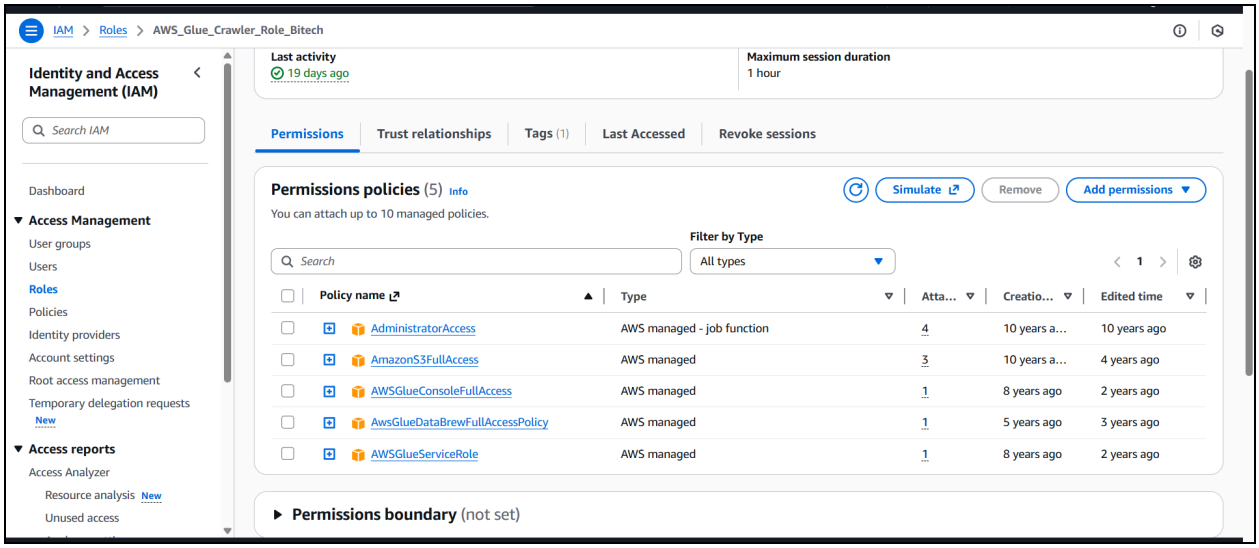
bitech_lambda_role - INLINE POLICY

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowSTSAssumeRole",
            "Effect": "Allow",
            "Action": "sts:AssumeRole",
            "Resource": [
                "arn:aws:iam::072012508476:role/bitech_risk_analyst_role",
                "arn:aws:iam::072012508476:role/bitech_marketing_role"
            ]
        },
        {
            "Sid": "AllowAthenaExecution",
            "Effect": "Allow",
            "Action": [
                "athena:StartQueryExecution",
                "athena:GetQueryExecution",
                "athena:GetQueryResults",
                "athena:StopQueryExecution"
            ],
            "Resource": [
                "arn:aws:athena:eu-central-1:072012508476:workgroup/primary",

"arn:aws:athena:eu-central-1:072012508476:workgroup/marketing_workgroup-lf",
                "arn:aws:athena:eu-central-1:072012508476:query_execution/*"
            ]
        },
        {
            "Sid": "AllowS3Access",
            "Effect": "Allow",
            "Action": [
```

```
                "s3:PutObject",
                "s3:GetObject",
                "s3:ListBucket",
                "s3:AbortMultipartUpload"
            ],
            "Resource": [
                "arn:aws:s3:::bitech-pbac-data-prd",
                "arn:aws:s3:::bitech-pbac-data-prd/*"
            ]
        },
        {
            "Sid": "AllowGlueMetadataAccess",
            "Effect": "Allow",
            "Action": [
                "glue:GetDatabase",
                "glue:GetTable",
                "glue:GetPartitions"
            ],
            "Resource": "*"
        },
        {
            "Sid": "AllowCloudWatchLogging",
            "Effect": "Allow",
            "Action": [
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": "arn:aws:logs:*:*:*"
        }
    ]
}
```

Similarly, the **AWS Glue crawler role (AWS_Glue_Crawler_Role_Bitech)** is configured to allow Glue to scan S3 data, infer schemas, and populate the Lake Formation catalog while respecting LF tag-based access policies.



Overall, this IAM and role design ensures a **secure, centralized, and auditable access control model**. Users never access raw S3 data directly; all operations are mediated via Lambda and API Gateway, with Lake Formation enforcing authorization rules. This combination of PBAC roles, LF tag mapping, and service-specific roles provides a robust, scalable framework for controlled, purpose-driven access to the Bitech data lake.

Note: In this project, only the Risk Analyst and Marketing roles are actively used for PBAC, while other roles like crawler and Lambda roles are strictly service focused.

6. Glue Data Catalog & Metadata Management

The AWS Glue Data Catalog acts as the **central metadata layer** for the Bitech PBAC data lake. All datasets stored in Amazon S3 across Bronze and Silver layers are registered as tables in the Glue Data Catalog. This enables schema discovery, query-based access through Athena, and governance enforcement via AWS Lake Formation.

The Data Catalog is populated and maintained using a combination of **Glue ETL jobs** and **Glue Crawlers**, ensuring that metadata always reflects the current state of data in S3.

6.1 Glue ETL Jobs (Bronze → Silver Processing)

Glue ETL jobs are used to transform raw ingested data into analytics ready datasets. For each data source (traffic, crime, events, weather, and insurance), a dedicated ETL job processes data from the Bronze layer and writes it to the Silver layer.

Key responsibilities of Glue ETL jobs in this project include:

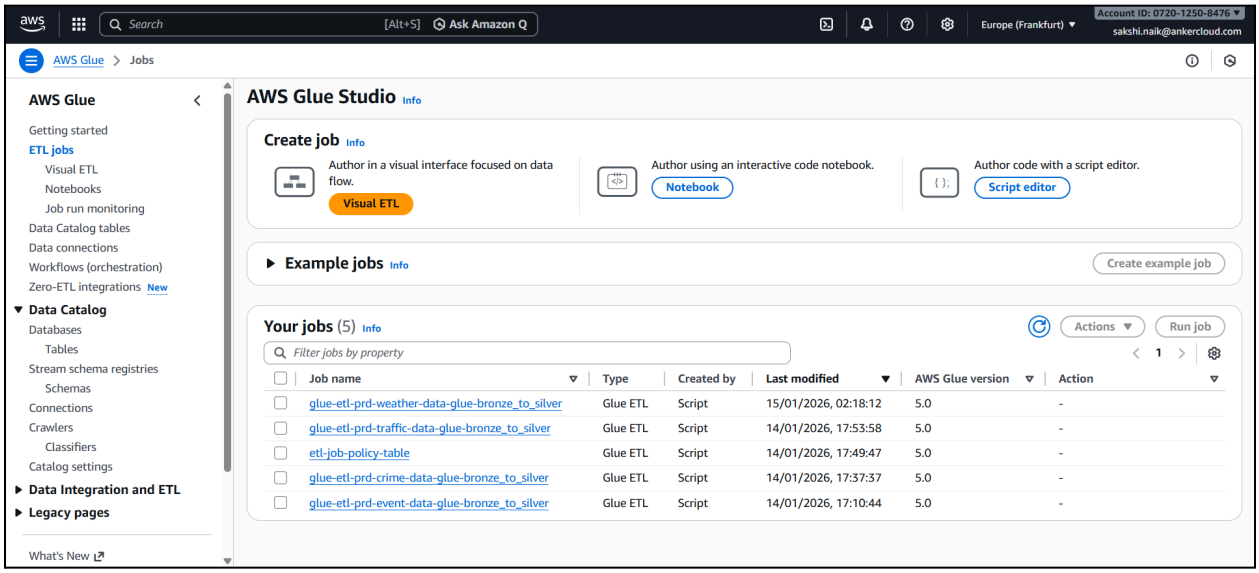
- Applying consistent schemas to raw data

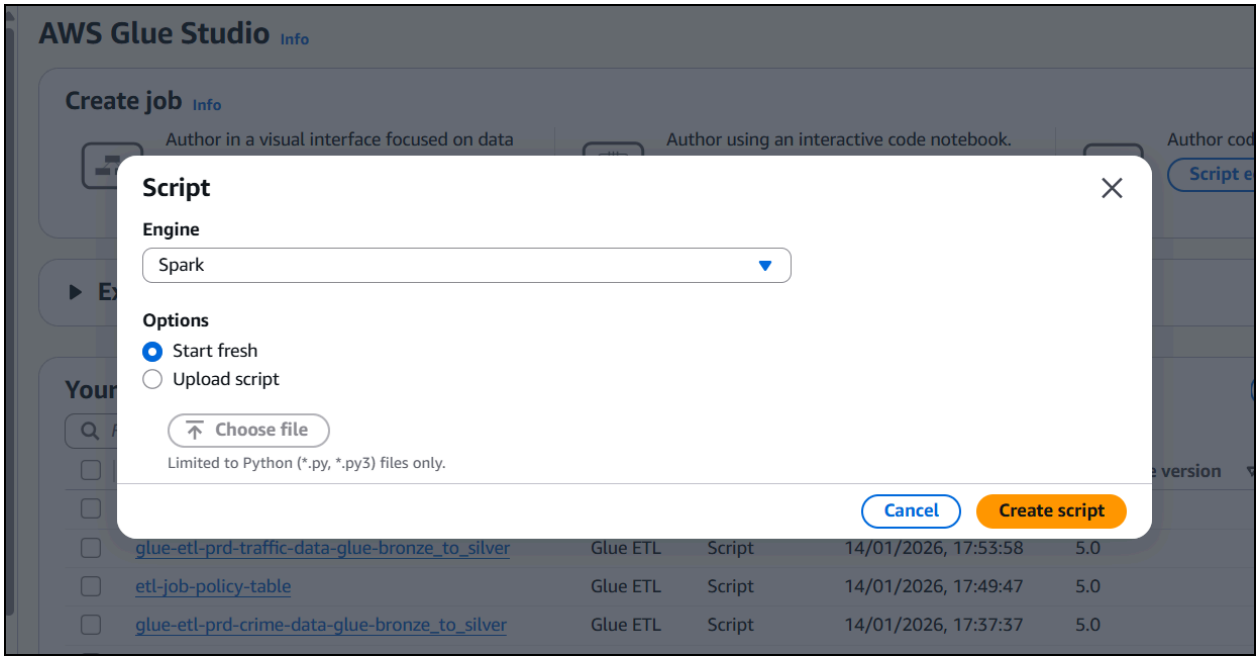
- Data cleansing and normalization
- Partitioning data using best practices (e.g., date-based partitions)
- Preparing structured outputs suitable for Athena queries

Once the ETL job completes, the transformed data is written to a well defined S3 location aligned with the medallion architecture.

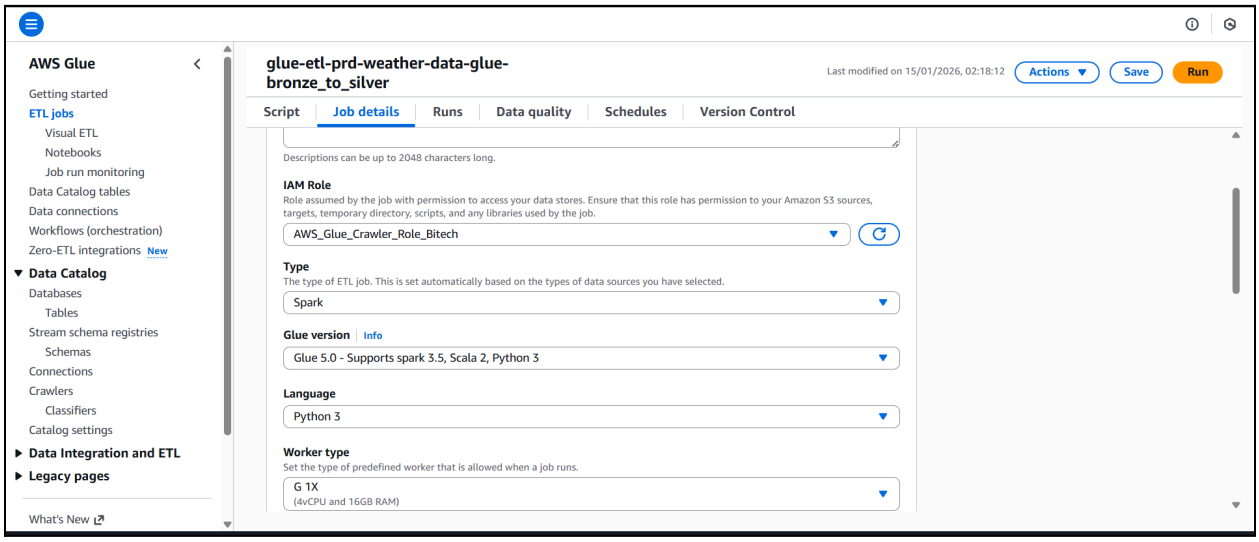
Steps to Create Glue ETL Job

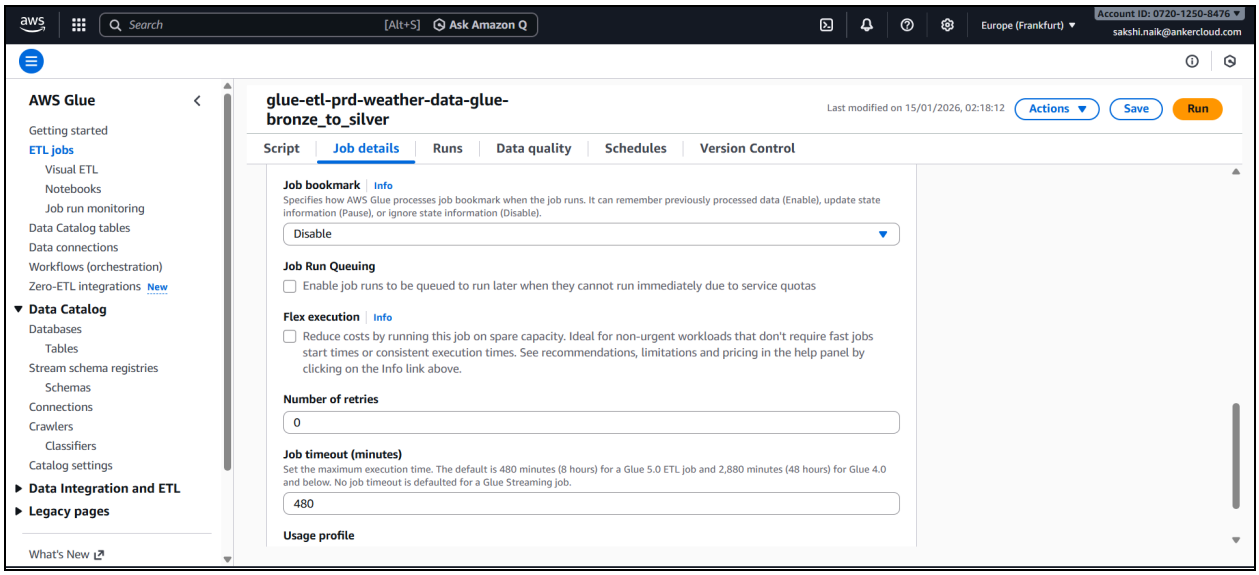
- Click on Script Editor
- Then select → Pyspark → Create Script





- Refer the below image and fill the Job details
- Remember to create a Role for AWS Glue with necessary policies attached.





- Refer the code below
- Similarly for all other sources that is crime, events, traffic and insurance data.

Glue ETL Job Code : glue-etl-prd-weather-data-glue-bronze_to_silver

This Glue job is used to transform raw weather data from the **Bronze layer into the Silver layer** in the Bitech data lake. The job reads CSV files recursively from the Bronze S3 location, allowing it to handle incoming raw data without assuming a fixed folder structure. After loading, the data is converted into a Spark DataFrame to apply structured transformations and validation rules required for analytics-ready datasets.

As part of Silver layer processing, the job standardizes time handling by converting the raw date field into a proper timestamp and deriving **year, month,** and **day** columns for partitioning. Records without valid timestamps are removed to enforce basic data quality. The cleansed and enriched data is then written back to S3 in **Parquet format**, partitioned by date, using a **full refresh overwrite strategy**. This ensures consistent, performant querying through Athena and controlled access via AWS Lake Formation.

Glue ETL Job Code : glue-etl-prd-traffic-data-glue-bronze_to_silver

This Glue job transforms raw traffic data from the Bronze layer into a structured Silver dataset within the Bitech data lake. The job reads CSV files recursively from S3, standardizes column names, and aligns the schema to ensure consistency across datasets. Basic validations are applied to confirm the presence of input data before processing continues.

A key responsibility of this job is robust timestamp construction. It combines separate date and time fields into a single event_timestamp, ensuring accurate time-based analytics. Records with invalid timestamps or partition values are filtered out to maintain Silver-layer data quality. The final dataset is written to S3 in Parquet format, partitioned by year, month, and day, using a full overwrite strategy to keep the Silver layer clean, consistent, and optimized for Athena queries and Lake Formation–based governance.

Glue ETL Job Code : glue-etl-prd-crime-data-glue-bronze_to_silver

This Glue job processes raw crime data from the Bronze layer and prepares it for analytical use in the Silver layer of the Bitech data lake. The job reads CSV files recursively from S3 using Glue DynamicFrames, ensuring that newly arriving files are picked up automatically. Column names are standardized during transformation to maintain a consistent Silver-layer schema across datasets.

A key feature of this job is custom date-time handling, where German-formatted date strings are parsed and converted into a proper timestamp column. Based on this timestamp, year, month, and day partition columns are derived to support efficient Athena queries. The data is written to S3 in Parquet format, using dynamic partition overwrite to safely refresh only affected partitions when the job is re-run, preventing duplicates while preserving existing clean data. Extensive logging ensures visibility into each stage of the transformation and write process.

Glue ETL Job Code : glue-etl-prd-event-data-glue-bronze_to_silver

This Glue job processes nationwide event data from the Bronze layer and prepares a clean, standardized Silver dataset for analytics in the Bitech data lake. The job reads event data from S3, normalizes column names, and safely parses event start and end dates into proper timestamp fields. Records with invalid or missing start dates are filtered out to ensure only reliable event data progresses to the Silver layer.

To support efficient querying and governance, the job derives year, month, and day partition columns from the event start date and handles missing geographic coordinates by applying default values. Duplicate events are removed using a business-relevant key, and a clean final schema is enforced before writing the data in Parquet format, partitioned by date. The Silver dataset is written using an overwrite strategy, ensuring consistency and optimal performance for Athena queries and Lake Formation–based access control.

Glue ETL Job Code : etl-job-policy-table

This Glue job transforms raw insurance data from the Bronze layer into a standardized Silver dataset within the Bitech data lake. The job reads CSV files recursively from S3, allowing it to process multiple incoming files without manual intervention. As part of schema standardization,

key columns such as customer names and city information are renamed to follow consistent naming conventions across the data lake.

To improve data usability and query consistency, the job normalizes German city names by removing special characters and enforcing lowercase formatting. Partition columns (year, month, and day) are derived from the policy inception_date, ensuring time-based partitioning aligned with analytical access patterns. Records with invalid or missing dates are filtered out, and the final dataset is written to S3 in Parquet format, partitioned by date, using an overwrite strategy to maintain a clean and query-efficient Silver layer governed by AWS Lake Formation.

6.2 Glue Crawlers (Schema Inference & Catalog Registration)

Glue Crawlers are responsible for **discovering schemas** and **registering tables** in the Glue Data Catalog based on the output produced by ETL jobs.

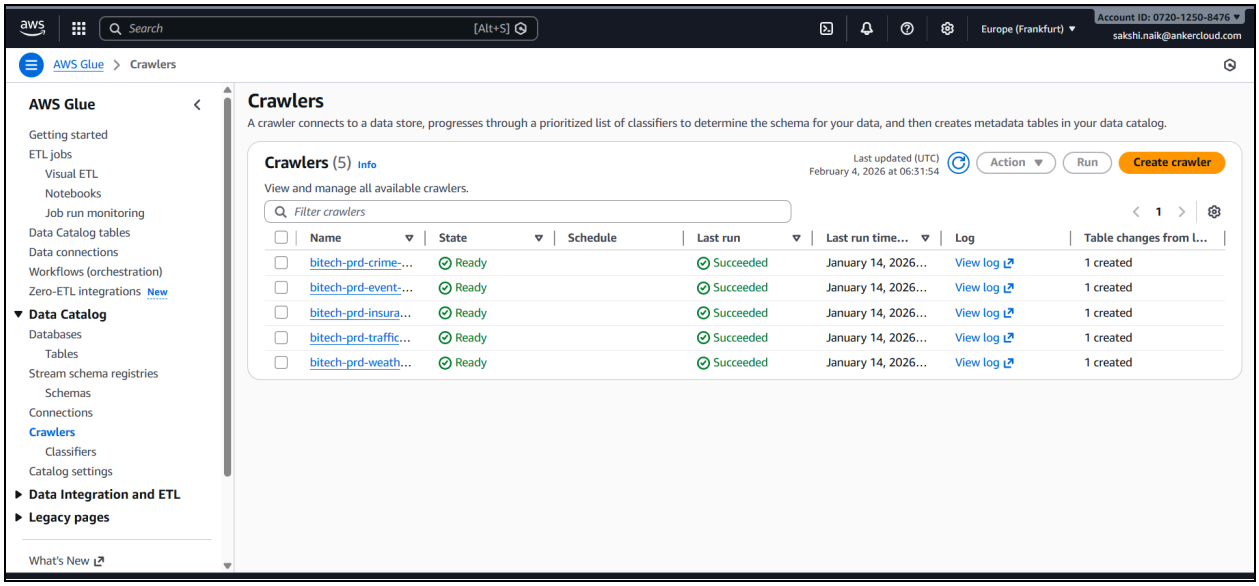
Crawler execution follows a controlled workflow:

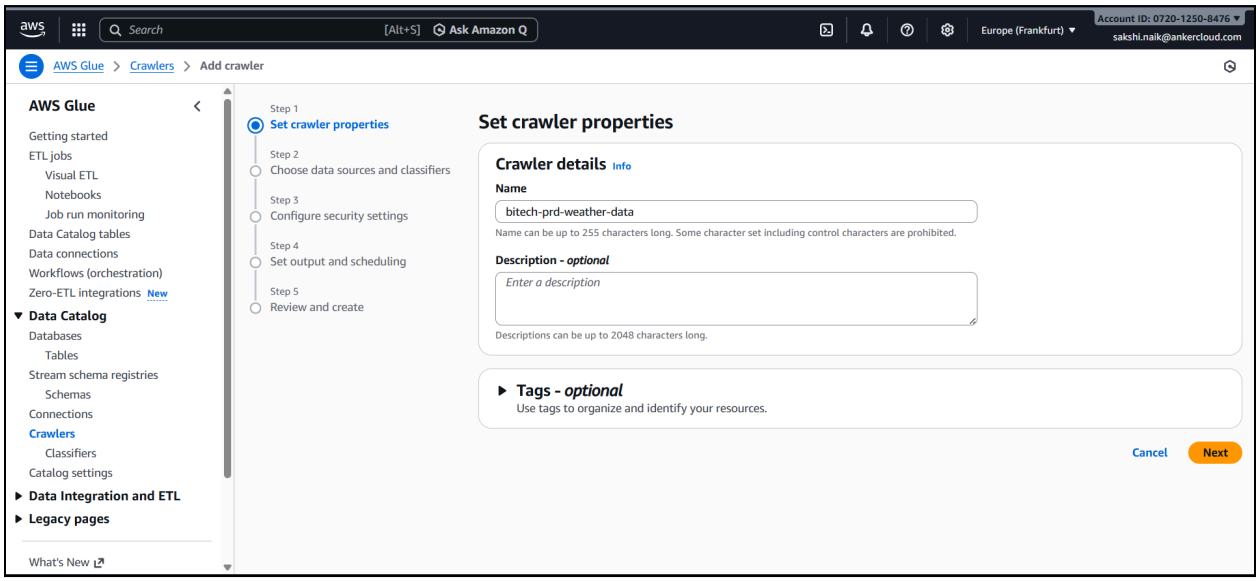
- Crawlers scan predefined S3 paths (Bronze or Silver layers)
- Schemas are inferred automatically from data files
- Databases and tables are created or updated in the Glue Data Catalog
- Partition metadata is refreshed to reflect newly added data

This ensures that Athena and Lake Formation always operate on accurate and up-to-date metadata without manual intervention.

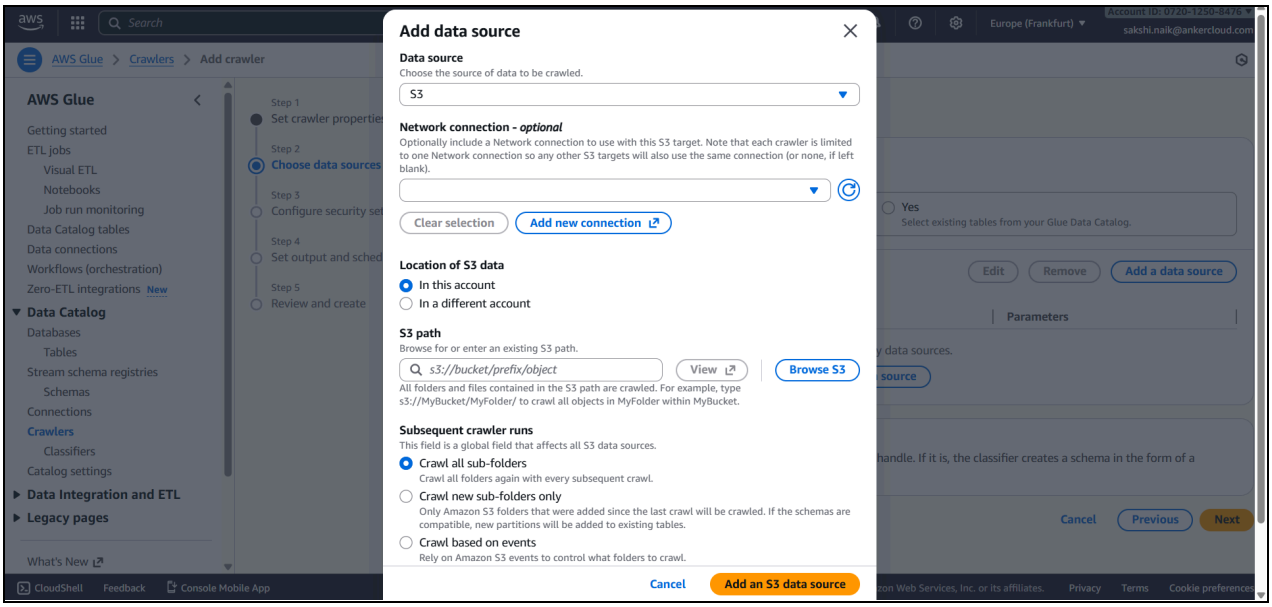
Steps to Create Glue Crawler

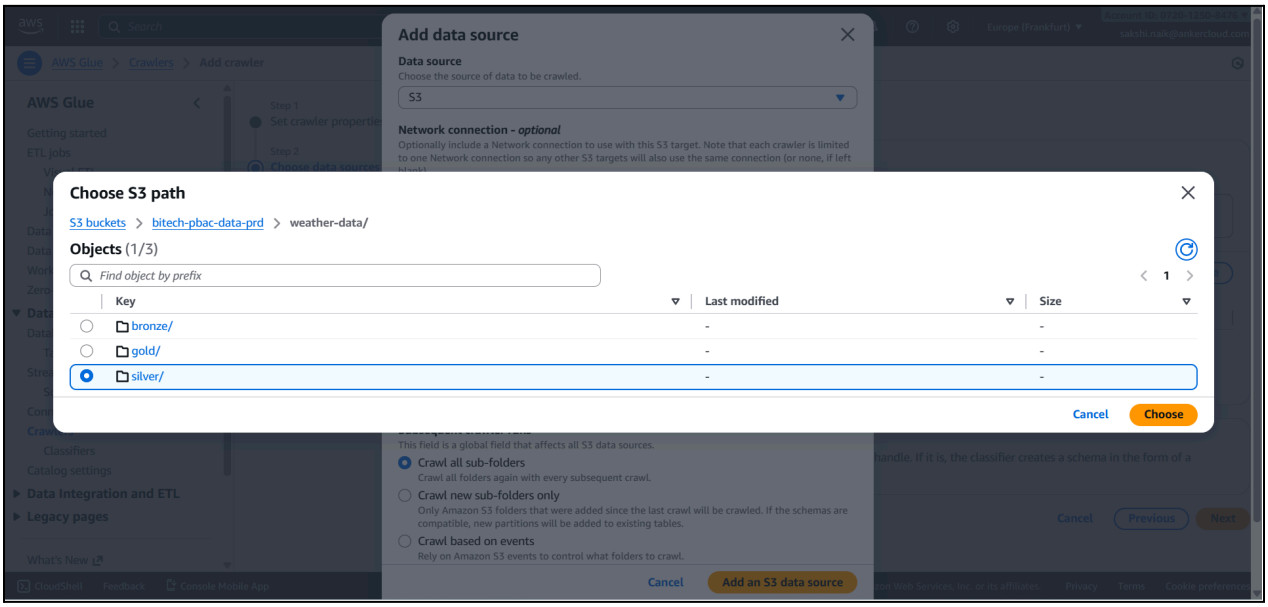
- Click on Create Crawler
- Fill in the Crawler Details → Click next (here Name : bitech-prd-weather-data)



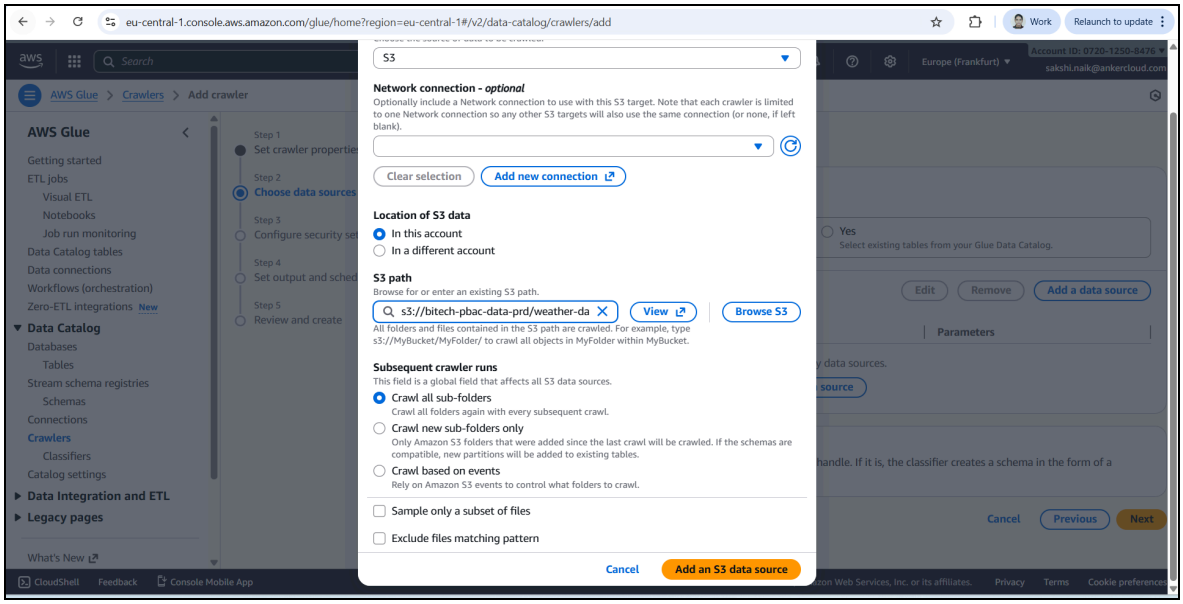


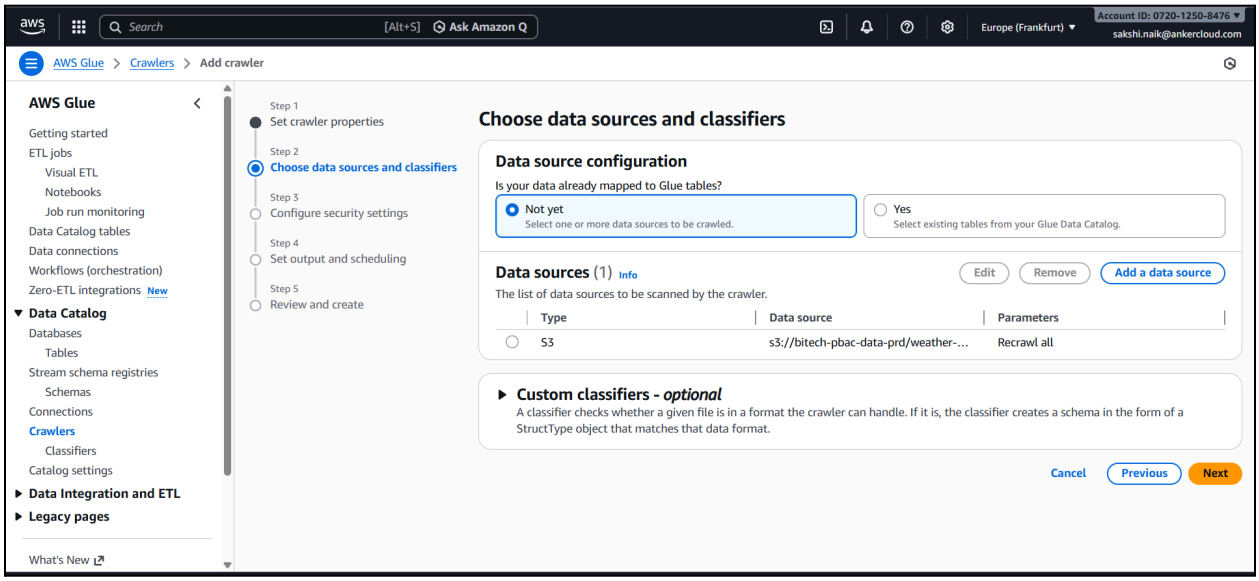
- In Data Source → Browse S3 → Go to S3 bucket (bitech-pbac-data-prd) → then click the required data source (here its weather-data) → silver → Choose



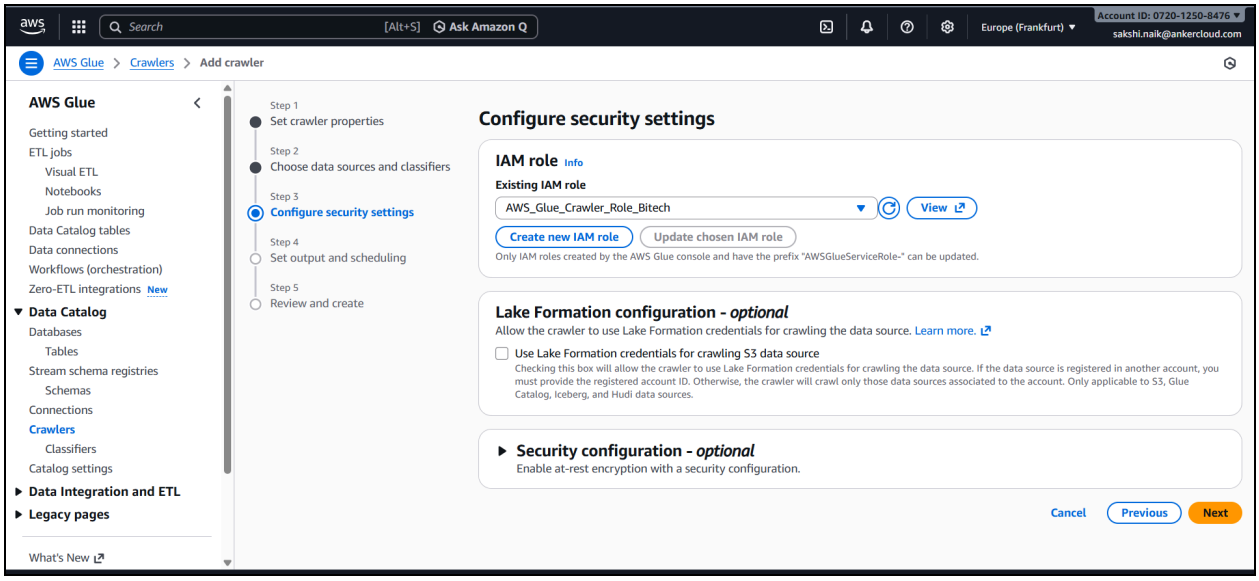


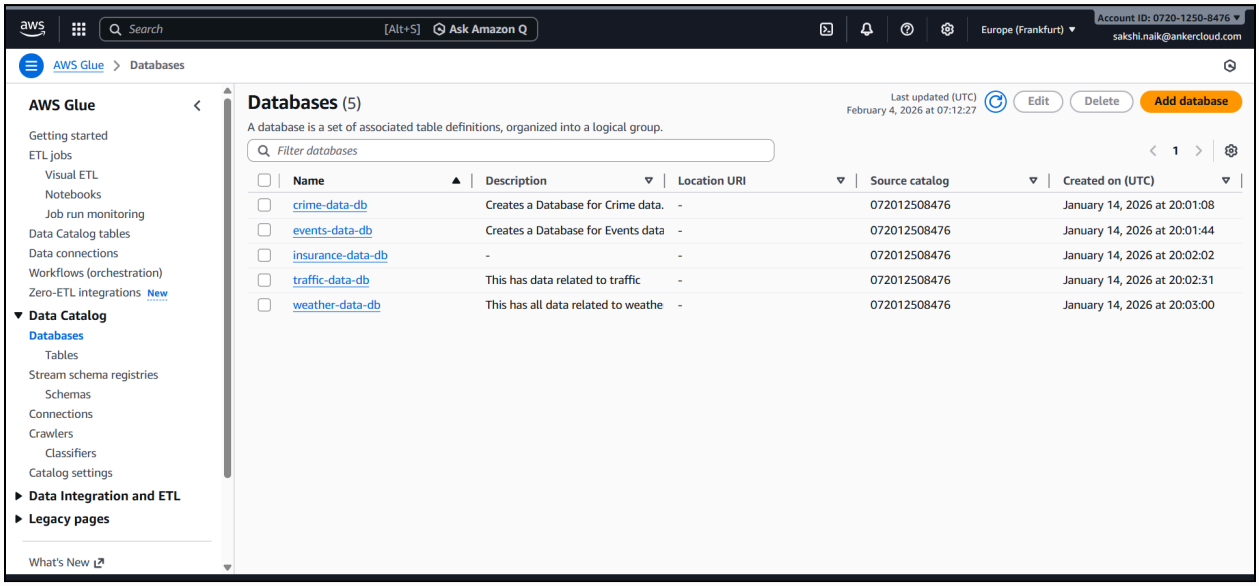
- After clicking Choose.
- Click on Crawl all sub-folders → Click Add S3 data source

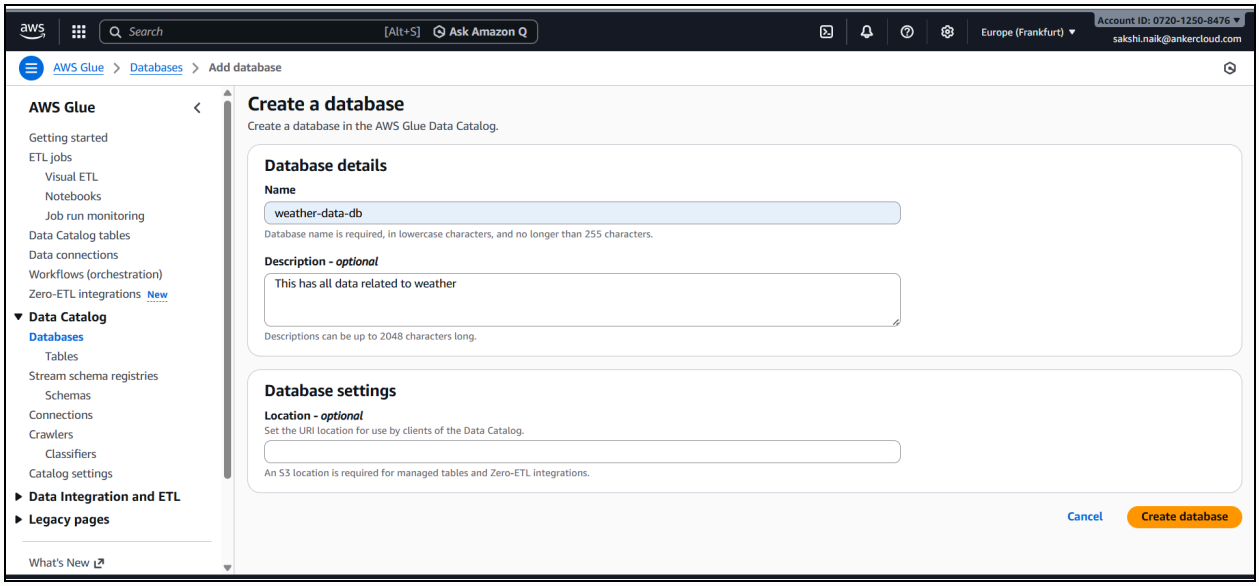




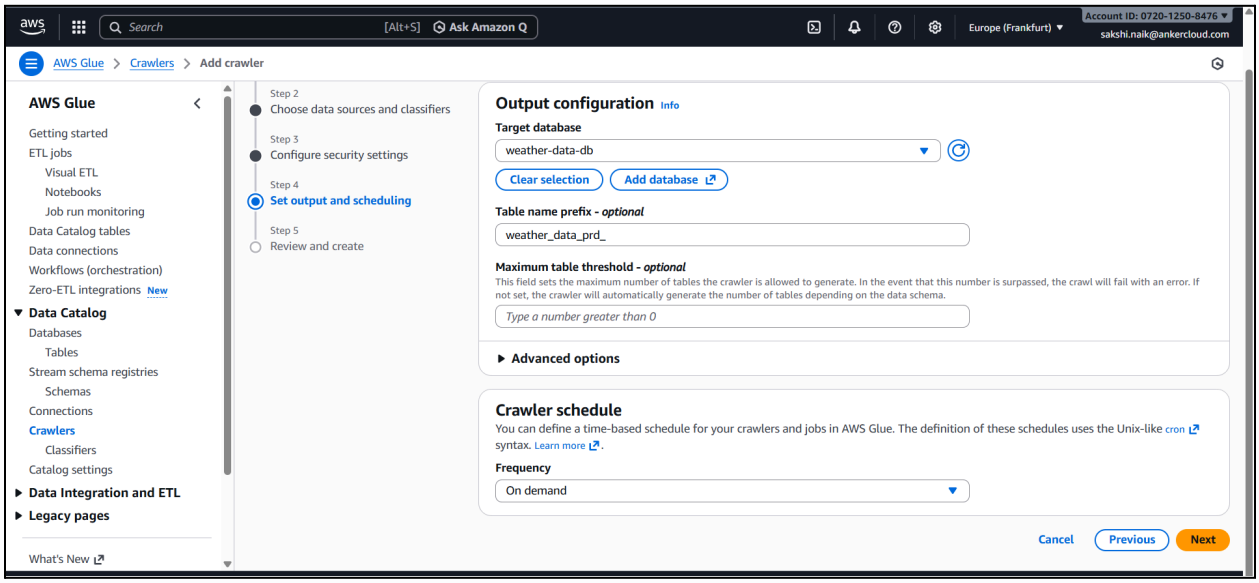
- Then select **IAM role (that is AWS_Glue_Crawler_Role_Bitech)** → Click Next
- Now go to Database inside Data Catalog → Give a database name weather-data-db → Add description → Click Create Database.
- Similarly perform for all other data sources.


Search
[Alt+S]
Ask Amazon Q
Europe (Frankfurt)
Account ID: 0720-1250-8476
sakshi.naik@ankercloud.com
AWS Glue > Crawlers > Add crawler
AWS Glue
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations New
Data Catalog
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
Data Integration and ETL
Legacy pages
What's New
Step 1
Set crawler properties
Step 2
Choose data sources and classifiers
Step 3
Configure security settings
Step 4
Set output and scheduling
Step 5
Review and create
Configure security settings
IAM role Info
Existing IAM role
AWS_Glue_Crawler_Role_Bitech
View
Create new IAM role
Update chosen IAM role
Only IAM roles created by the AWS Glue console and have the prefix "AWSGlueServiceRole-" can be updated.
Lake Formation configuration - optional
Allow the crawler to use Lake Formation credentials for crawling the data source. Learn more.
Use Lake Formation credentials for crawling S3 data source
Checking this box will allow the crawler to use Lake Formation credentials for crawling the data source. If the data source is registered in another account, you must provide the registered account ID. Otherwise, the crawler will crawl only those data sources associated to the account. Only applicable to S3, Glue Catalog, Iceberg, and Hudi data sources.
Security configuration - optional
Enable at-rest encryption with a security configuration.
Cancel
Previous
Next

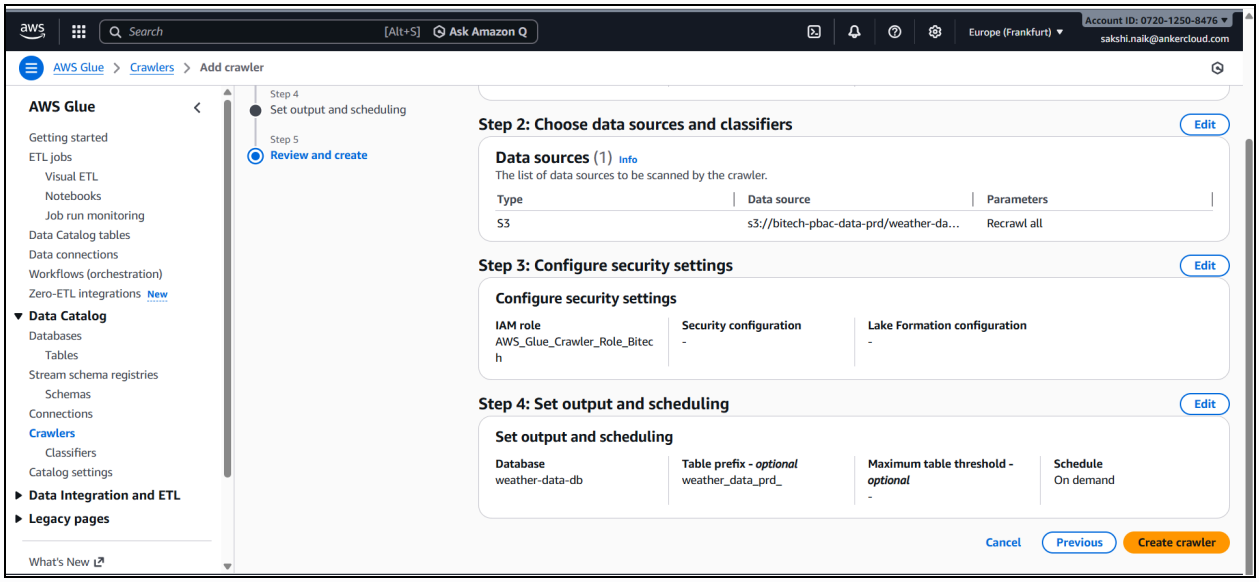

Search
[Alt+S]
Ask Amazon Q
Europe (Frankfurt)
Account ID: 0720-1250-8476
sakshi.naik@ankercloud.com
AWS Glue > Databases
AWS Glue
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations New
Data Catalog
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
Data Integration and ETL
Legacy pages
What's New
Databases (5)
A database is a set of associated table definitions, organized into a logical group.
Last updated (UTC)
February 4, 2026 at 07:12:27
Edit
Delete
Add database
Filter databases
1
Name
Description
Location URI
Source catalog
Created on (UTC)
crime-data-db
Creates a Database for Crime data.
-
072012508476
January 14, 2026 at 20:01:08
events-data-db
Creates a Database for Events data
-
072012508476
January 14, 2026 at 20:01:44
insurance-data-db
-
-
072012508476
January 14, 2026 at 20:02:02
traffic-data-db
This has data related to traffic
-
072012508476
January 14, 2026 at 20:02:31
weather-data-db
This has all data related to weathe
-
072012508476
January 14, 2026 at 20:03:00

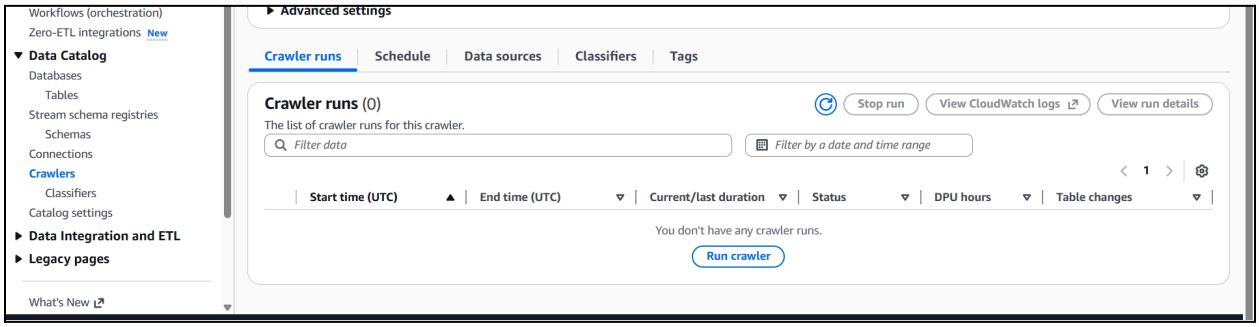


- After selecting the Target Database → and adding Table name prefix → Select Crawler schedule → On demand → Click Next

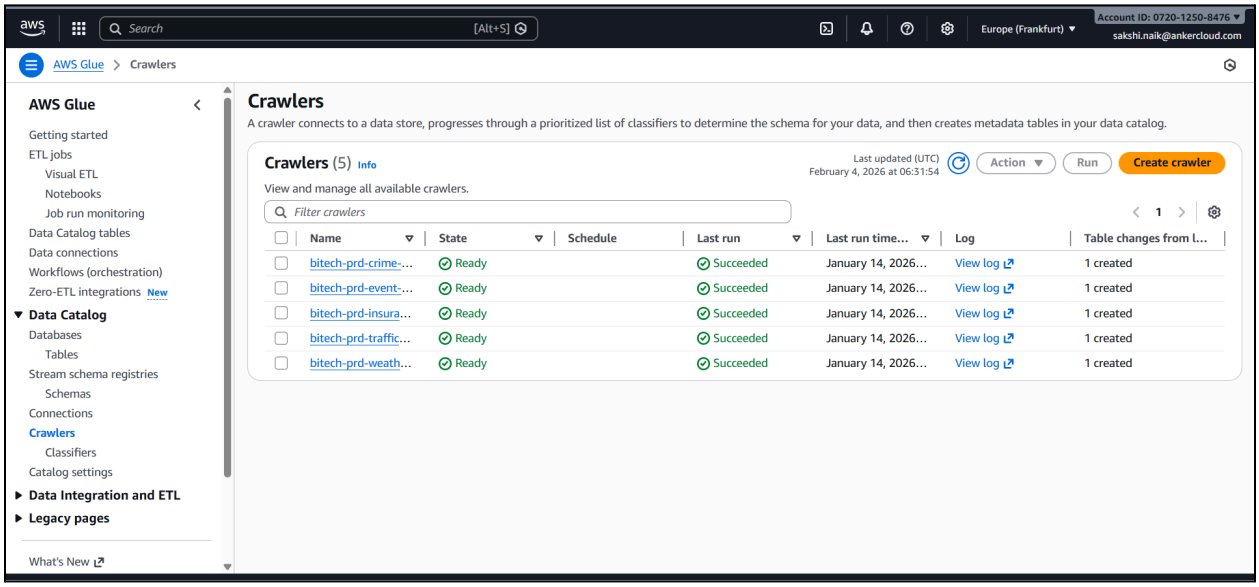


- Review → Click Create Crawler





- Then Click Run Crawler → After that wait till you see → Succeeded



6.3 Glue Databases & Table Organization

Each major data domain is organized into its own Glue database to maintain clear separation and governance:

- traffic-data-db
- crime-data-db
- events-data-db
- weather-data-db
- insurance-data-db

Within each database, tables correspond to Silver datasets stored in S3. This structured catalog design simplifies permission management, query execution, and LF Tag assignment.

7. Lake Formation Authorization Model

Once tables are available in the Glue Data Catalog, they are automatically visible in AWS Lake Formation. Lake Formation uses these cataloged tables as the enforcement point for:

- LF-Tag assignment
- Table-level and column-level access control
- Purpose Based Access Control (PBAC)

Glue itself does not enforce access it provides metadata. All authorization decisions are enforced downstream by Lake Formation when queries are executed via Athena.

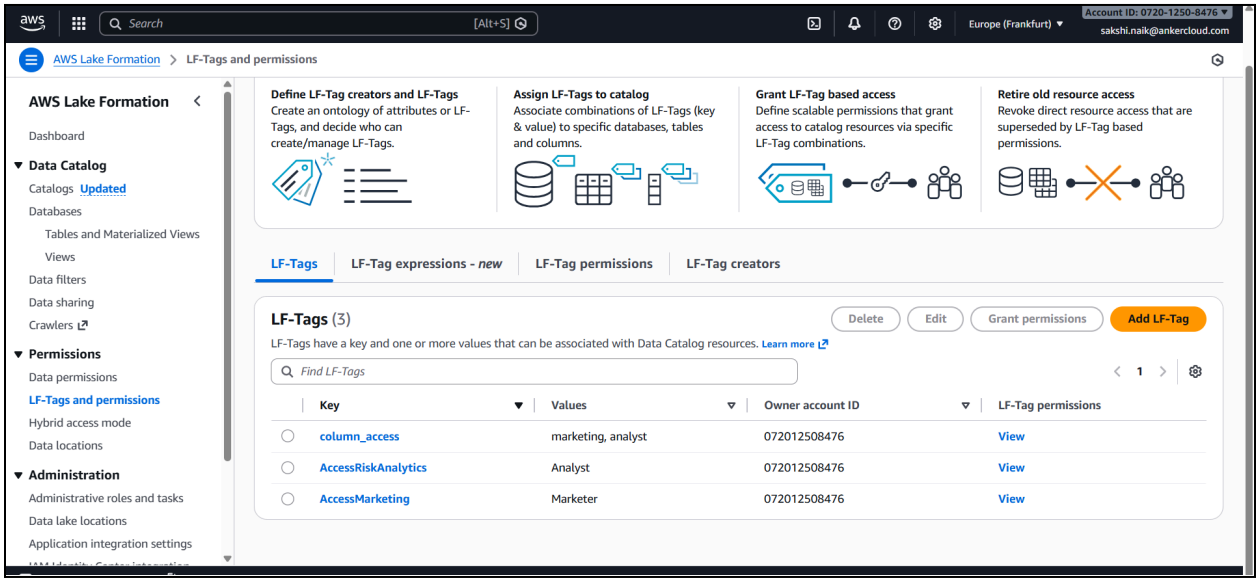
7.1 LF-Tag Creation and Table Classification

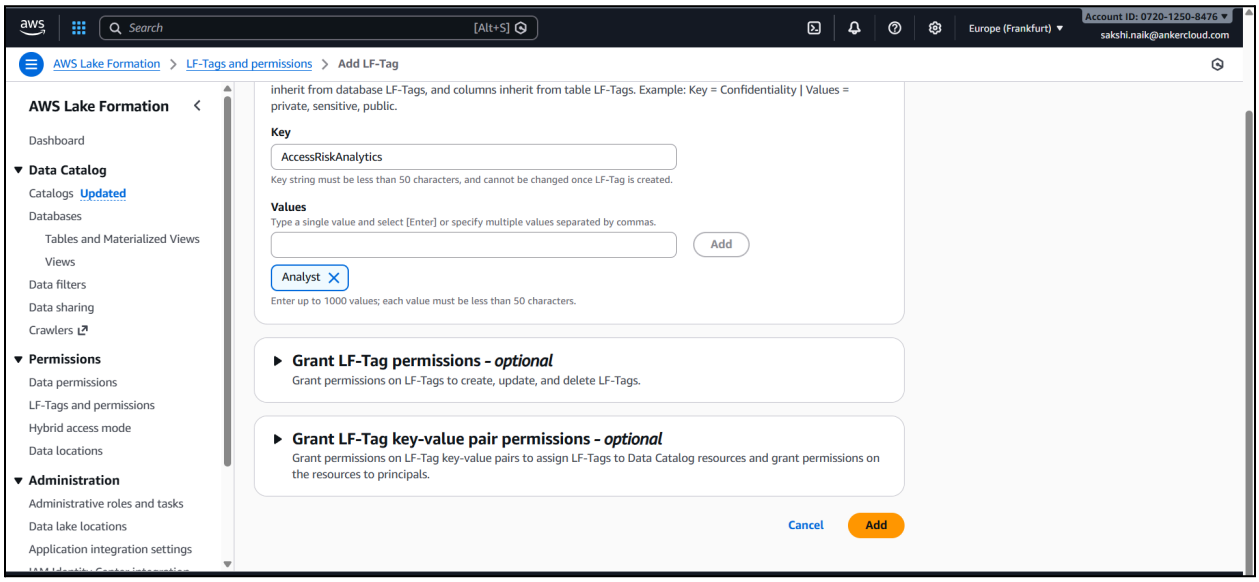
In the Bitech PBAC architecture, Lake Formation LF-Tags are used to represent business purposes rather than individual users or applications. Each LF-Tag defines a logical access intent such as risk analytics or marketing insights. These tags are centrally created and then attached to Glue Catalog tables, primarily at the Silver layer, to classify data based on its intended usage. By tagging tables instead of granting direct permissions, the system ensures that access decisions remain consistent, scalable, and decoupled from individual IAM roles, enabling centralized governance as the number of datasets and users grows.

Purpose:
LF-Tags represent business purpose or an access intent (PBAC) and are the core authorization mechanism in the Bitech platform.

Steps:

- Open **AWS Lake Formation Console**
- Navigate to Permissions → **LF-Tags and permissions** → Click Add LF tag → Create the necessary tags (Key and Value) → Click Add





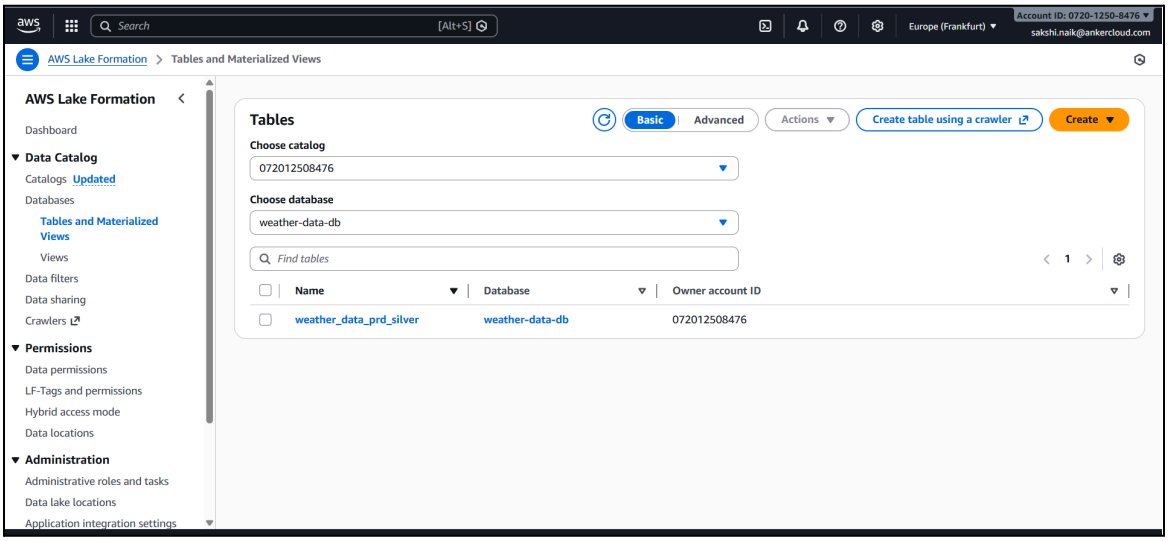
- Create LF-Tags based on purpose:
    - AccessRiskAnalytics = Analyst

- AccessMarketing = Marketer

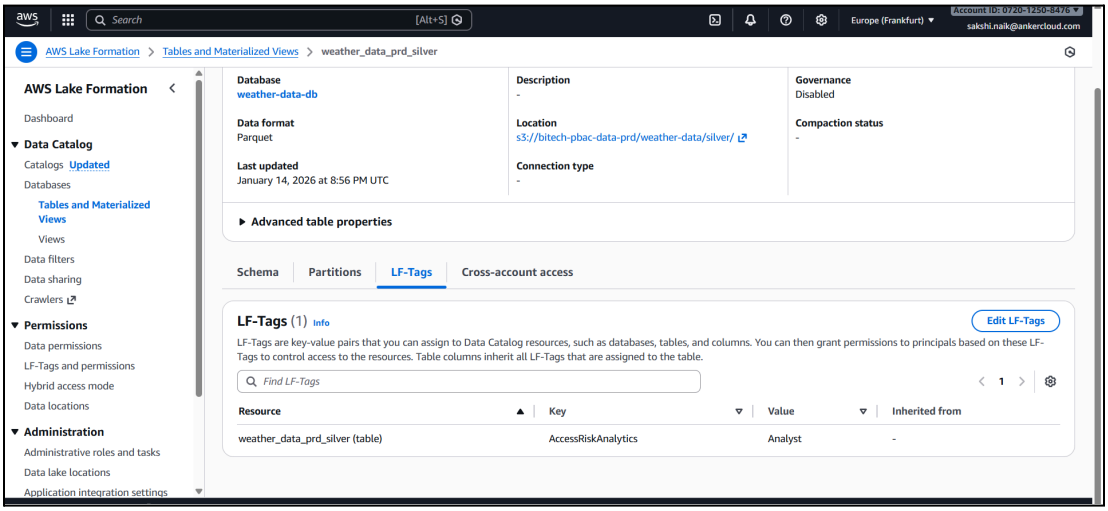
- Verify the tags are visible under **LF-Tags catalog**

Assign LF-Tags to Data Assets:

- In AWS Lake formation → Go to **Data Catalog** → **Tables and Materialized Views**



- Select the required Glue databases and tables → Inside the table → Go to LF tags → Choose **Edit LF-Tags**



- Attach appropriate LF-Tags:

    - Crime / insurance / weather/ event tables → AccessRiskAnalytics = Analyst
    - Column level insurance data / traffic/ event → AccessMarketing = Marketer
- Save changes

At this point, tables are classified by purpose, but no one has access yet.

7.2 Data Access Permissions Using LF-Tags

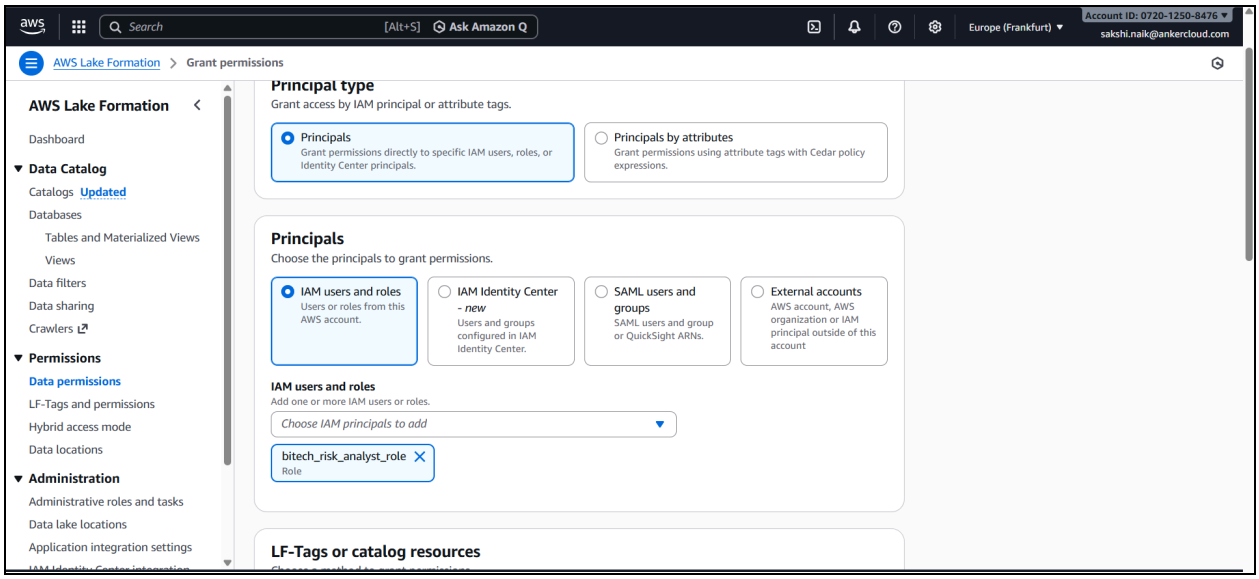
Data access in the platform is controlled via **Lake Formation tags** applied at the table level and column level, combined with IAM role based permissions. Each table is tagged with LF-Tags representing access groups (e.g., AccessRiskAnalytics = Analyst, AccessMarketing = Marketer). When a user assumes an IAM role (e.g., bitech_risk_analyst_role), Lake Formation evaluates the LF-Tags on the requested table. If the LF-Tags match the role's allowed tags, the query is authorized; otherwise, it is denied. This approach ensures fail-closed security while still enabling table level control and simplified permission management across the platform.

Purpose:

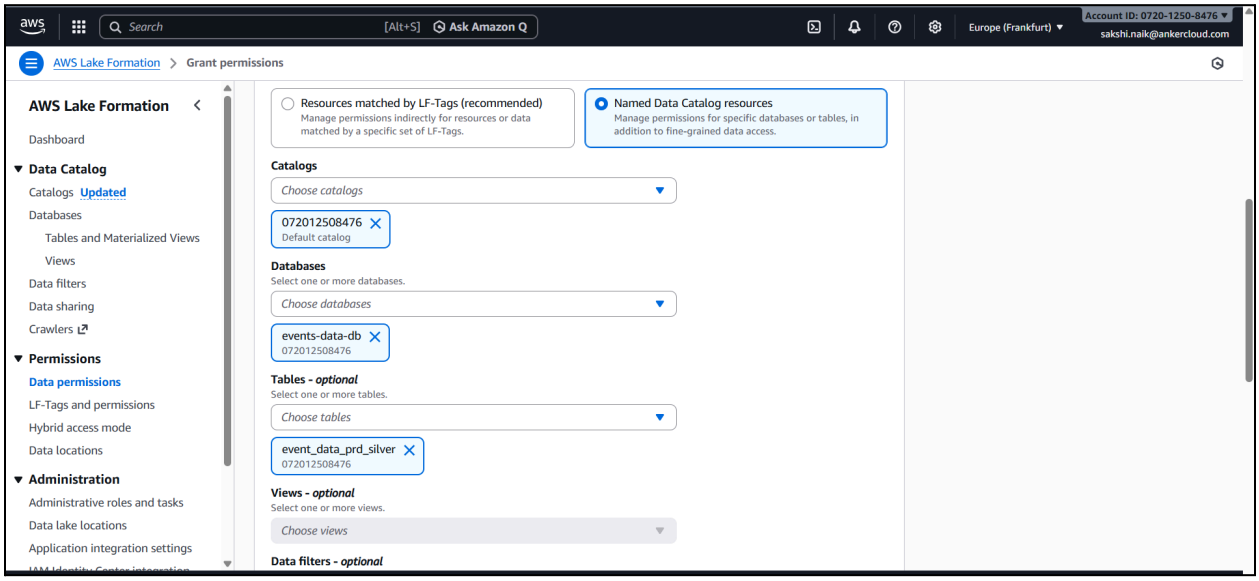
- Centralize access control via LF-Tags
- Enable scalable PBAC (Purpose/Policy Based Access Control) without manually granting each table per role.

Steps (Table Level Tag + Grant Flow):

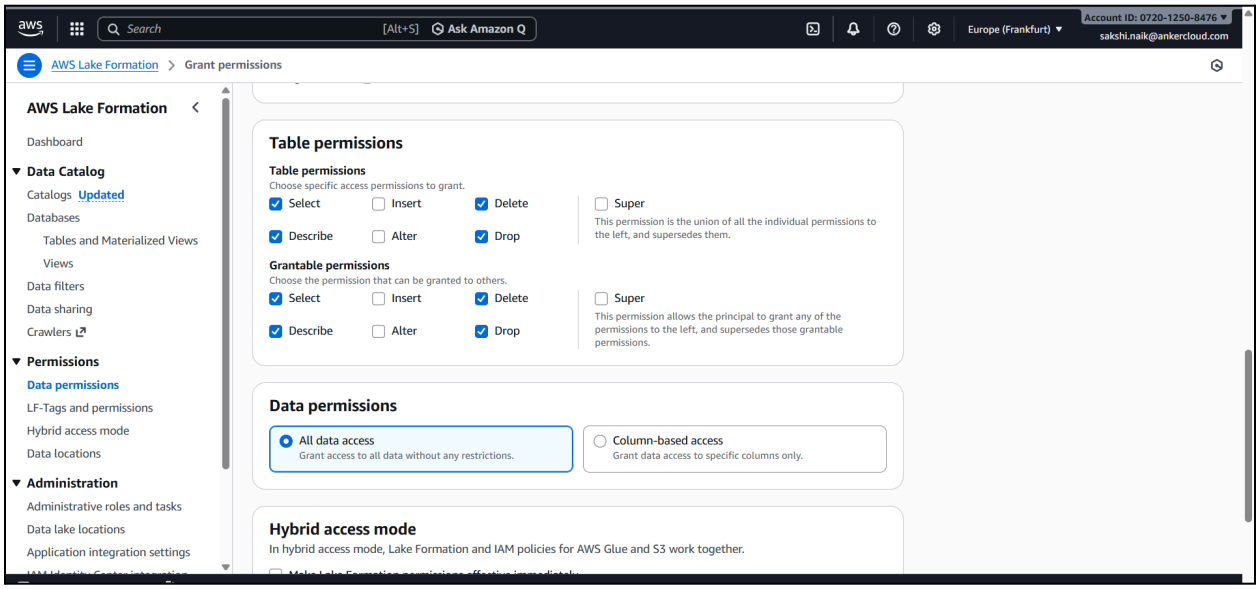
- Go to **Lake Formation** → **Tables** and select the tables
- Apply **LF-Tags** to the tables according to access groups
    - Example: AccessRiskAnalytics = Analyst
    - Example: AccessMarketing = Marketer
- Go to **Lake Formation** → **Data Permissions** → **Grant (** Refer this https://eu-central-1.console.aws.amazon.com/lakeformation/home?region=eu-central-1#permissions-list)



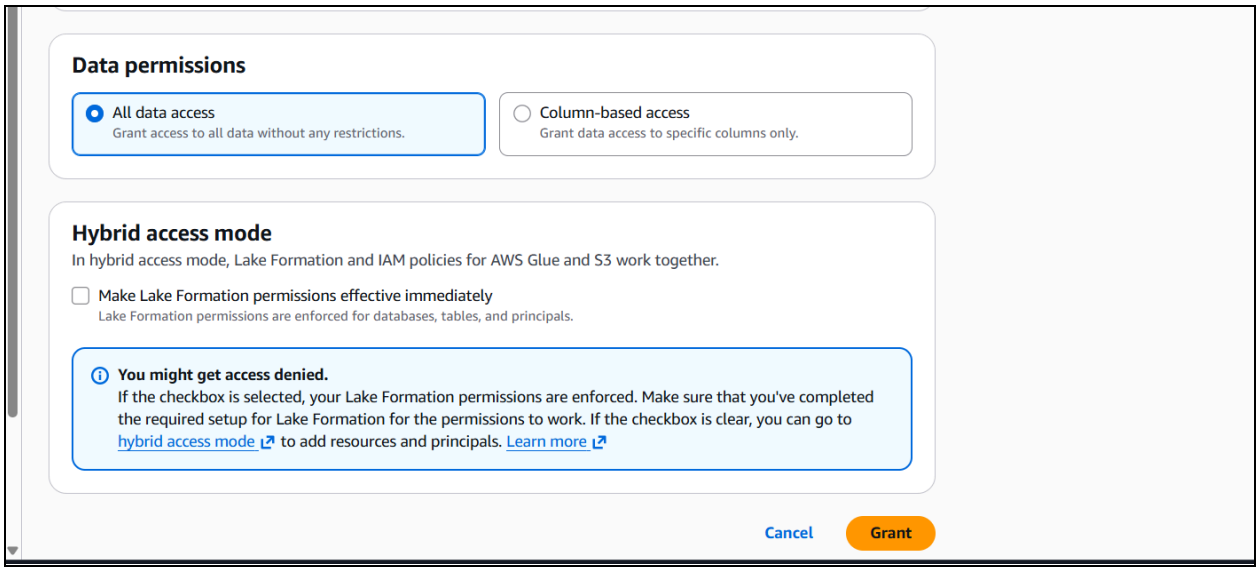
- Under **Principals**, select the IAM roles to assign:
    - bitech_risk_analyst_role
    - bitech_marketing_role
- Under LF-Tags or catalog resources → choose **Named Catalog Resources**
- Select **Resources**: either database or tables



- Assign permissions:
    - SELECT (required)
    - DESCRIBE (optional, for schema visibility)
    - DELETE (optional)
    - DROP (optional)



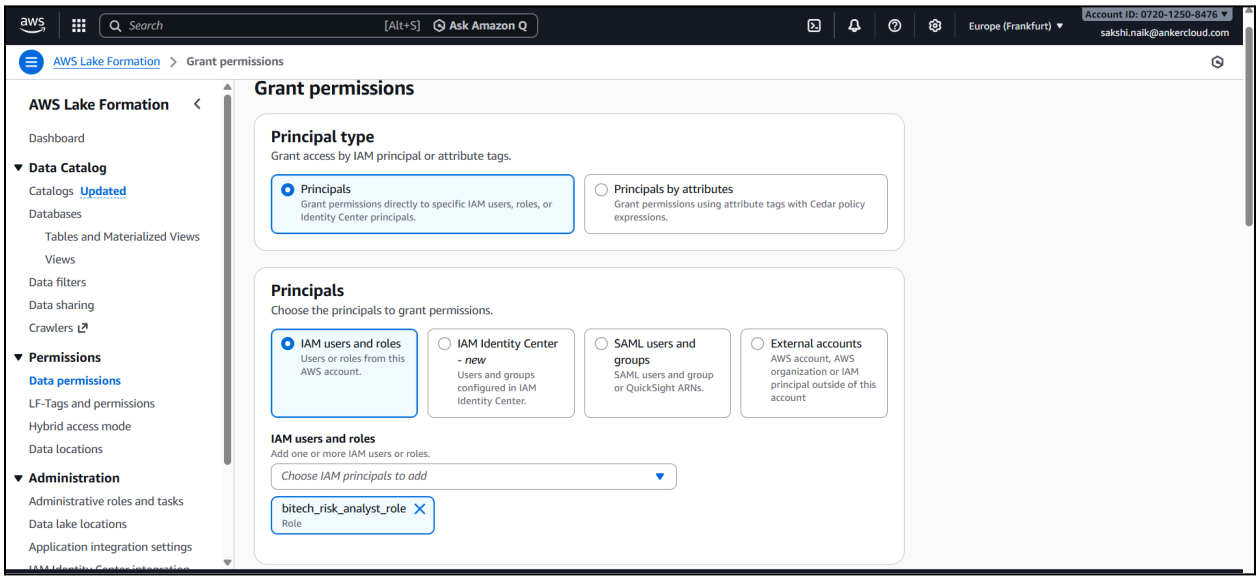
- Now in Data Permissions you can either select All data access or Column based access

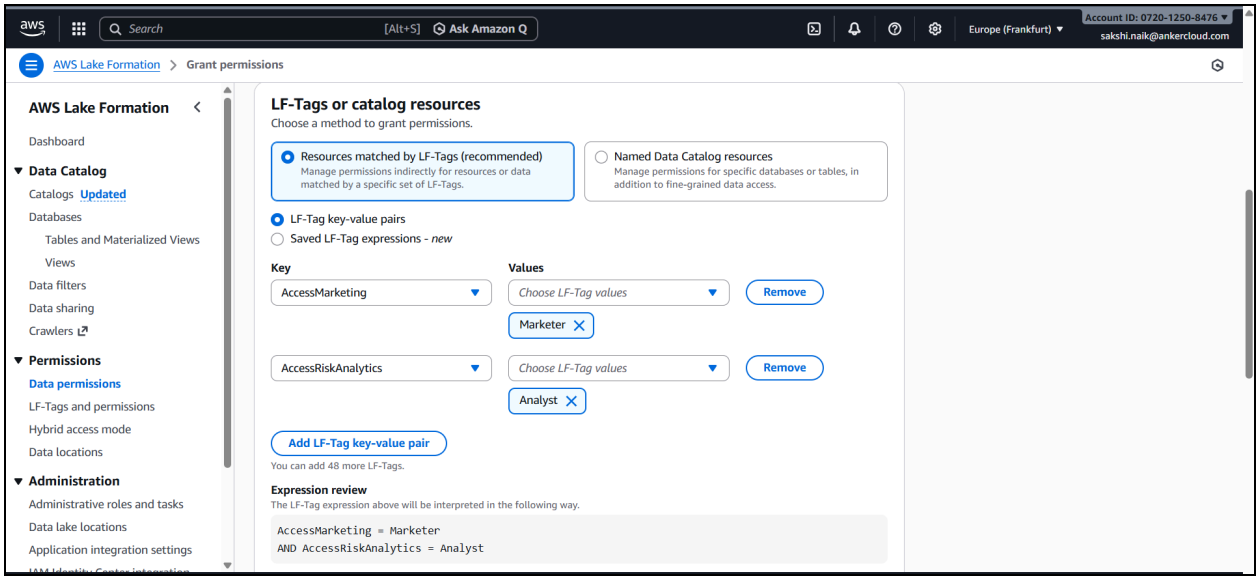


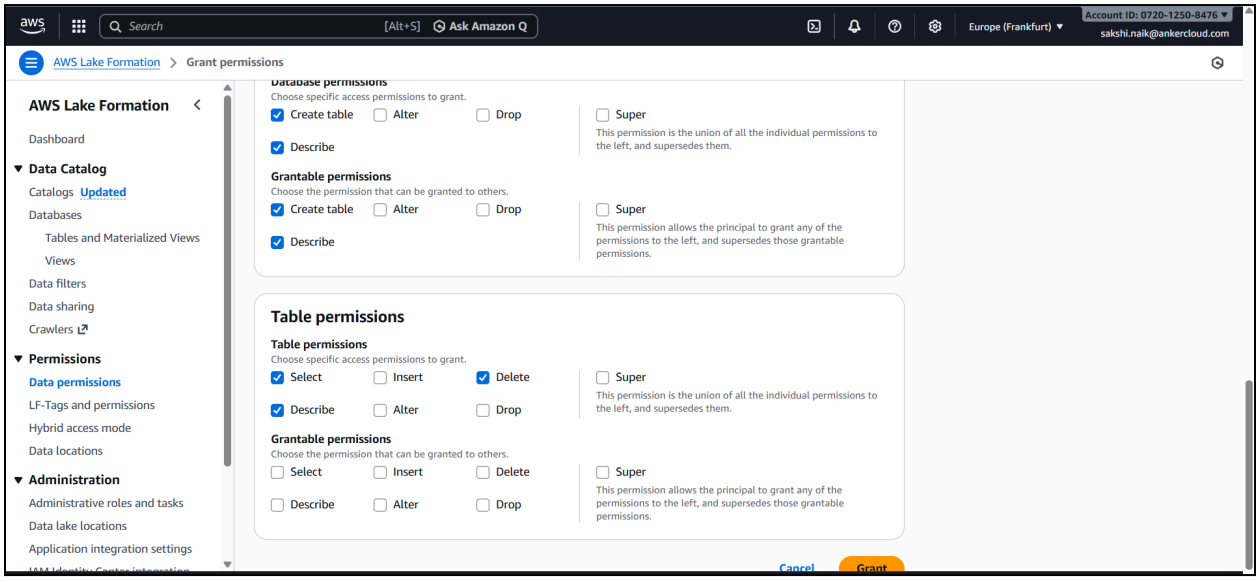
- Click **Grant**

Now this was all based on fine grained data permissions that is catalog based selection only.

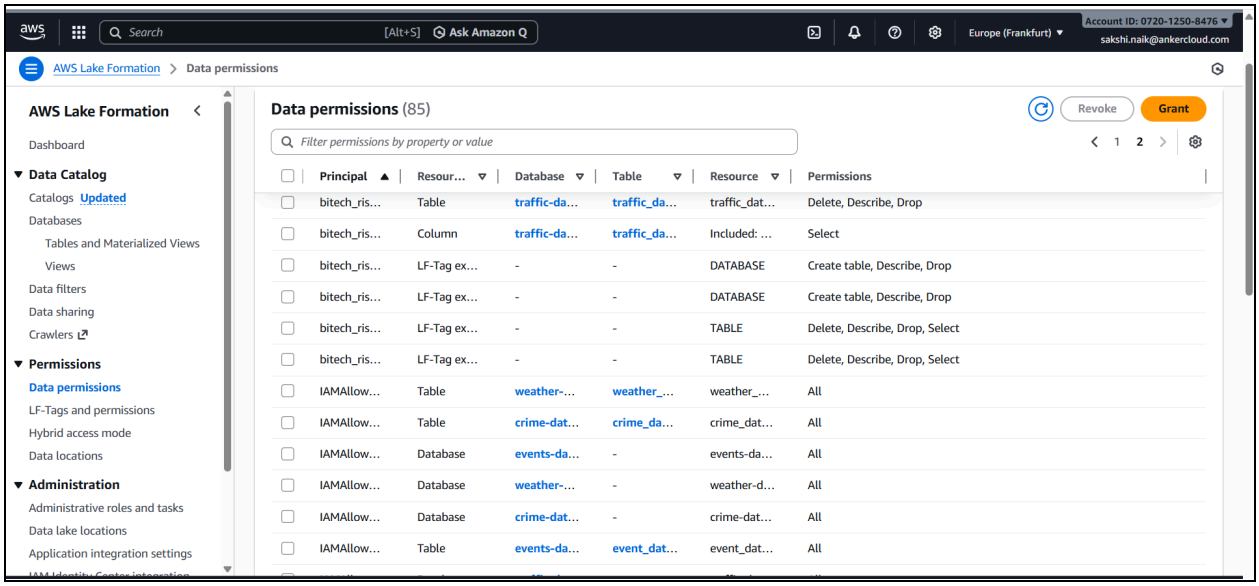
- Go again to Lake Formation → Data Permissions → Grant → Select IAM users and roles → In LF-Tags or catalog resources → Select LF-Tag key-value pairs → Choose the LF tags (Key value pair you created) → And then do Database permissions and table permissions accordingly → then click Grant







Now that we have assigned all required data permissions, LF-Tag based permissions have been used to grant database and table level access, enabling principals to discover and access the required catalog resources. All fine-grained data permissions, including column level, have been granted using named catalog resources (catalog → database → table) and not via LF-Tags. This approach ensures controlled access at scale while maintaining precise data visibility where required.



7.3 Column Level Access Control Using Data Filters

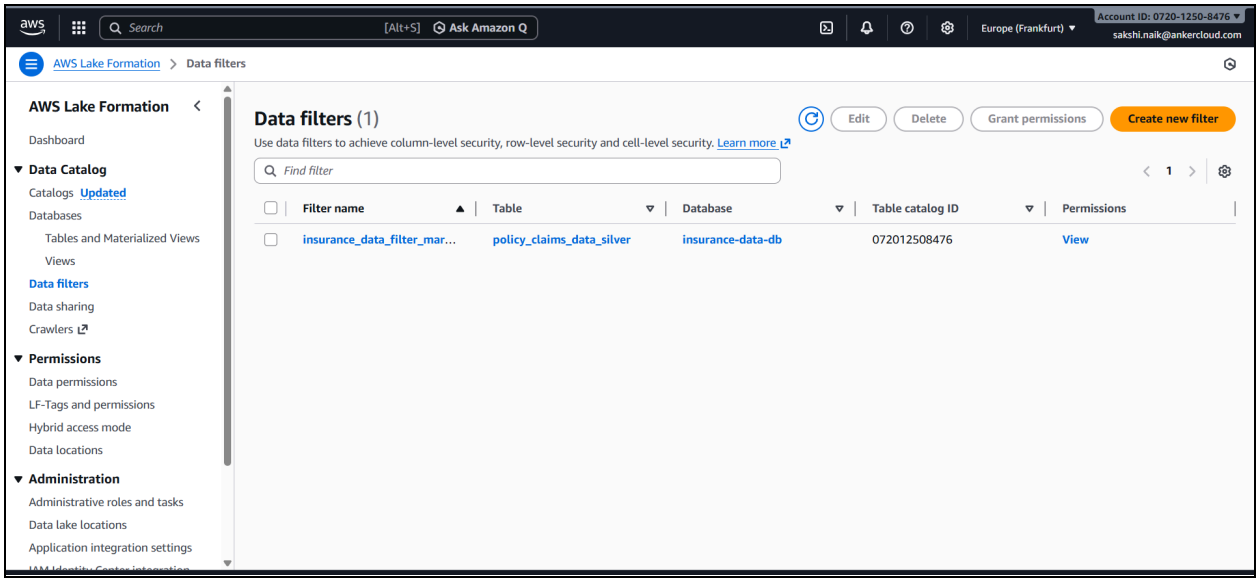
For datasets that require fine grained control, Lake Formation data filters are used to enforce Column level data access restrictions.

Data filters allow administrators to expose only approved columns from a table while hiding sensitive or restricted attributes, without duplicating data or modifying schemas. These filters are applied to IAM roles and evaluated transparently during query execution, ensuring that users can query the same table but see only the columns they are authorized to access. This approach strengthens data governance while keeping analytics workflows simple and consistent.

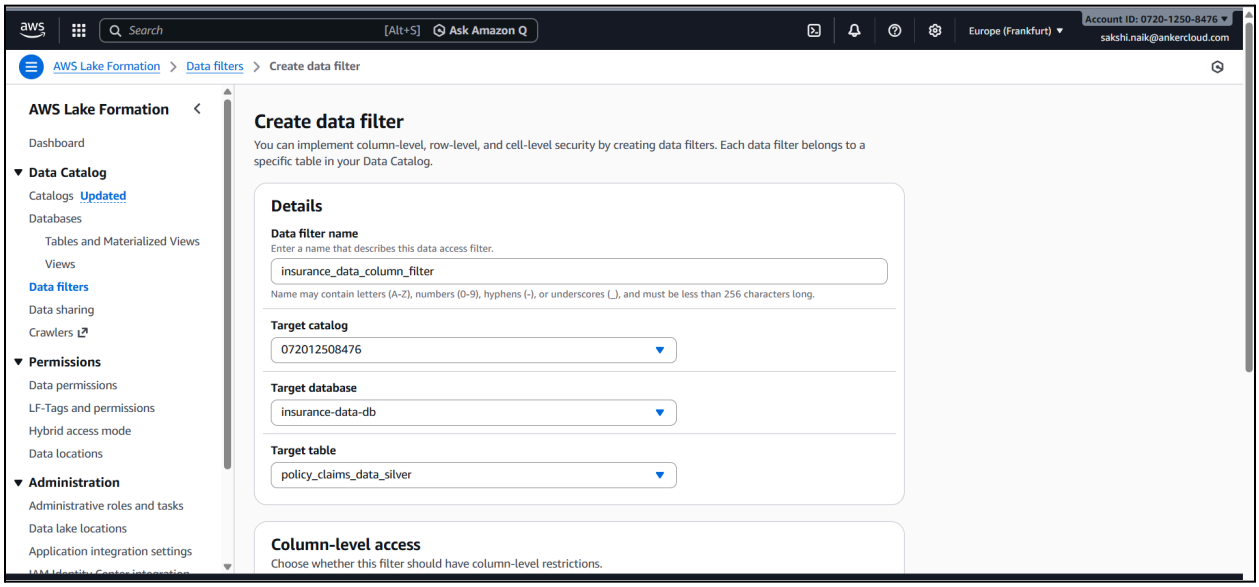
AWS Lake Formation → Data Filters : https://eu-central-1.console.aws.amazon.com/lakeformation/home?region=eu-central-1#data-filters

Steps :

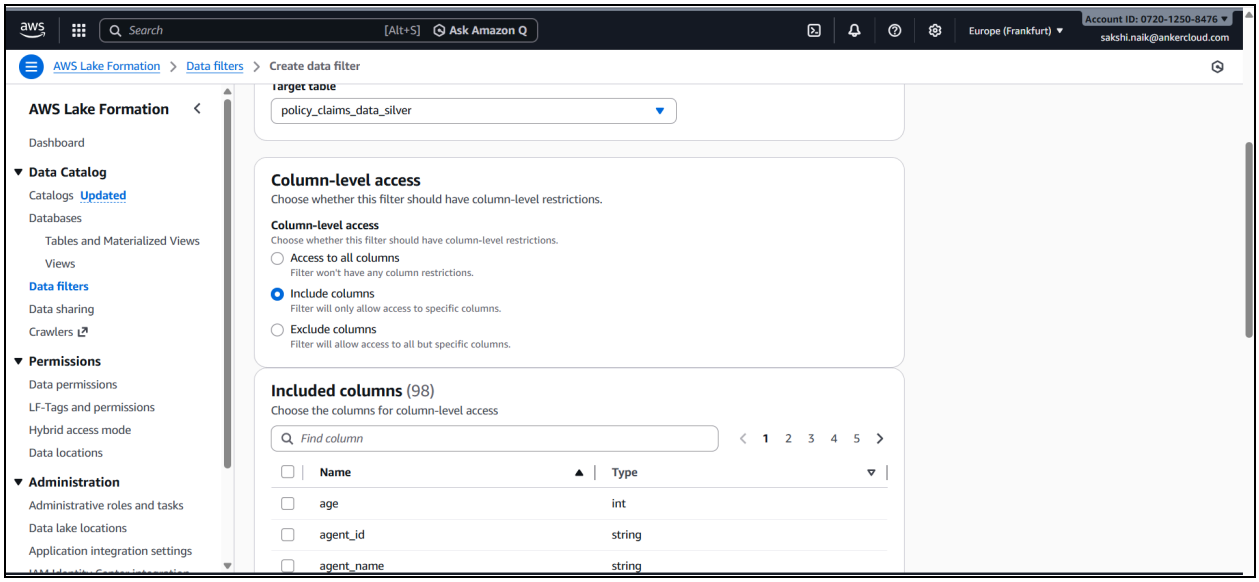
- Go to **Lake Formation** → Data Catalog → **Data Filters** → Create new filter



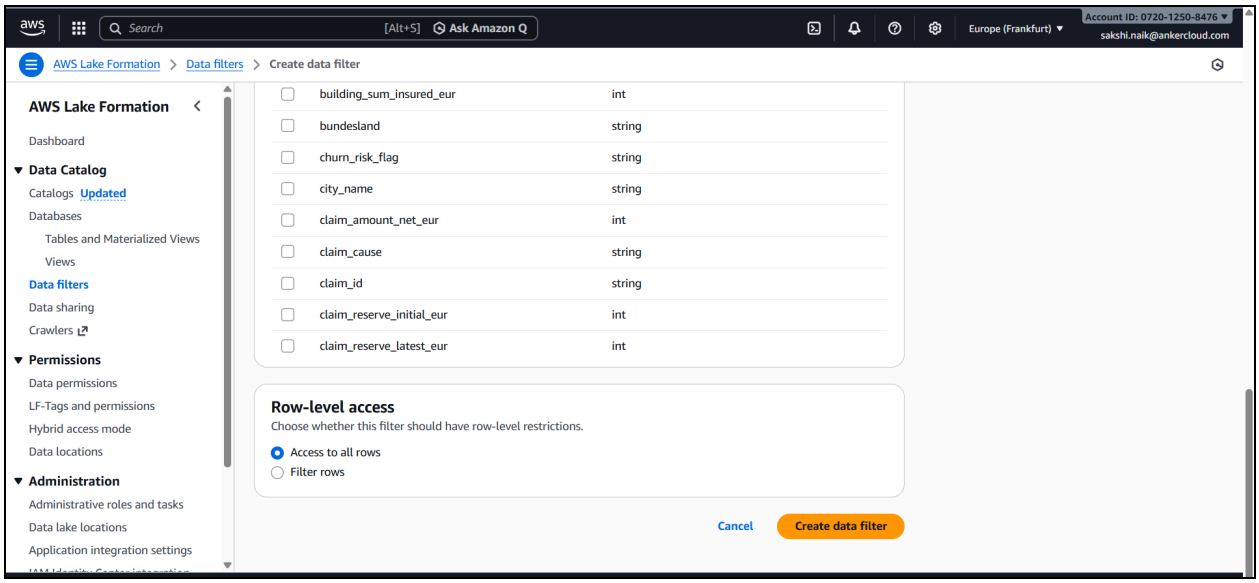
- Enter a data filter name (based on what data you want to create a data filter) → Select database → Table

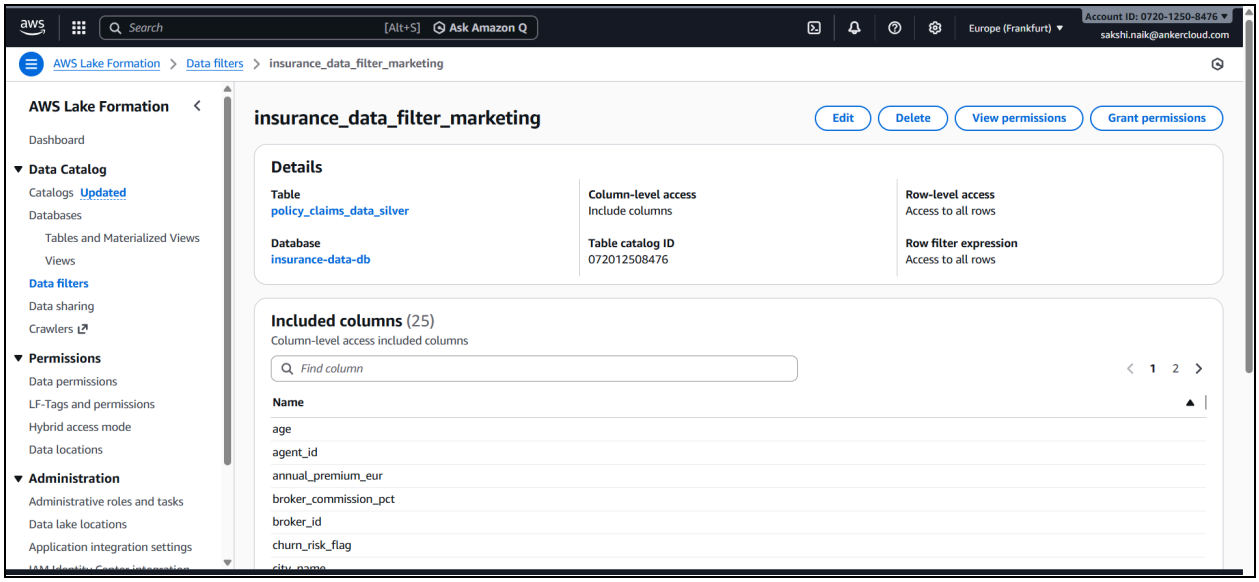


- Then in Column level access we have options like Include columns (Filter will only allow access to specific columns) or Exclude columns (Filter will allow access to all but specific columns) → Select the option which suitables your use case



- Here we have selected Include Columns → And then selected 25 columns which we wanted for the marketing role → In Row-level access → Access to all rows.

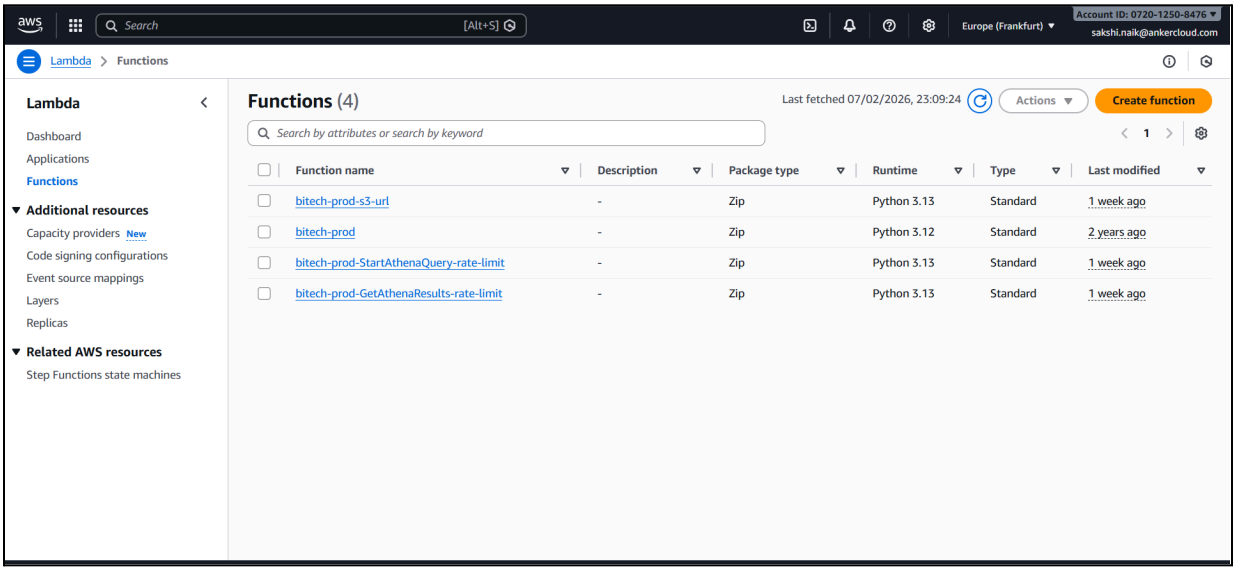




8. Lambda Implementation

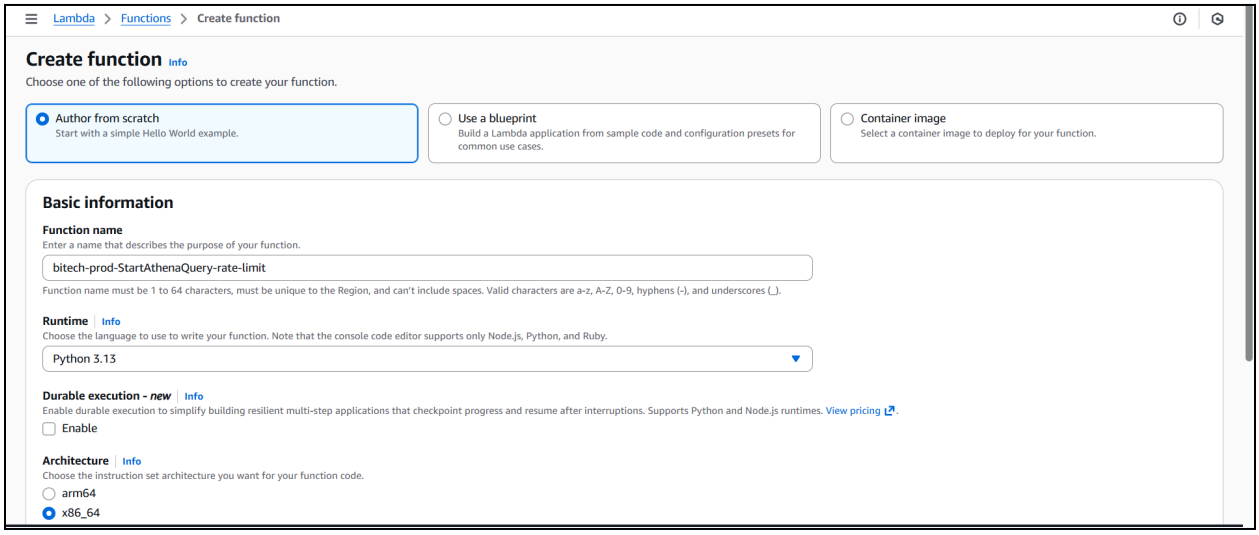
AWS Lambda functions implement the execution layer of the Bitech PBAC platform. Each Lambda is designed around a single responsibility, aligned with a specific data access pattern, and enforces purpose-based access by dynamically assuming IAM roles and relying on AWS Lake Formation for final authorization decisions.

The Lambda layer is intentionally split into query driven access and object-based access, enabling flexibility in consumption while preserving centralized governance, auditing, and fail-closed security.

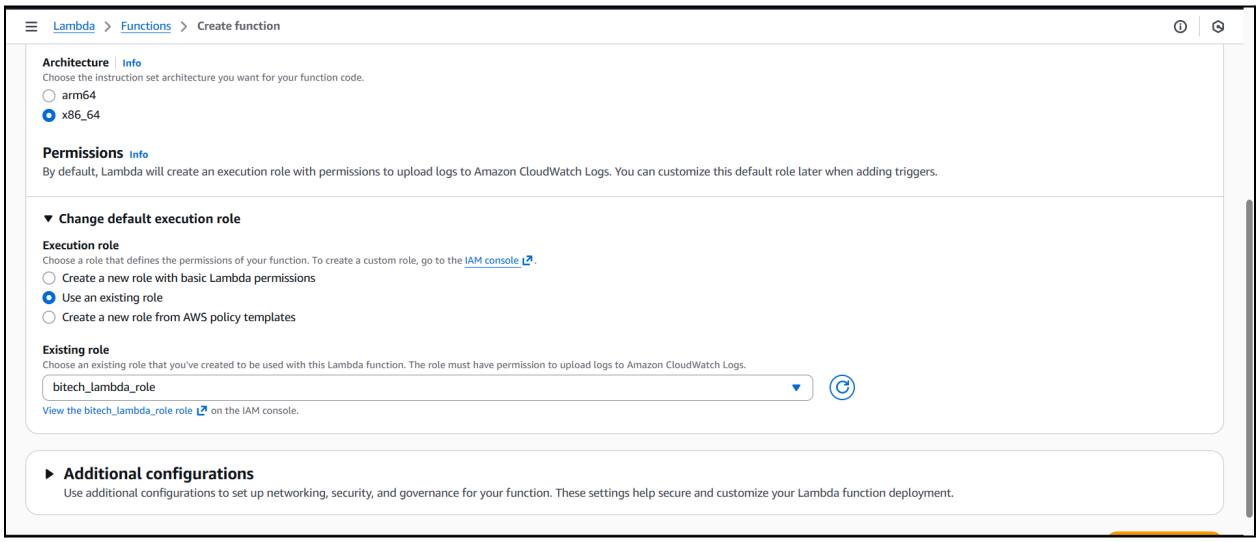


Steps:

- Click on Create Function
- Click Author from scratch → Give Function Name → Select suitable Runtime → Architecture → Choose x86_64



- In Change default execution role → Select the role which we created for lambda → that is **bitech_lambda_role → Click Create function.**



8.1 Athena Based Data Access Approach

Data access via Athena in this solution is divided into two parts, implemented as separate Lambdas:

- StartAthenaQuery Lambda - Initiates query execution asynchronously.
- GetAthenaResults Lambda - Polls for query completion and returns results.

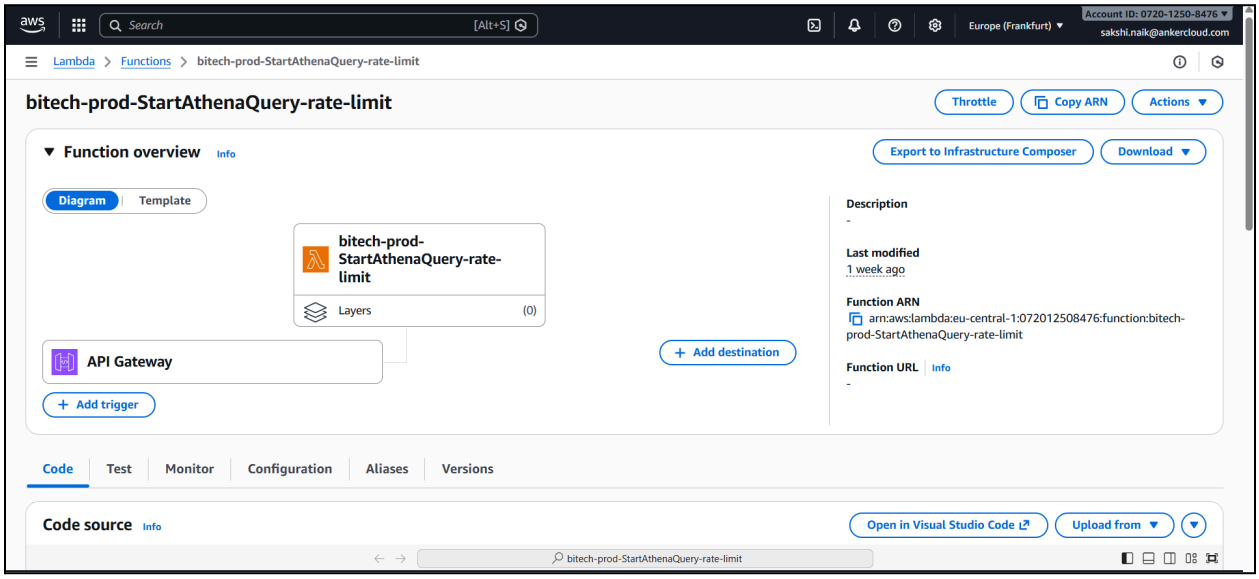
So the question is Why asynchronous it is because API Gateway has a 29-second timeout limit, which is insufficient for complex Athena queries. Using two separate Lambdas allows:

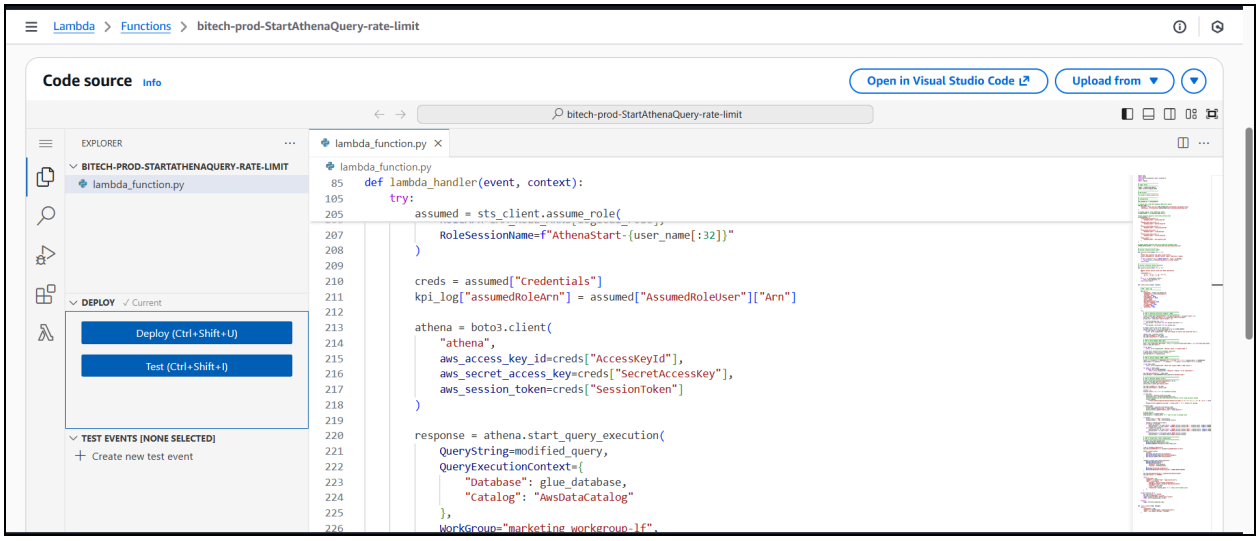
- Immediate query submission (StartAthenaQuery)
- Result retrieval at a later time (GetAthenaResults)
- Avoiding timeout failures and improving user experience

8.1.1 Lambda 1 Code Link : bitech-prod-StartAthenaQuery-rate-limit

Code :
https://eu-central-1.console.aws.amazon.com/lambda/home?region=eu-central-1#/functions/bitech-prod-StartAthenaQuery-rate-limit?tab=code

Purpose:
This lambda Starts an Athena query asynchronously and returns a queryExecutionId for later retrieval, while enforcing Purpose based access control (PBAC) via Lake Formation permissions.





Workflow Steps :

- Identity & PBAC Check
    - Extracts user identity from Cognito claims.
    - Maps user groups to a logical role (analyst, marketing).
    - Ensures the user belongs to exactly one authorized role.
    - This mapping is critical because the IAM role assumed determines which Lake Formation tables the user can query.
- Parse SQL Request & Table Validation
    - Extracts SQL query from request body.
    - Extracts tables from FROM / JOIN clauses.
    - Validates that each table exists in the pre registered Glue resources.
    - Purpose: Only queries against Lake Formation managed tables are allowed, ensuring PBAC is enforced at the table level.
- Optional Dynamic Filters
    - Applies city_name or postal_code filters safely.

    - Sanitizes input and normalizes German characters (ä, ö, ü, ß).
    - Injects filters into query safely while respecting existing WHERE/LIMIT clauses.
    - This ensures that user provided filters cannot bypass PBAC restrictions.
- Assume Role & Start Query
    - Uses STS to assume the IAM role mapped to the user's logical role.
    - Role assumption ensures that Athena executes queries with only the permissions granted by Lake Formation.
    - Starts Athena query asynchronously.
    - Logs the queryExecutionId and query metadata for auditing.
- KPI / Audit Logging
    - Logs user, query, tables, filters, and status.
    - Enables monitoring of PBAC compliance and query execution patterns.

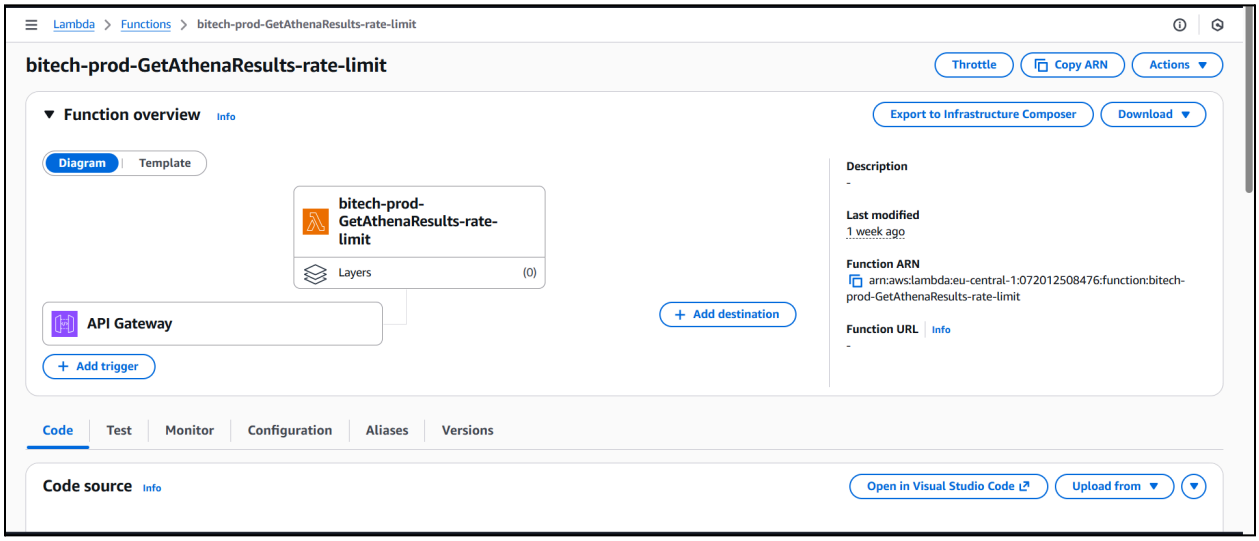
Key Lake Formation Points:

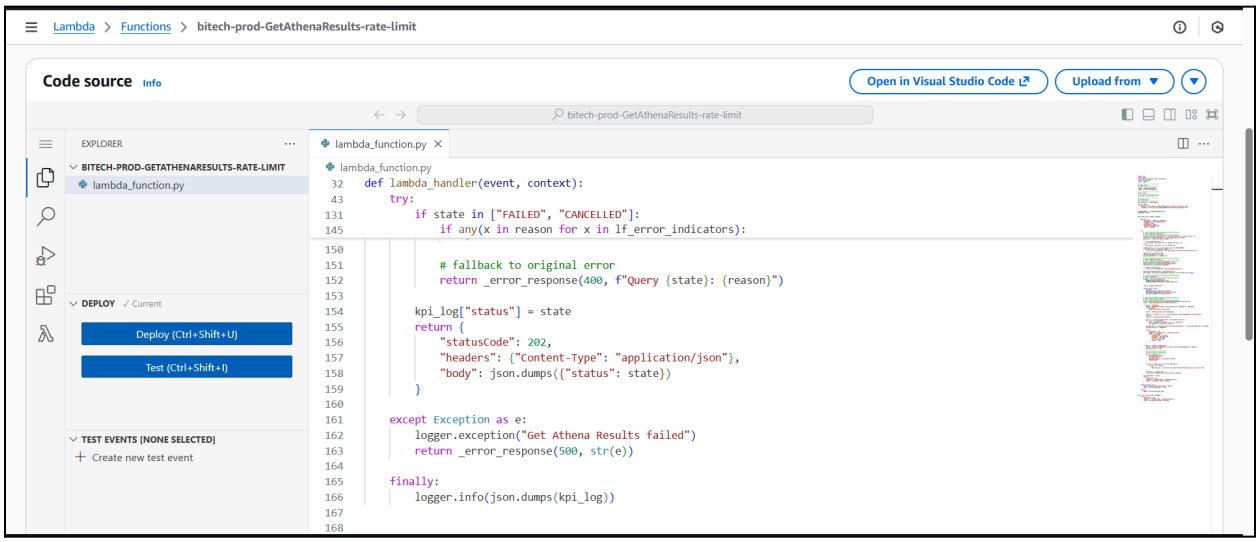
- Access control is enforced via IAM role assumption.
- Only registered Glue tables are queryable.
- Query execution happens under Lake Formation-governed permissions, preventing unauthorized data access.
- Filters prevent SQL injection while maintaining PBAC constraints.

8.1.2 Lambda 2 Code : bitech-prod-GetAthenaResults-rate-limit

Code:
https://eu-central-1.console.aws.amazon.com/lambda/home?region=eu-central-1#/functions/bitech-prod-GetAthenaResults-rate-limit?tab=code

Purpose:
This lambda Retrieves results of a previously submitted Athena query, maintaining Lake Formation based access control.





Workflow Steps:

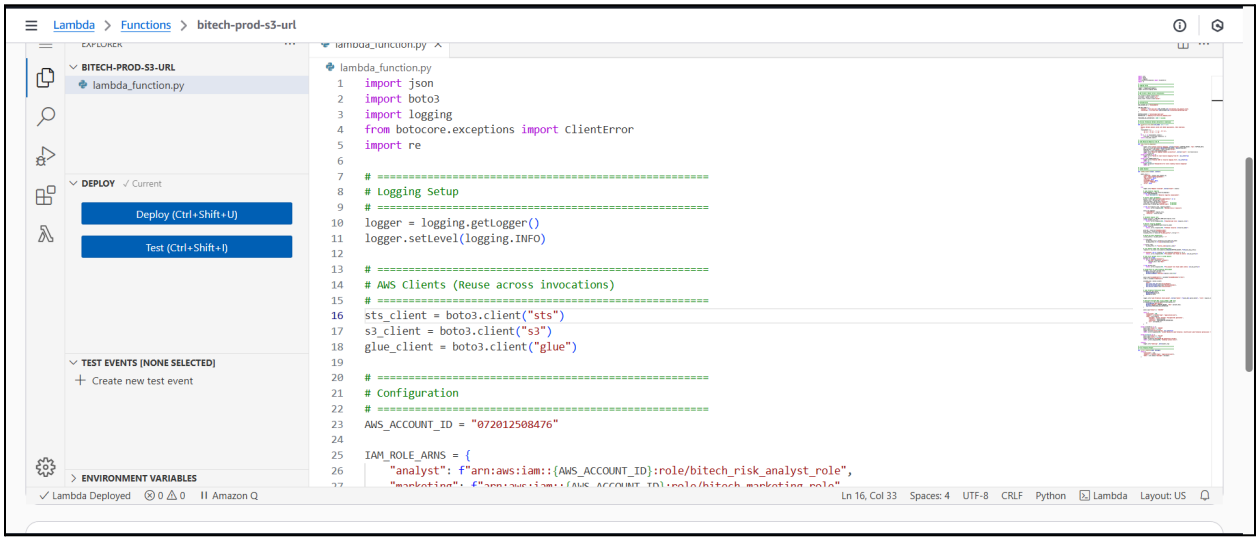
- Identity & PBAC Check
    - Similar to Lambda 1: maps Cognito claims to a logical role.
    - Ensures the user belongs to exactly one authorized role.
    - This ensures that the results fetched respect the Lake Formation permissions associated with that role.

- Assume Role for Result Retrieval
    - Uses STS to assume the IAM role corresponding to the logical role.
    - Athena client operates under this role, so only query results the user is authorized to see are accessible.
- Query Status Check & Result Retrieval
    - Checks Athena query execution state (RUNNING, SUCCEEDED, FAILED, CANCELLED).
    - On success, returns results in pages (PAGE_SIZE = 1000) with pagination token handling.
- Lake Formation-Friendly Error Handling
    - Detects errors such as:
        - AccessDeniedException
        - Relation contains no accessible columns
        - UNAUTHORIZED
    - Returns friendly error messages to indicate Lake Formation permission restrictions rather than generic Athena errors.
- KPI / Audit Logging
    - Logs user, role, queryExecutionId, and query status.
    - Provides visibility into PBAC enforcement and compliance.

Key Points:

- Access to query results is strictly controlled via the IAM role assumed. Users cannot bypass Lake Formation restrictions unauthorized tables or columns result in a friendly access denied message.
- Supports secure pagination while respecting PBAC.

8.2 S3 Based Data Access Approach (S3 presigned URL : Data Download)



8.2.1 Lambda 3 Code : bitech-prod-s3-url

Code :
https://eu-central-1.console.aws.amazon.com/lambda/home?region=eu-central-1#/functions/bitech-prod-s3-url?tab=code

This Lambda generates a presigned S3 URL for accessing parquet files while enforcing purpose based access control (PBAC) through Lake Formation permissions. It first validates the user's logical role (e.g., analyst or marketing) and ensures the requested resource exists in the registered resource mapping stored in S3. Optional filters like city_name and postal_code are normalized and appended to the S3 key prefix, ensuring access is only to authorized data partitions.

Access enforcement is performed by assuming the IAM role associated with the logical role and calling **glue.get_table()** on the target Glue table. This step ensures the user has Lake Formation permissions for the table. If the permission check fails, the Lambda returns a clear "Access Denied" error, preventing unauthorized data access while maintaining PBAC compliance.

Once the Lake Formation check passes, the Lambda generates a time-limited presigned URL for the first available parquet file in the specified S3 prefix. All operations, including assumed roles, resource accessed, and final status, are logged for audit and compliance purposes, providing end-to-end traceability of PBAC enforcement from identity validation to S3 access.

9. API Gateway and AWS cognito

API Gateway and AWS Cognito together form the authentication and request entry layer in Bitech PBAC platform.
They ensure that only authenticated users can invoke Lambda functions and that user identity and role context is securely propagated downstream for Lake Formation based authorization. API Gateway acts as the single external interface to the platform, while Cognito provides identity verification and group-based role mapping. This design ensures that Lambda functions never need to handle authentication logic directly and can focus solely on authorization and execution.

9.1 Amazon API Gateway Set up

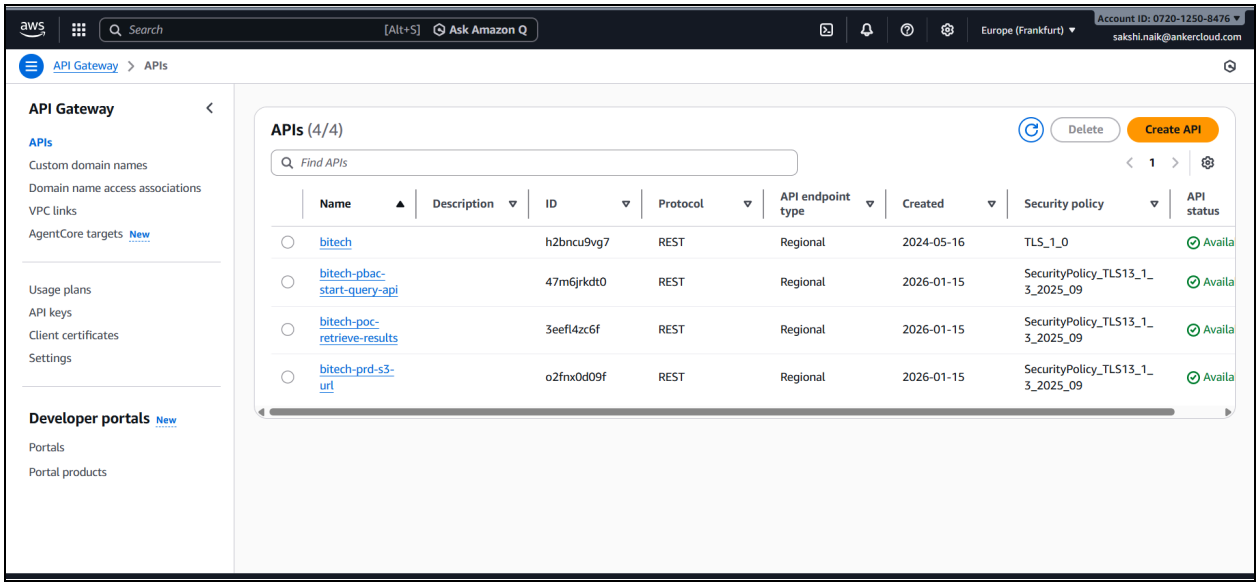
Amazon API Gateway here serves as the controlled entry point for all data access operations in the platform. Every request to Athena-based queries or S3 based downloads is routed through API Gateway before reaching Lambda.

So the Key responsibilities the current set up of API Gateway includes:

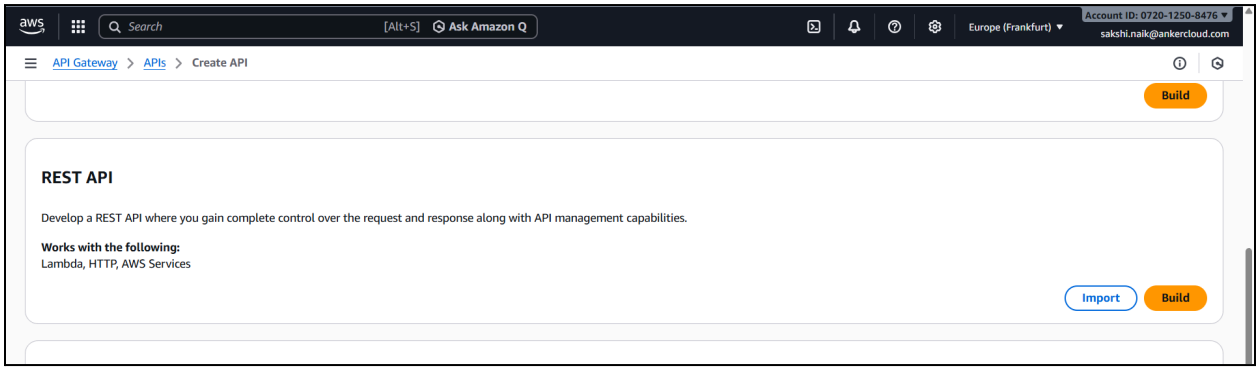
- Exposing REST APIs for:
    - Starting Athena queries
    - Fetching Athena results
    - Generating S3 presigned URLs
- JWT token id validation using Amazon Cognito authorizers
- Rate limiting and throttling to protect backend services
- Request forwarding with full identity context to Lambda

9.1.1 Steps to Configure API Gateway with Rate Limiting

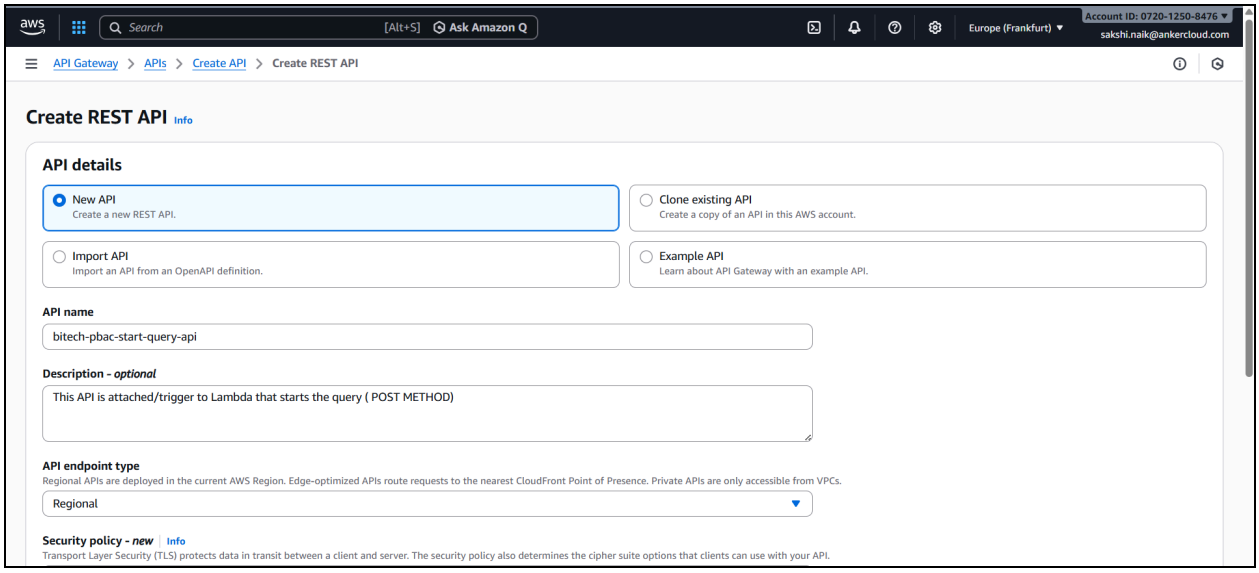
- Go to AWS API Gateway (https://eu-central-1.console.aws.amazon.com/apigateway/main/apis?region=eu-central-1#) → Click Create API



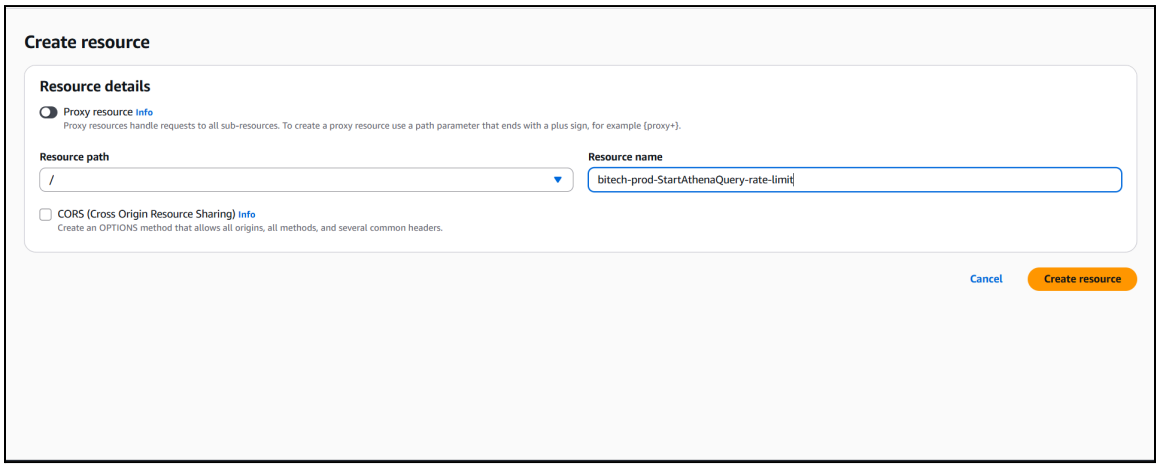
- Then click REST API → Build

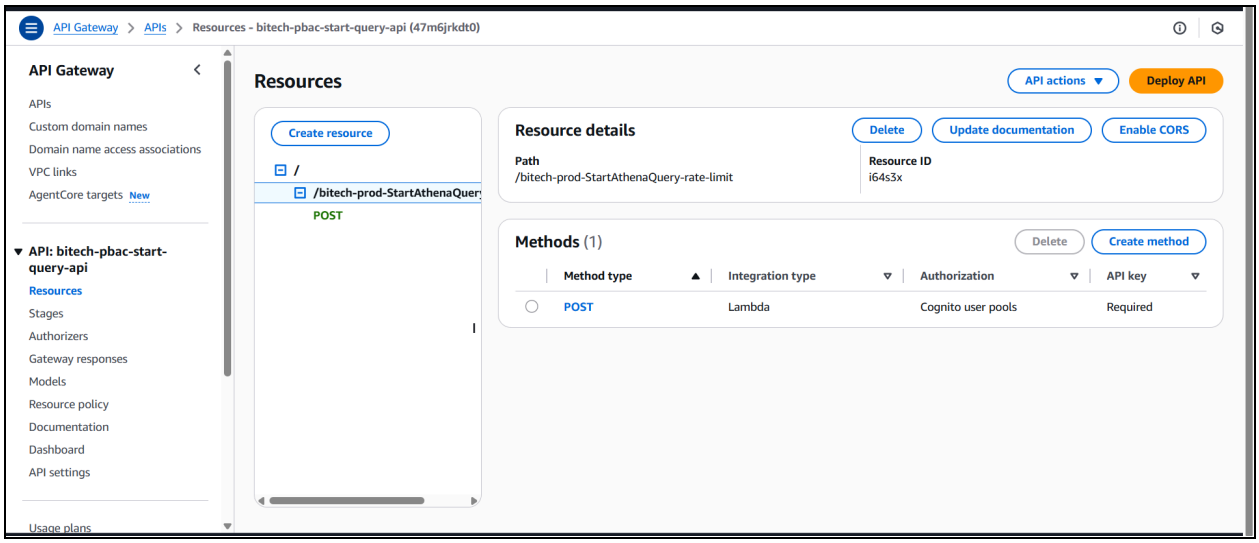


- Click New API → Name the API (here as you can see in screenshot bitech-pbac-start-query-api) →Then API endpoint type select as Regional → Select necessary Security policy (refer Screenshot for rest of configuration below) → Click Create API.

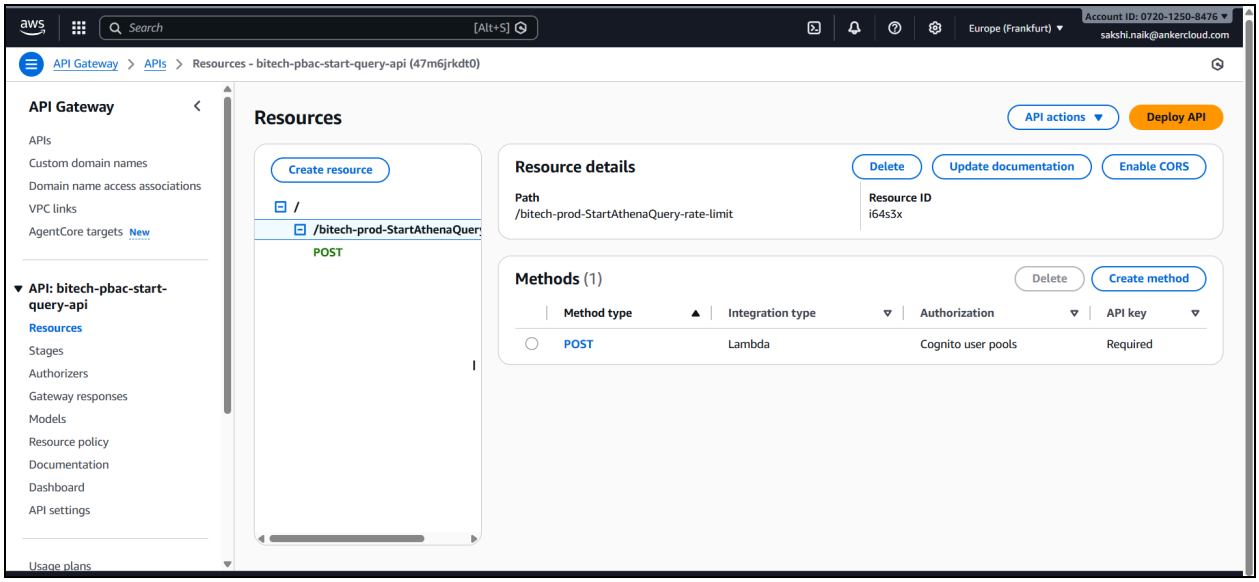


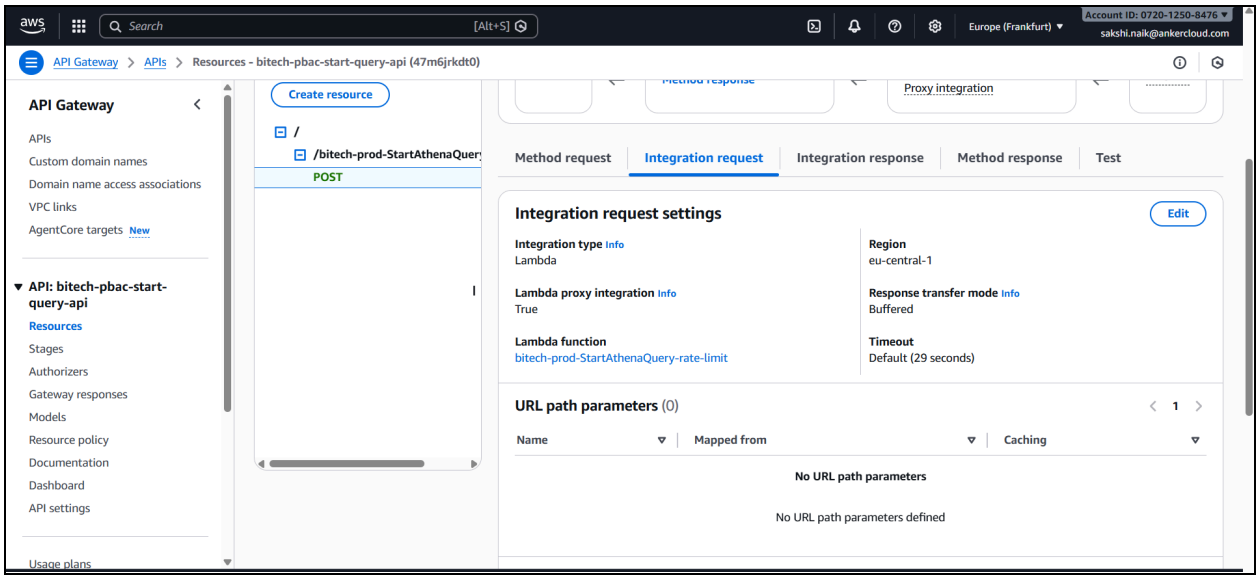
- Click Resource → Give a Resource Name → Create Resource.

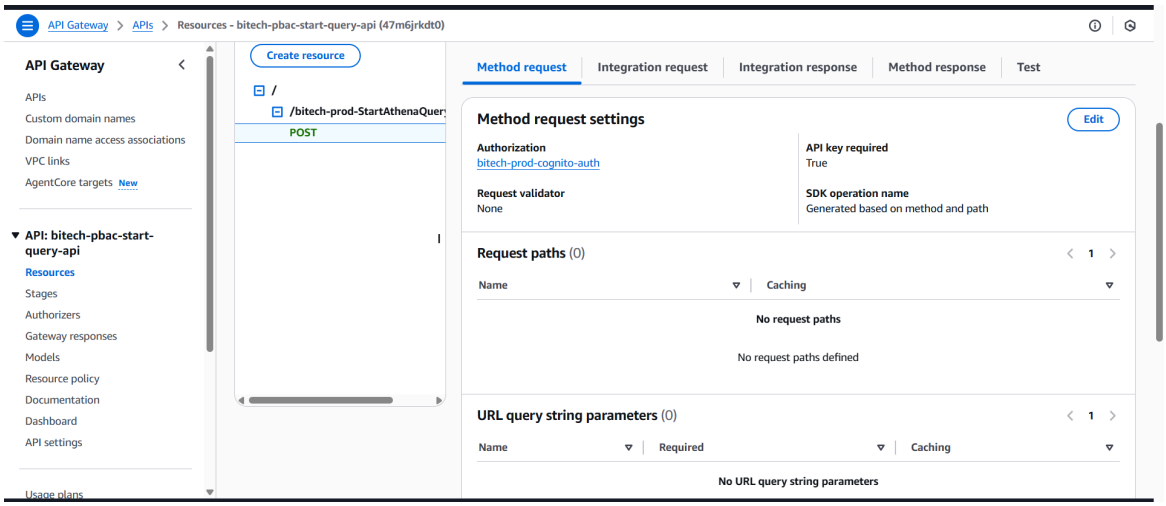




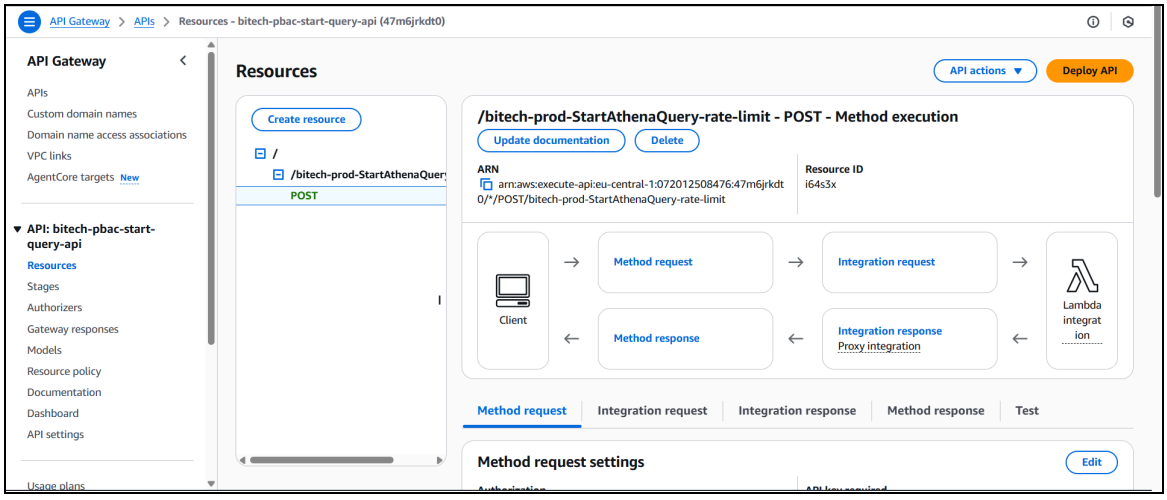
- Now Create Method → Select Method Type → POST for start query and GET to retrieve results.
- Select the integration type as Lambda Function, enable Lambda proxy integration, and choose the Lambda function that should be triggered by the API.
- In Method request settings → Select the Cognito user Authorizer and then → Check API key required.

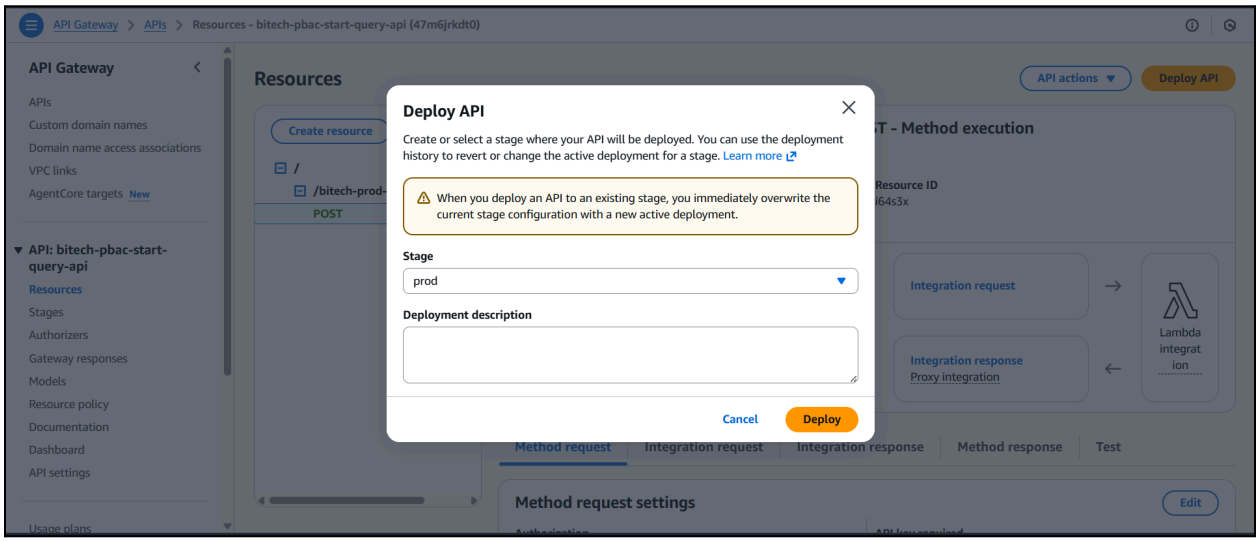




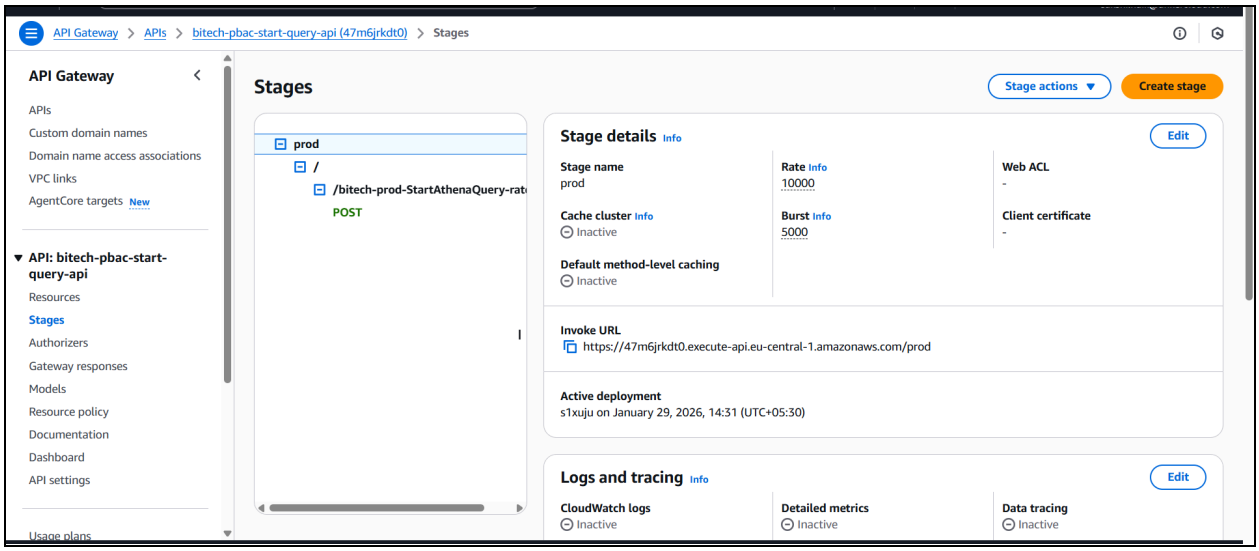


- Then click Deploy API → Create a New Stage → prod/test according to you → Deploy



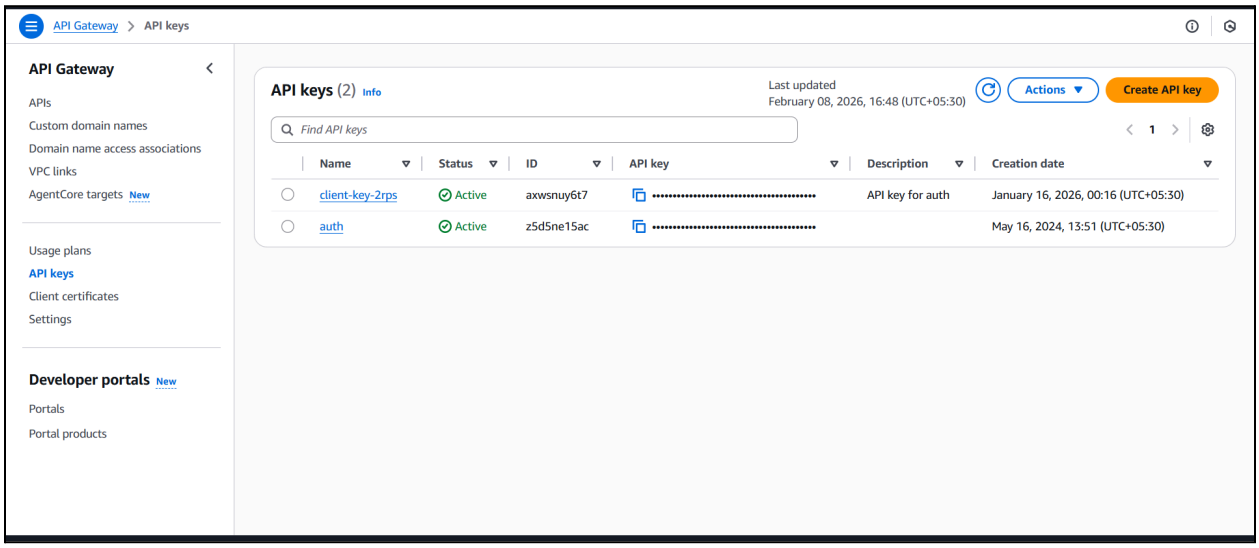


- In stages you see like below image → Invoke URL is generated

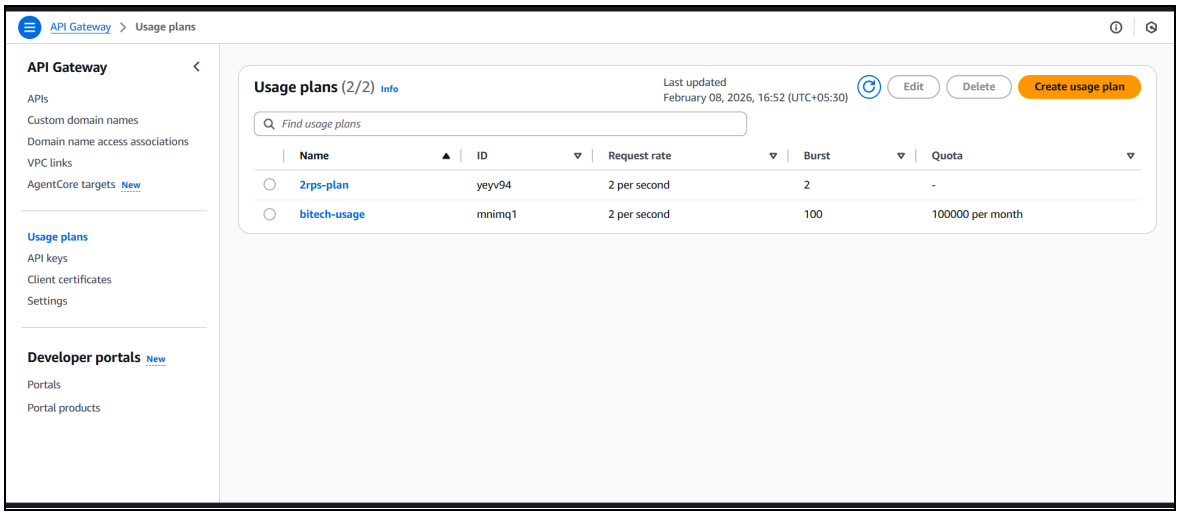


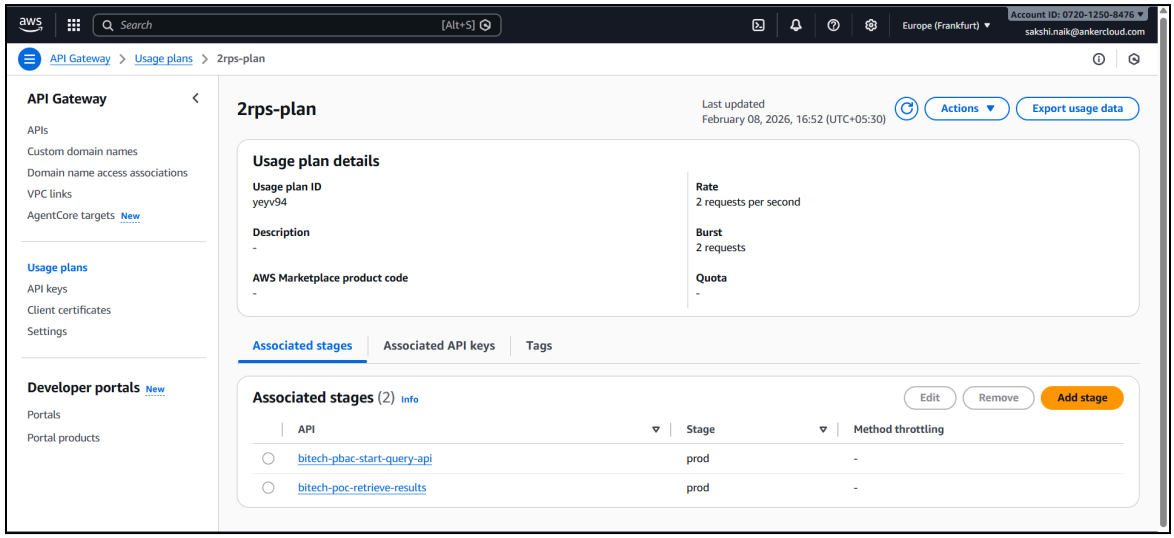
Steps to Configure Rate Limiting (2 RPS) Using API Gateway

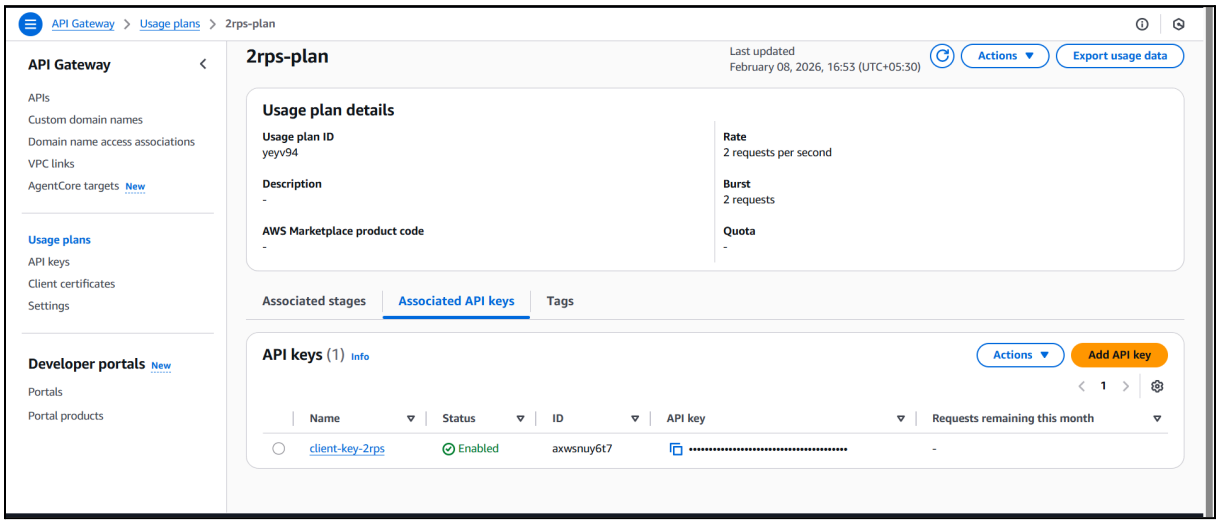
- Create one API key for the client. Go to API Gateway → API Keys, click Create API Key, set the name as client-key-2rps, and save it.



- Create one usage plan with a 2 requests-per-second limit. Navigate to API Gateway → Usage Plans → Create, name the plan 2rps-plan, set throttling with Rate = 2 and Burst = 2, then save. Optionally, configure a daily quota such as 10,000 requests per day.


API Gateway > Usage plans
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets New
Usage plans
API keys
Client certificates
Settings
Developer portals New
Portals
Portal products
Usage plans (2/2) Info
Last updated
February 08, 2026, 16:52 (UTC+05:30)
Edit
Delete
Create usage plan
Find usage plans
Name
ID
Request rate
Burst
Quota
2rps-plan
yeyv94
2 per second
2
-
bitech-usage
mnimq1
2 per second
100
100000 per month


Search
[Alt+S]
Europe (Frankfurt)
Account ID: 0720-1250-8476
sakshi.naik@ankercloud.com
API Gateway > Usage plans > 2rps-plan
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets New
Usage plans
API keys
Client certificates
Settings
Developer portals New
Portals
Portal products
2rps-plan
Last updated
February 08, 2026, 16:52 (UTC+05:30)
Actions
Export usage data
Usage plan details
Usage plan ID
yeyv94
Description
-
AWS Marketplace product code
-
Rate
2 requests per second
Burst
2 requests
Quota
-
Associated stages
Associated API keys
Tags
Associated stages (2) Info
Edit
Remove
Add stage
API
Stage
Method throttling
bitech-pbac-start-query-api
prod
-
bitech-poc-retrieve-results
prod
-


API Gateway > Usage plans > 2rps-plan
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets New
Usage plans
API keys
Client certificates
Settings
Developer portals New
Portals
Portal products
2rps-plan
Last updated
February 08, 2026, 16:53 (UTC+05:30)
Actions
Export usage data
Usage plan details
Usage plan ID
yeyv94
Description
-
AWS Marketplace product code
-
Rate
2 requests per second
Burst
2 requests
Quota
-
Associated stages
Associated API keys
Tags
API keys (1) Info
Actions
Add API key
Name
Status
ID
API key
Requests remaining this month
client-key-2rps
Enabled
axwsnuy6t7
-

- Add both APIs to the usage plan. Each REST API has its own stage and both must be attached. Open the usage plan, select Add API Stage, choose the API bitech-pbac-start-query-api and its stage (dev/test/prod), then add another API stage for bitech-poc-retrieve-results using the same stage.

- Attach the API key to the usage plan. Inside the usage plan, go to API Keys → Add API Key and select client-key-2rps.

- Deploy both APIs → Open each API individually → go to Actions → Deploy API → select the appropriate stage and deploy.

After completing these steps, both APIs will share the same usage plan and enforce a combined limit of 2 requests per second per API key. Any client exceeding this limit will receive a 429 Too Many Requests response. Clients must include the API key in every request using the x-api-key header.

Similarly do all the steps for bitech-poc-retrieve-results (https://eu-central-1.console.aws.amazon.com/apigateway/main/apis/3eefl4zc6f/resources?api=3eefl4zc6f&experience=rest&region=eu-central-1) → Remember to Deploy the API.

9.2 AWS Cognito Set up

AWS Cognito is used as the identity provider for the PBAC platform and handles user authentication using email-based login. All users are created and managed within a single Cognito User Pool, providing a centralized and scalable identity store for the platform.

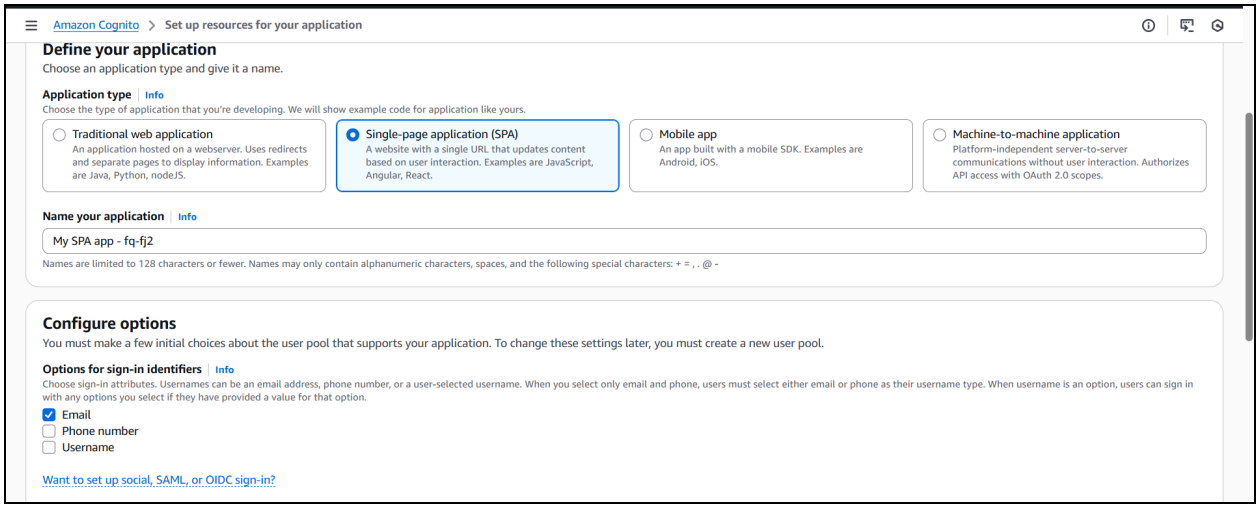
Currently, two Cognito groups are configured to represent business purposes:

- analyst
- marketing

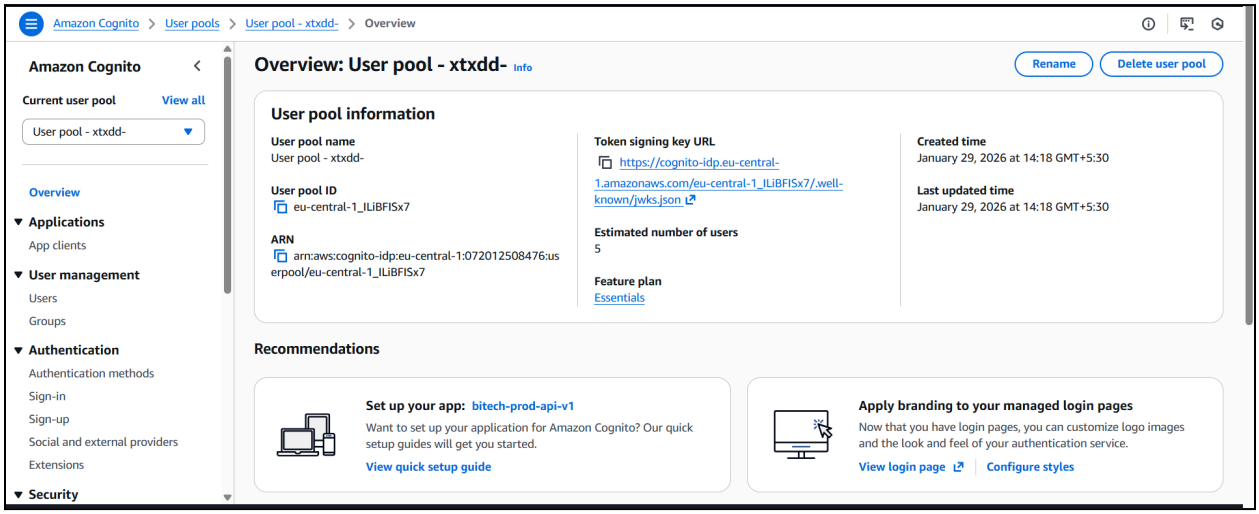
Multiple users can be added to the Cognito User Pool and assigned to one of these groups. Group membership defines the user's logical role, which is later used for purpose-based access control. The design supports easy onboarding of new users without any changes to Lambda, API Gateway, and Lake Formation policies.

9.2.1 Configure Amazon Cognito Authentication for a REST API

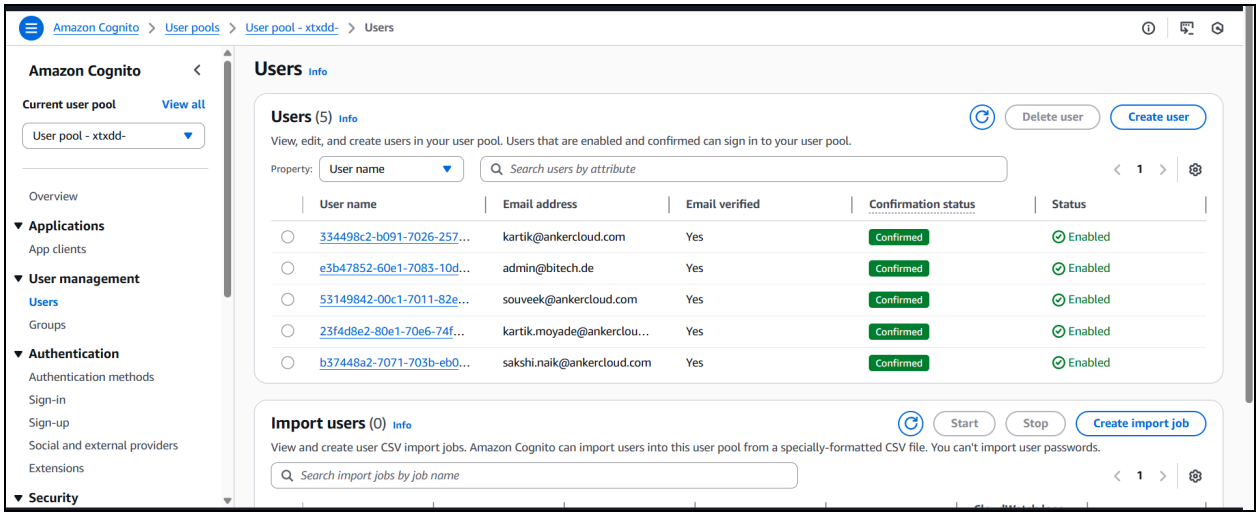
- Go to Amazon Cognito → Create user pool



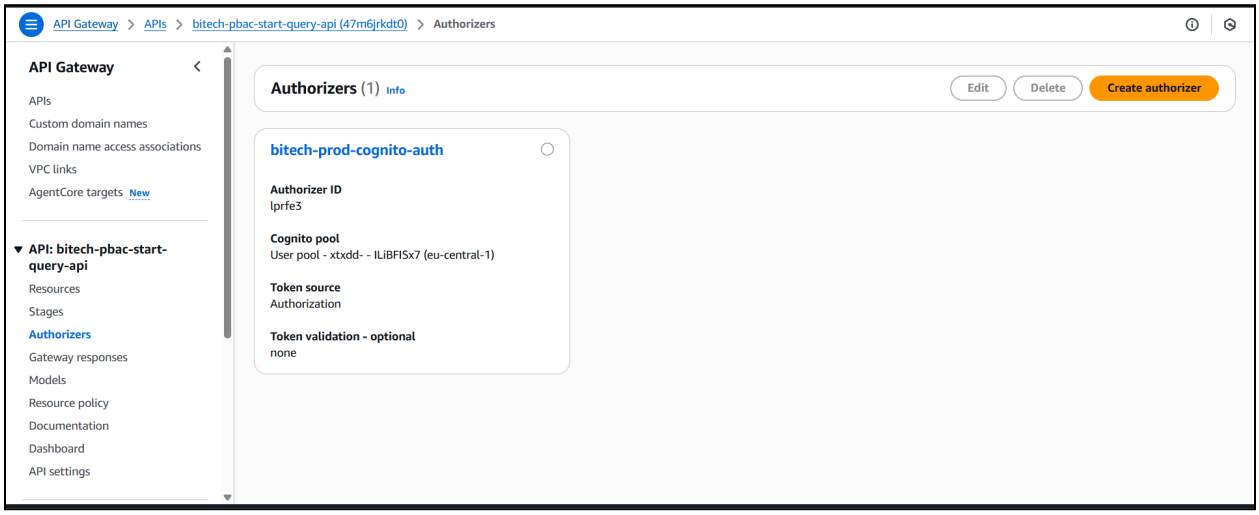
- Select Email as the sign in option
- Set app client name to **bitech-prod-api-v1**
- Uncheck Generate client secret
- Review and create the user pool (e.g.,User pool - xtxdd-)



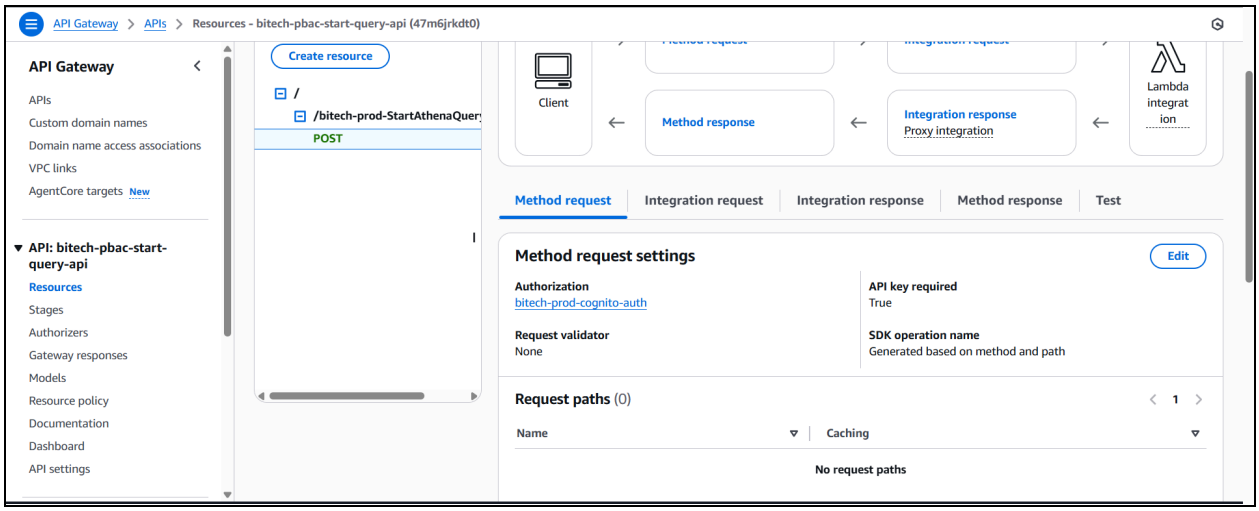
- Open the user pool and go to Groups
- Create a group named analyst & marketing
- Go to Users tab



- Create a user with a real email → Confirm users via email before login
- Open API Gateway and select your REST API



- Go to Authorizers → Create Authorizer → Set name to **bitech-poc-cognito-auth**
- Select Cognito as the authorizer type → Choose user pool - xtxdd-
- Set token source to Authorization → Create the authorizer → Go to Resources in the REST API → Select the endpoint → Click the POST method
- Open Method Request → Change Authorization from NONE to bitech-poc-cognito-auth → Remove userName and role from URL query parameters
- Click Actions → Deploy API → Select stage (e.g. prod) → Click Deploy

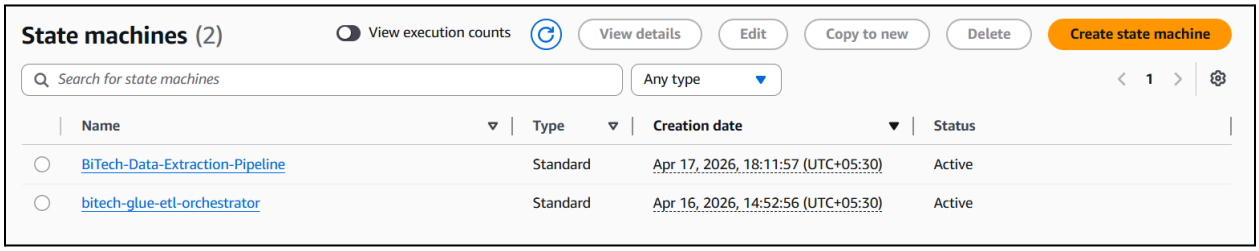


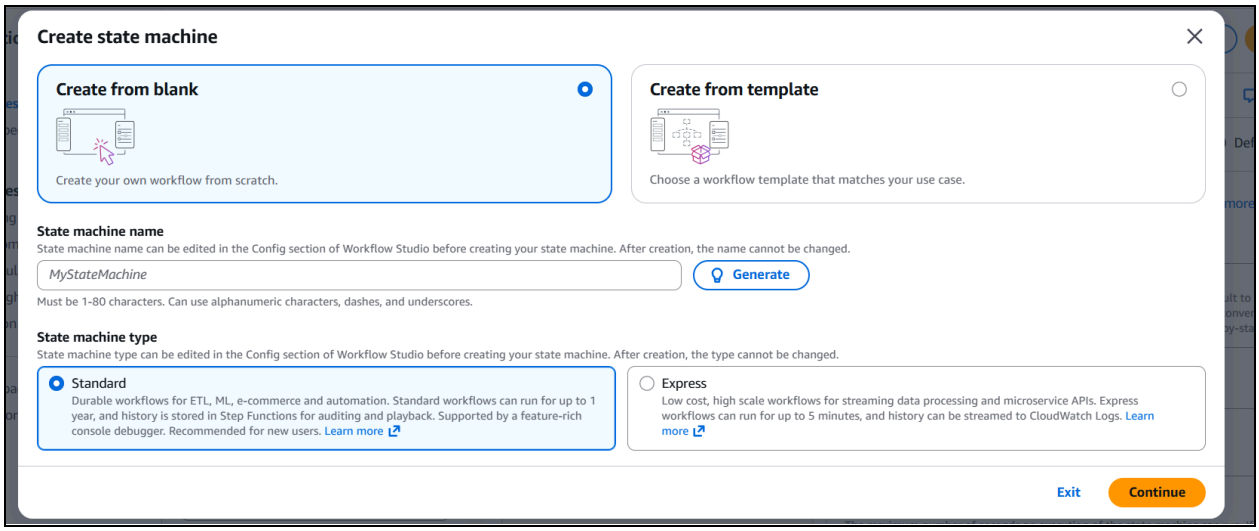
10. Step Function - Orchestration

To set up Step function we need to create State Machine

10.1 Steps to Create State Machine

- Go to Step Function → Create State Machine
- Name the State Machine → Standard → Continue





- Select Author with code snippets (or Write your workflow in code).
- Choose JSON as the definition language.
- Paste the Step Function JSON definition in the Code tab → Click Create.
- Open the created State Machine.
- Click Edit if further changes are required.
- Configure the IAM execution role with permissions for:
    - AWS Glue (`glue:StartJobRun`)
    - CloudWatch Logs
    - Any additional services used in the workflow.
- Save the changes.
- Click Start Execution to test the workflow.

This is an Explanation of our **AWS Step Functions workflow** that runs your ETL pipeline.

Step 10.1.1: Run Glue Jobs in Parallel

- The workflow starts at `RunGlueJobs`
- It runs 5 AWS Glue jobs at the same time (parallel execution):
    - Crime data job
    - Event data job

    - Weather data job
    - Policy data job
    - Traffic data job

Each job:

- Uses `glue:startJobRun.sync` → waits until job completes
- Has retry logic:
    - Retry up to 3 times
    - Wait 30 seconds between retries
    - Uses exponential backoff

Step 10.1.2: Start Glue Crawlers

- After ALL Glue jobs finish successfully
- It calls a Lambda function:
  `start-glue-crawlers`

Purpose:

- Starts AWS Glue Crawlers
- Crawlers update metadata/catalog tables after ETL

Step 10.1.3: Wait for Crawlers

- Waits for 60 seconds
- This gives crawlers time to start/process

Step 10.1.4: Check Crawler Status

- Calls another Lambda: `check-glue-crawlers-status`

This Lambda:

- Checks if all crawlers are finished
- Returns:

  { "all_done": true/false }

Step 10.1.5: Decision (Loop Until Done)

- A Choice state checks:
    - If `all_done = true` → go to success
    - Else → go back to Wait step (loop)

So basically:

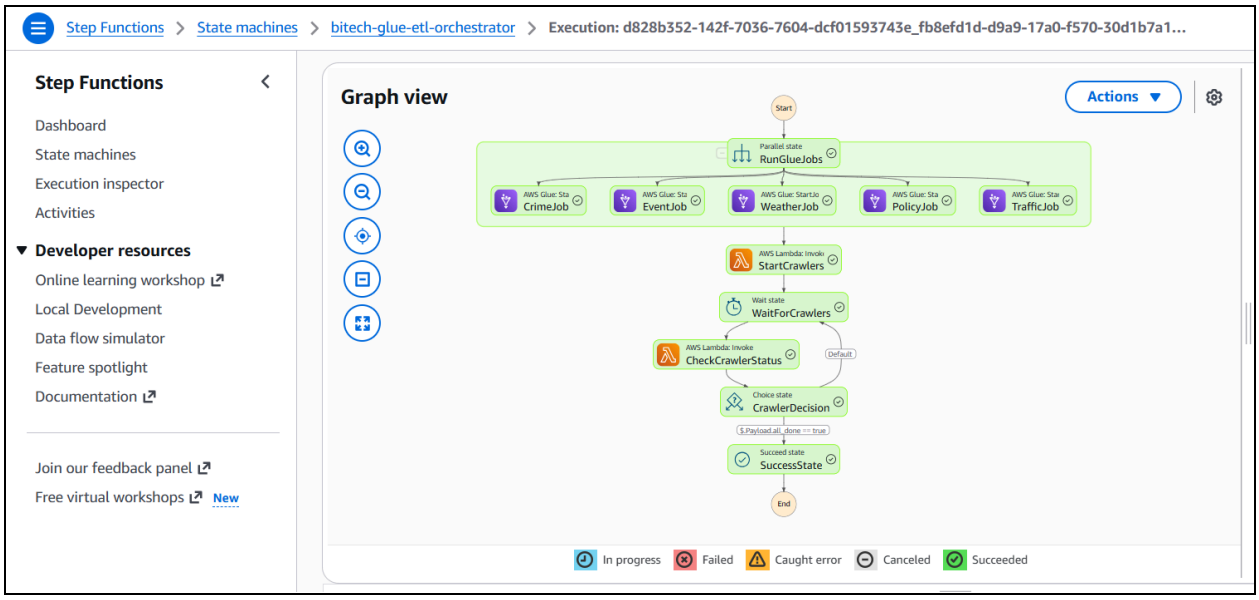
- Keep checking every 60 seconds
- Until crawlers finish

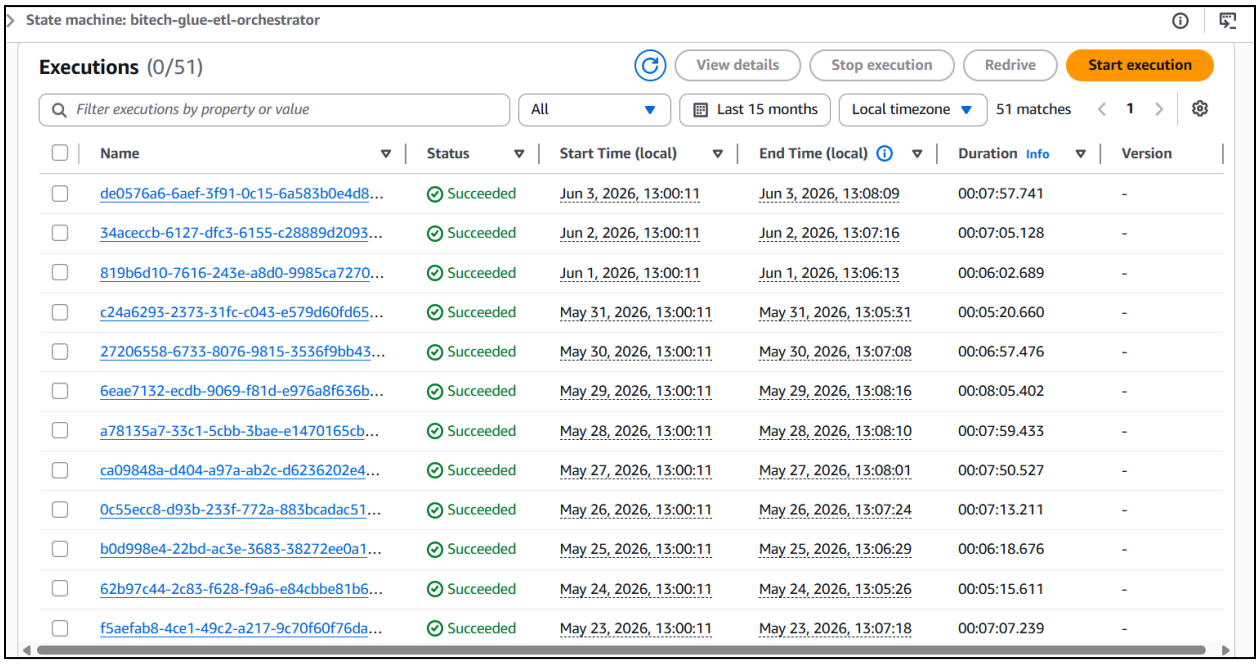
Step 10.1.6: Success

- When everything is done → workflow ends successfully

```
None

Run Glue Jobs (parallel)
        ↓
Start Crawlers
        ↓
Wait 60 sec
        ↓
Check status
        ↓
If not done → repeat wait
        ↓
If done → Success
```





10.2 Steps to Eventbridge

- Navigate to Amazon EventBridge.
- Go to Rules → Scheduled Rules (Legacy).
- Click Create Scheduled Rule.

- Enter the rule name: `bitech-daily-etl-trigger`.
- Configure the schedule using the cron expression: `30 7 * * ? *`.
- Select the ETL Step Function State Machine as the target.
- Configure the required IAM permissions.
- Review and create the rule.
- Verify that the rule status is Enabled.

10.2.1 EventBridge Rule Set Up

Rule Name: bitech-daily-etl-trigger

What it does:

- It automatically triggers your ETL pipeline every day

Schedule:

- Cron: `30 7 * * ? *`
- Runs at: 07:30 AM UTC daily (≈ 1:00 PM IST)

Purpose:

- No manual work needed
- Ensures **daily data processing pipeline runs automatically**

10.3 Step Function State Machine CODE :

```JSON
{
  "StartAt": "RunGlueJobs",
  "States": {
    "RunGlueJobs": {
      "Type": "Parallel",
      "Branches": [
        {
```

```
      "StartAt": "CrimeJob",
      "States": {
        "CrimeJob": {
          "Type": "Task",
          "Resource": "arn:aws:states:::glue:startJobRun.sync",
          "Parameters": {
            "JobName": "glue-etl-prd-crime-data-glue-bronze_to_silver"
          },
          "Retry": [
            {
              "ErrorEquals": [
                "States.TaskFailed"
              ],
              "IntervalSeconds": 30,
              "MaxAttempts": 3,
              "BackoffRate": 2
            }
          ],
          "End": true
        }
      }
    },
    {
      "StartAt": "EventJob",
      "States": {
        "EventJob": {
          "Type": "Task",
          "Resource": "arn:aws:states:::glue:startJobRun.sync",
          "Parameters": {
            "JobName": "glue-etl-prd-event-data-glue-bronze_to_silver"
          },
          "Retry": [
            {
              "ErrorEquals": [
                "States.TaskFailed"
              ],
              "IntervalSeconds": 30,
              "MaxAttempts": 3,
              "BackoffRate": 2
            }
          ],
          "End": true
        }
      }
    },
```

```
{
  "StartAt": "WeatherJob",
  "States": {
    "WeatherJob": {
      "Type": "Task",
      "Resource": "arn:aws:states:::glue:startJobRun.sync",
      "Parameters": {
        "JobName": "glue-prd-weather-data-bronze_silver_v2"
      },
      "Retry": [
        {
          "ErrorEquals": [
            "States.TaskFailed"
          ],
          "IntervalSeconds": 30,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "End": true
    }
  }
},
{
  "StartAt": "PolicyJob",
  "States": {
    "PolicyJob": {
      "Type": "Task",
      "Resource": "arn:aws:states:::glue:startJobRun.sync",
      "Parameters": {
        "JobName": "etl-job-policy-table"
      },
      "Retry": [
        {
          "ErrorEquals": [
            "States.TaskFailed"
          ],
          "IntervalSeconds": 30,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "End": true
    }
  }
```

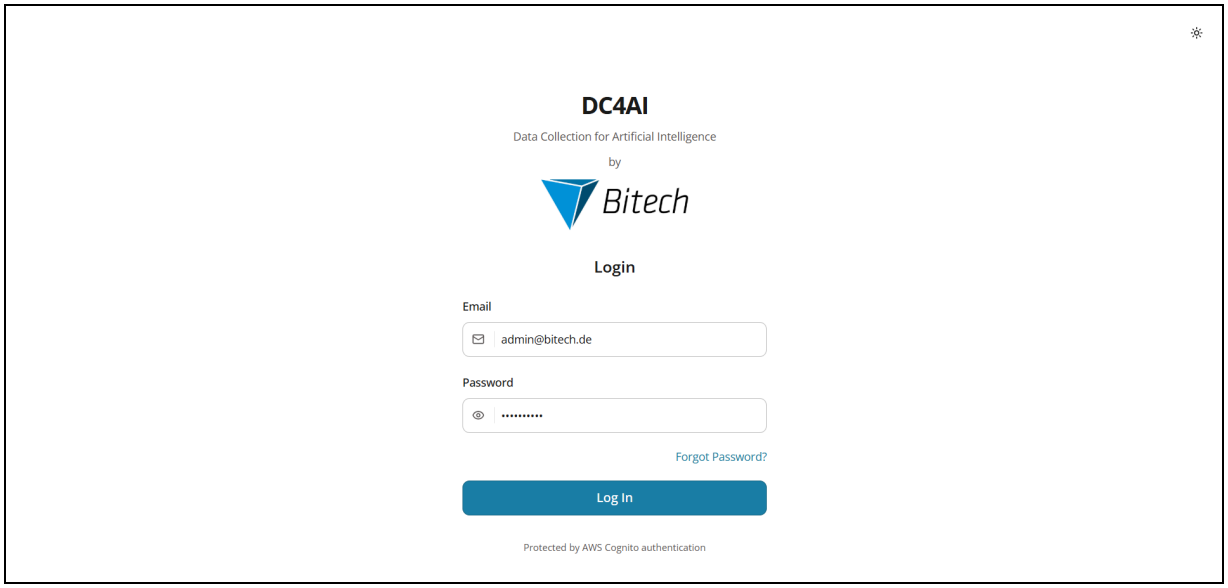
```
      },
      {
        "StartAt": "TrafficJob",
        "States": {
          "TrafficJob": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun.sync",
            "Parameters": {
              "JobName": "glue-etl-prd-traffic-data-glue-bronze_to_silver"
            },
            "Retry": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "IntervalSeconds": 30,
                "MaxAttempts": 3,
                "BackoffRate": 2
              }
            ],
            "End": true
          }
        }
      }
    ],
    "Next": "StartCrawlers"
  },
  "StartCrawlers": {
    "Type": "Task",
    "Resource": "arn:aws:states:::lambda:invoke",
    "Parameters": {
      "FunctionName": "start-glue-crawlers",
      "Payload": {}
    },
    "Next": "WaitForCrawlers"
  },
  "WaitForCrawlers": {
    "Type": "Wait",
    "Seconds": 60,
    "Next": "CheckCrawlerStatus"
  },
  "CheckCrawlerStatus": {
    "Type": "Task",
    "Resource": "arn:aws:states:::lambda:invoke",
    "Parameters": {
```

```
        "FunctionName": "check-glue-crawlers-status",
        "Payload": {}
      },
      "Next": "CrawlerDecision"
    },
    "CrawlerDecision": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.Payload.all_done",
          "BooleanEquals": true,
          "Next": "SuccessState"
        }
      ],
      "Default": "WaitForCrawlers"
    },
    "SuccessState": {
      "Type": "Succeed"
    }
  }
}
```

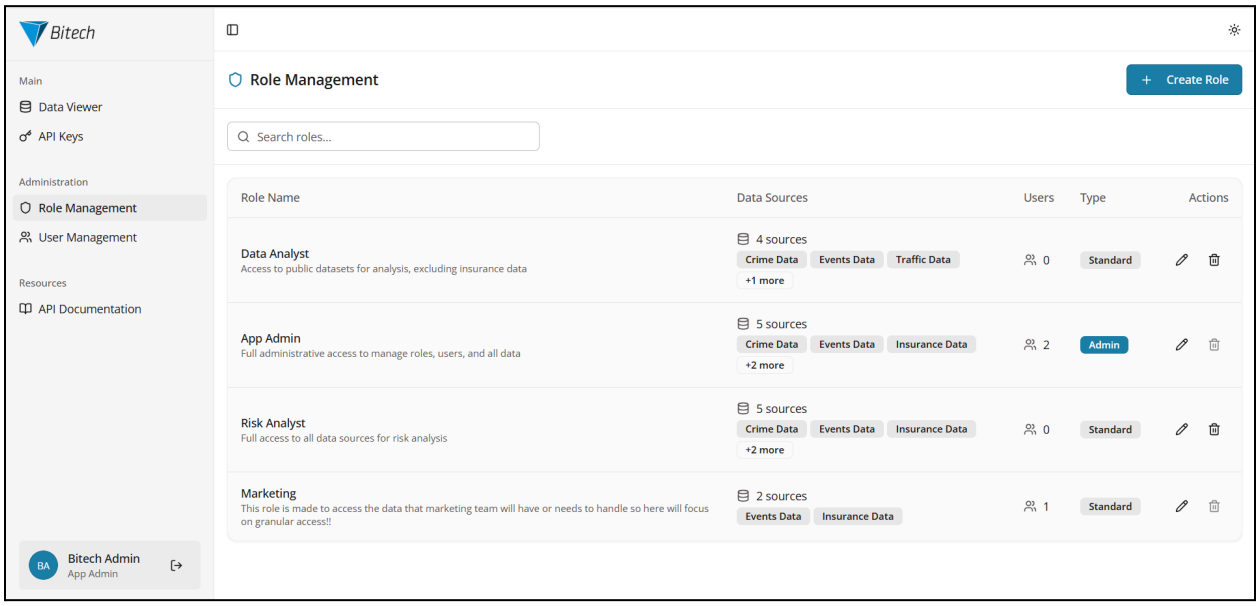
11. End-to-End Data Access Demo (UI Walkthrough)

This section demonstrates how to access data end-to-end through the UI, from login to final data retrieval, to help new users understand the complete flow.

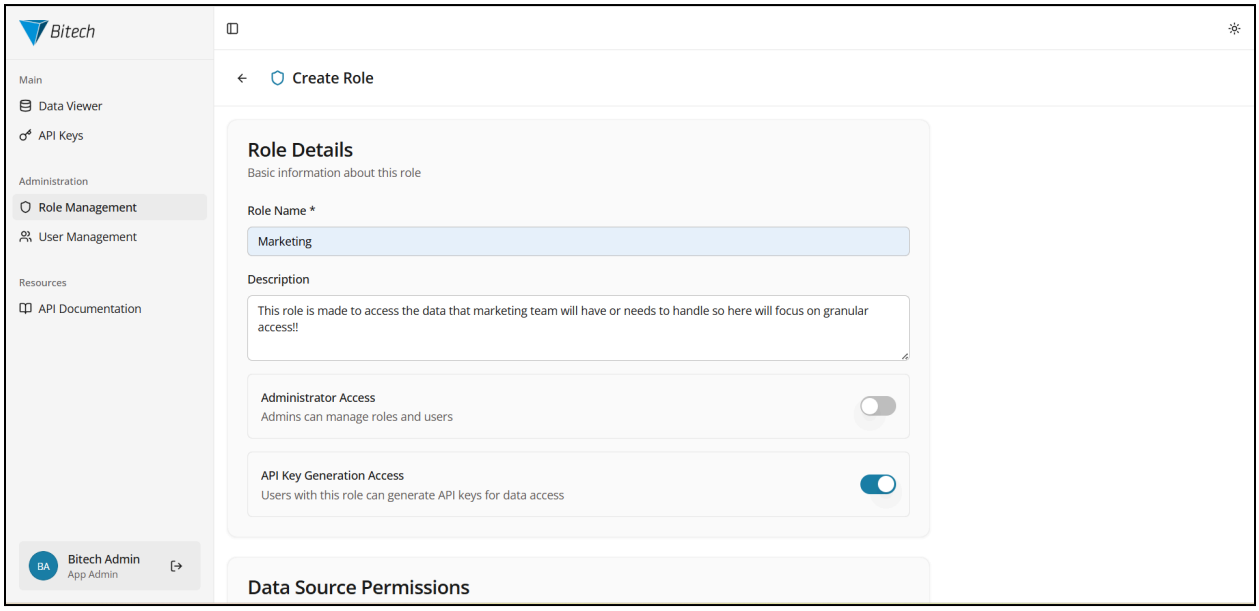
- User Login Page → Enter Email ID say example (serge.nsilu@bitech.de) and Password → Serge123@Bitech
  (This is the Admin as of now you can change this)



- Roles can be created and managed from the **Role Management** screen.
- Users can be created and assigned the required role from the **User Management** screen.
- **API keys** can be created for the logged in user to access data programmatically.
- Data access can be viewed in **Data Viewer** or accessed via Postman using APIs.

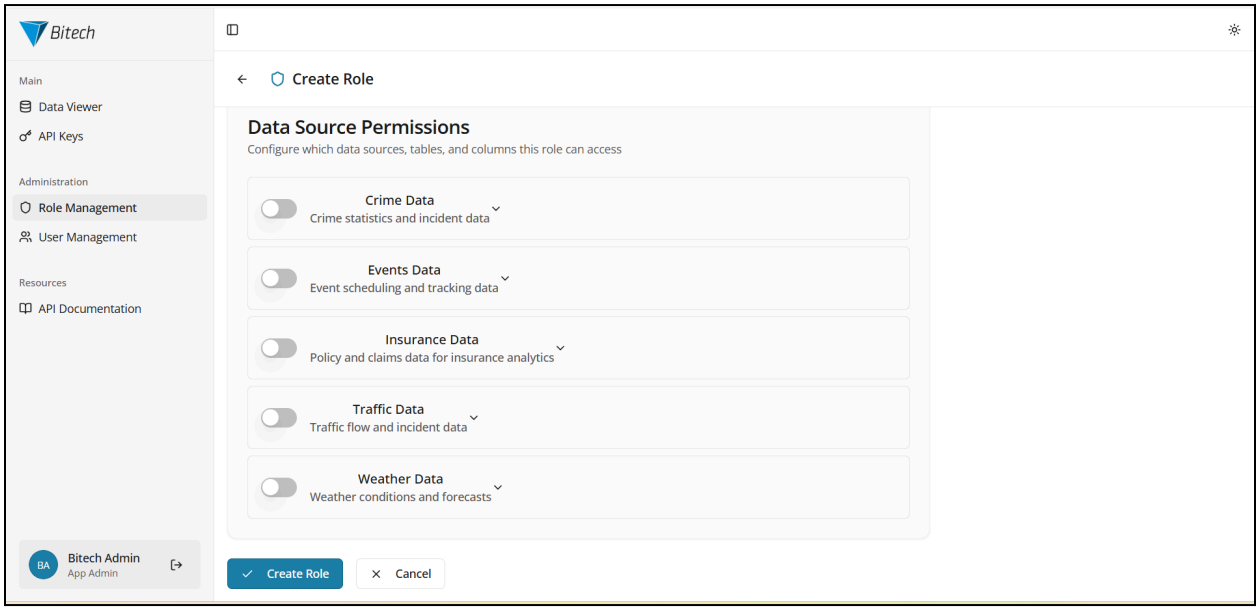


- Lets talk about Data granularity example → For that lets create a role that is Marketing → Always Generate API key so that we can see and access data.

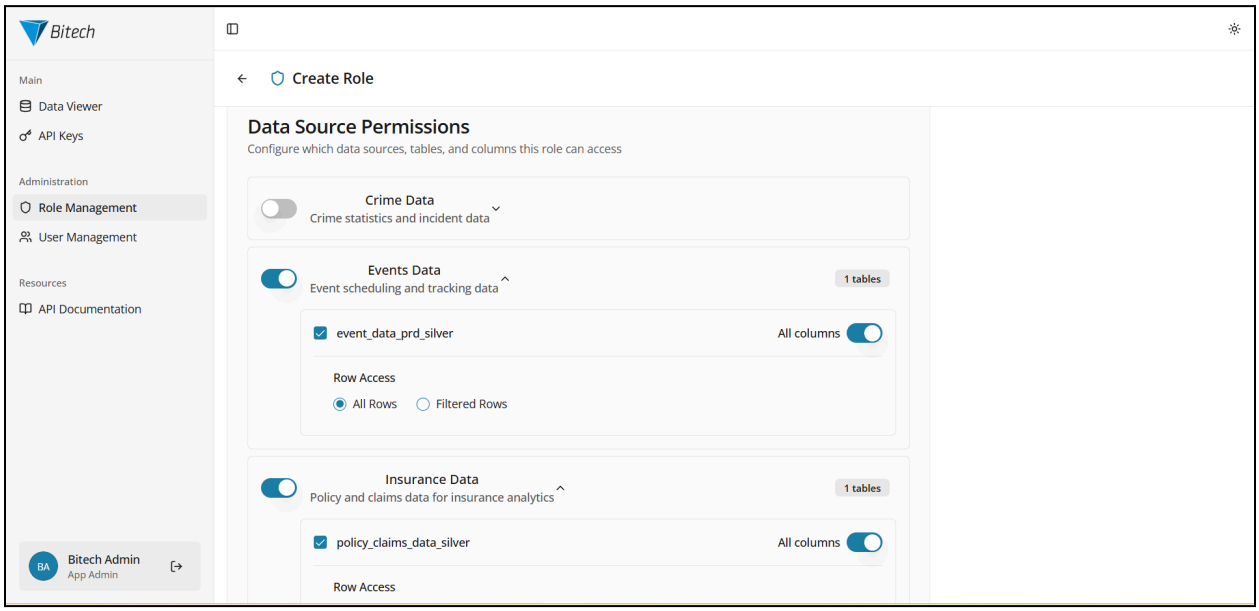


- Data Source Permissions is the core control point where Lake Formation access is enforced. At this step select the required data sources that the Marketing role is permitted to access defining the scope of datasets available to the role.

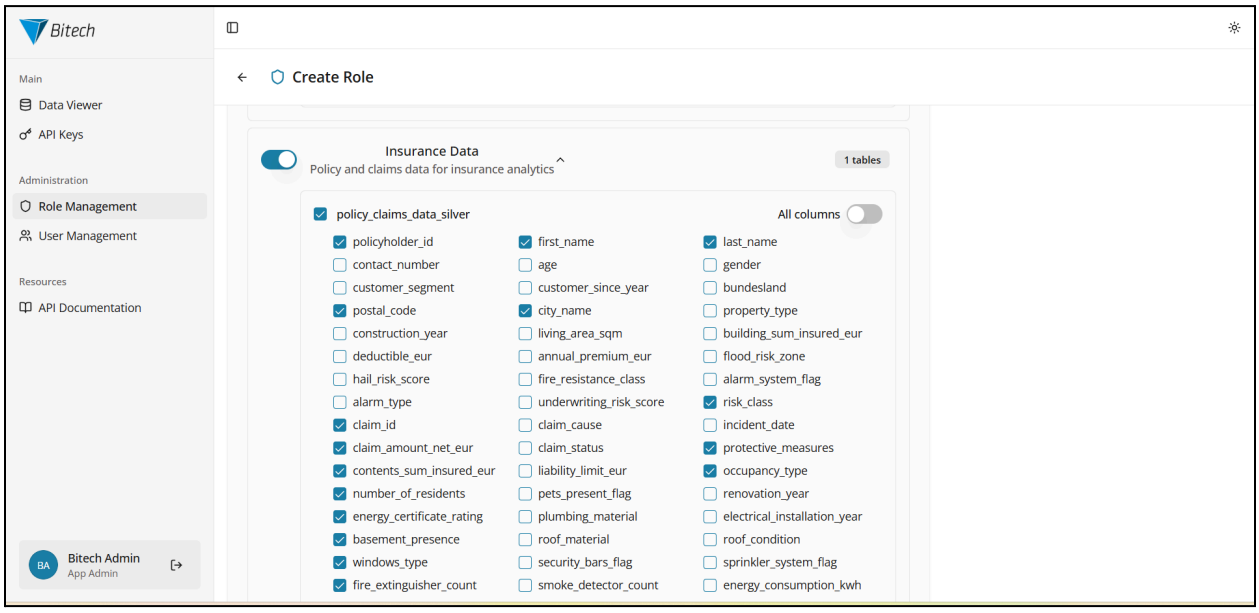
- So here I have selected 2 data sources → that is events and insurance data → here we can have full Table level and Column level access.



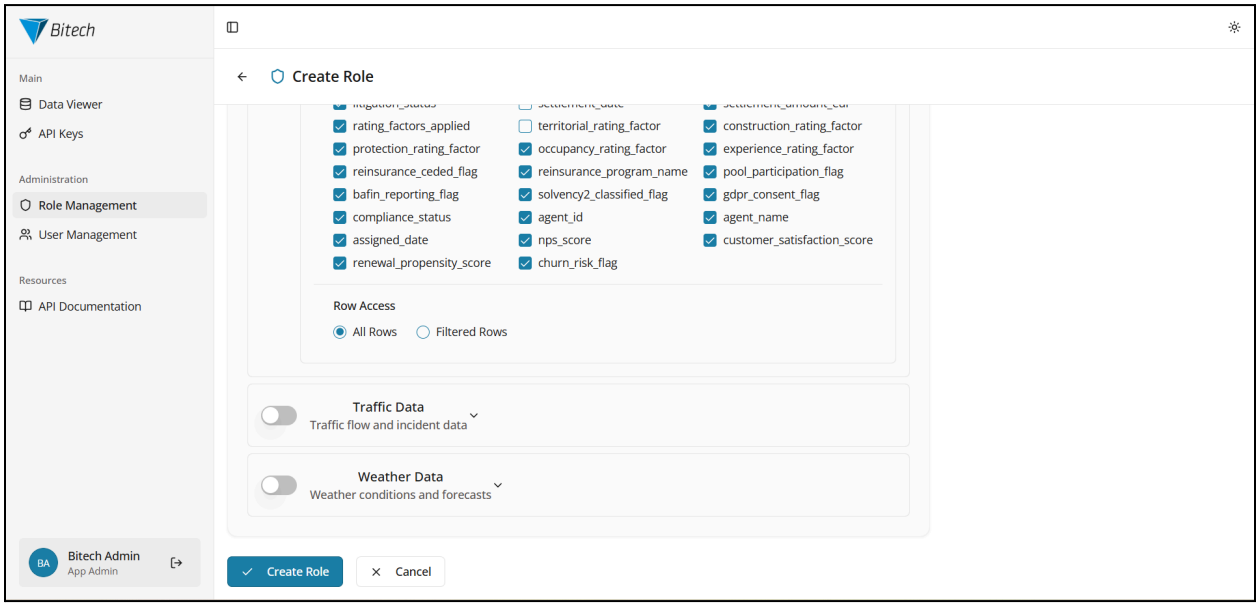
- For Column level access → we can Disable the All columns → And select what columns you want to filter or include in it so that that particular marketing role (which user is attached too) would see only that data.



- Like here in insurance data → Row access → All rows (We can also do row level filtering)



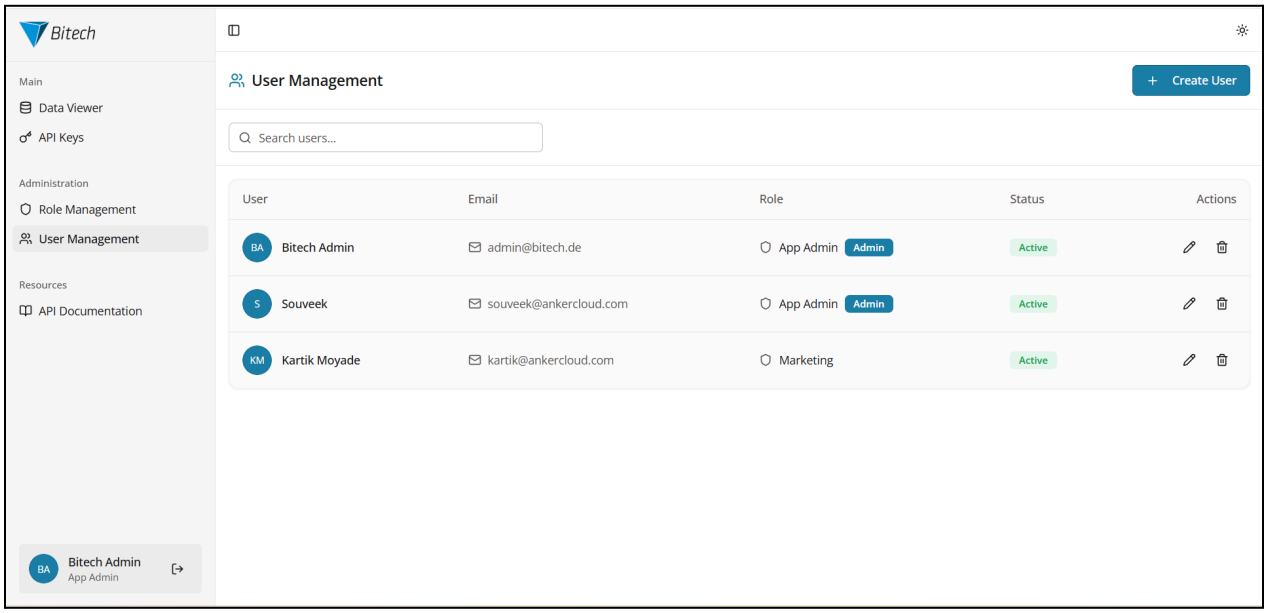
- Click Create Role.



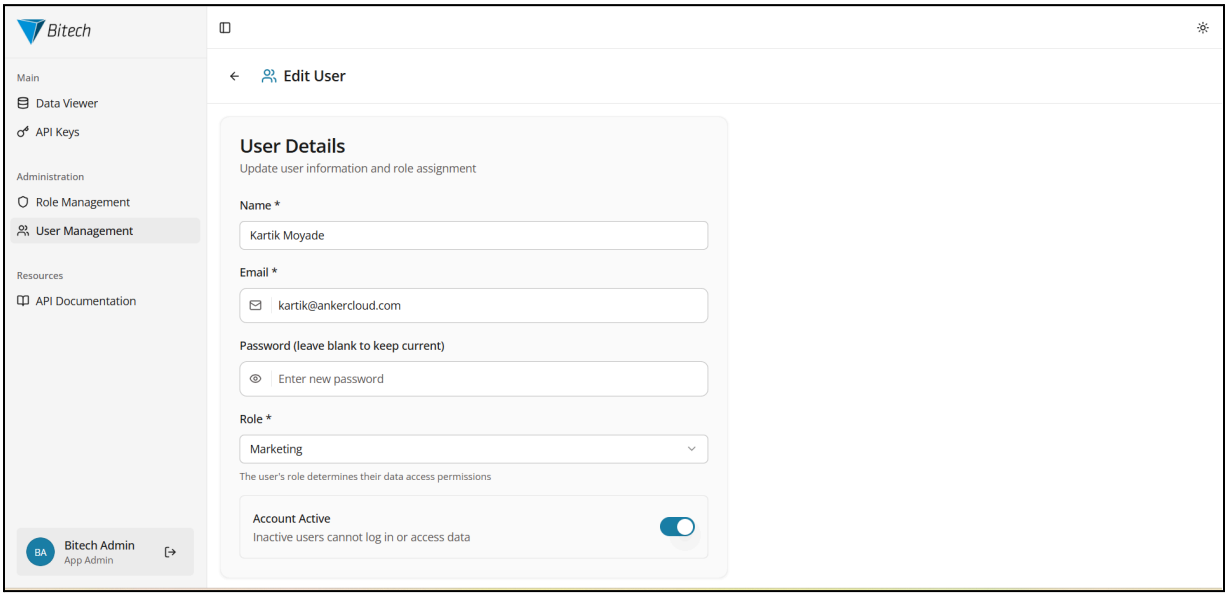
- This is how it looks once created.



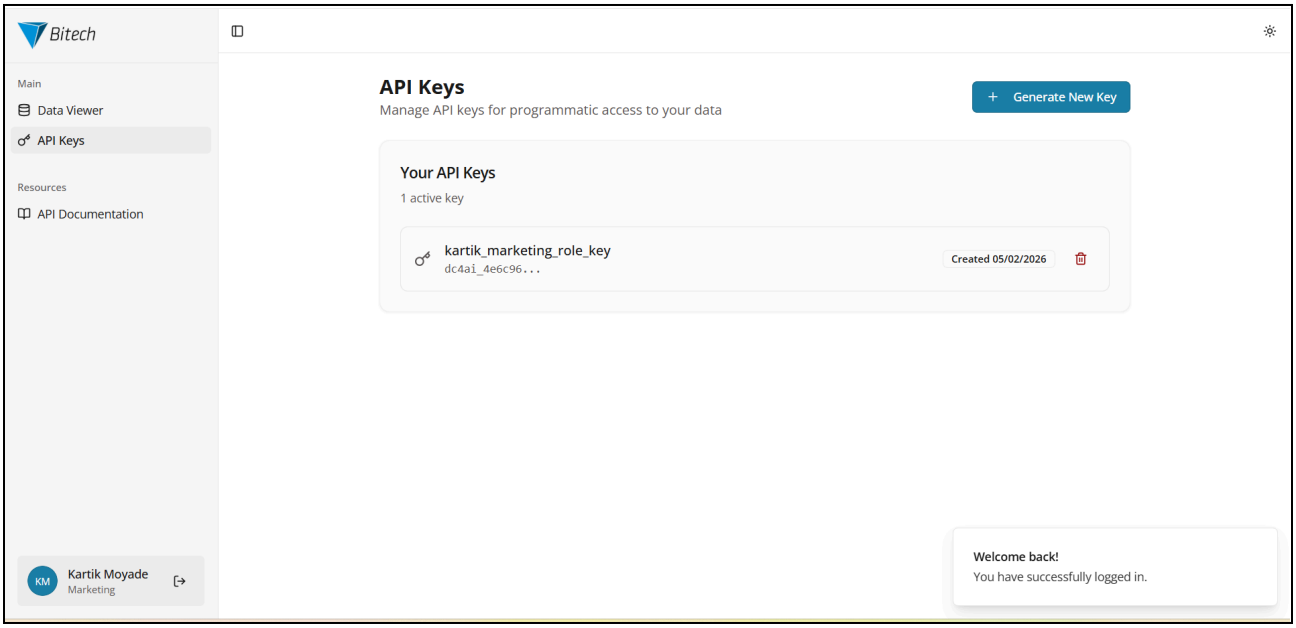
- Now lets create User



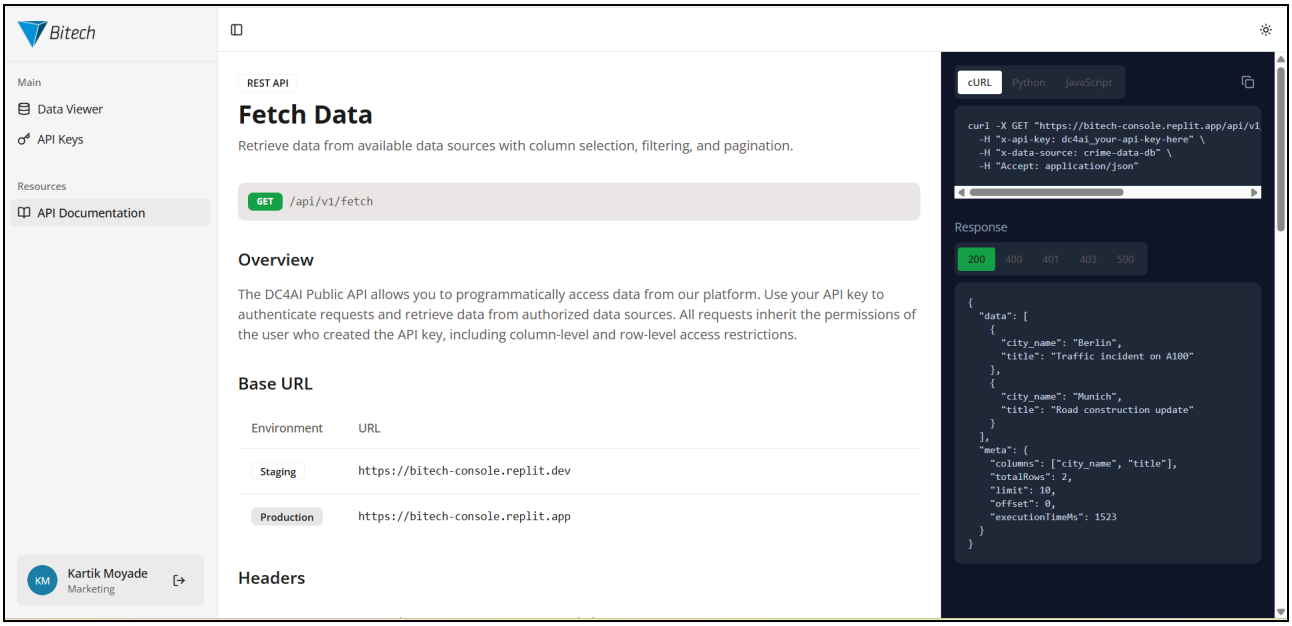
- Fill in all required fields → This user created will be reflected in the Cognitto user pool.



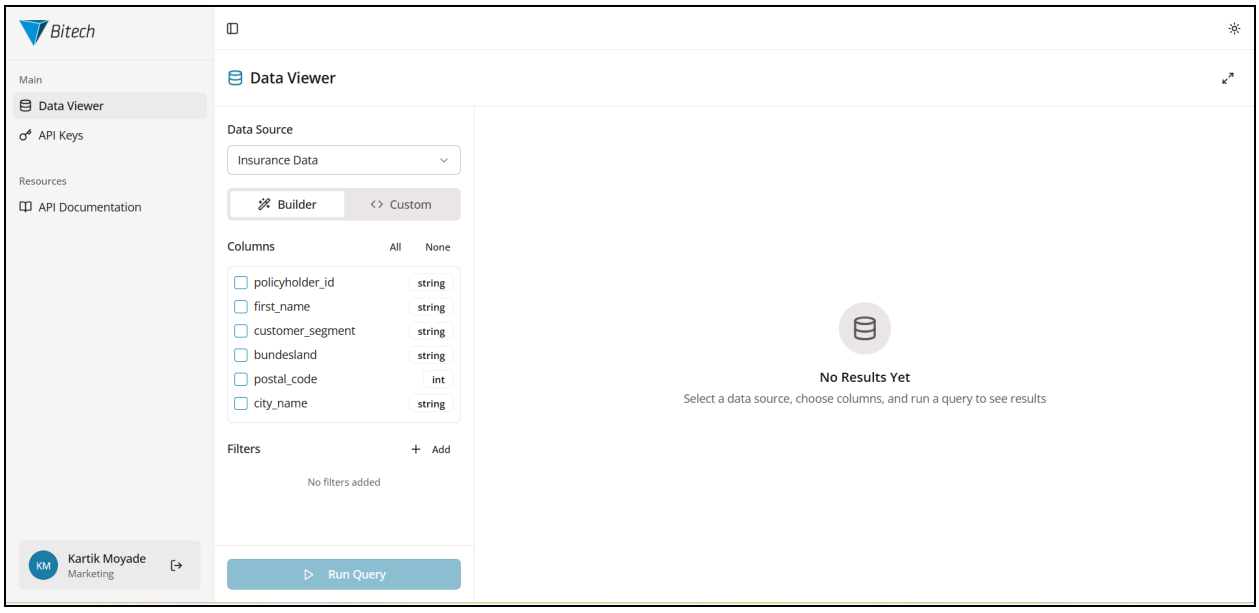
- Now login the user that is created (in this case we created for Marketing) → Generate API Key for that user.

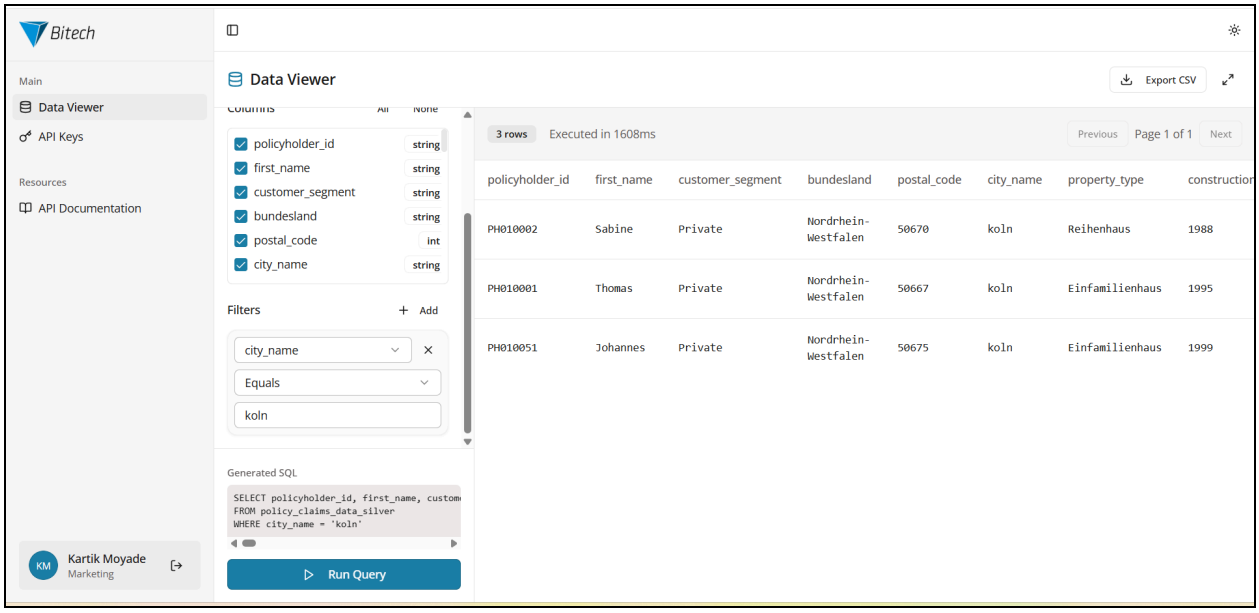


- Refer to the API documentation to understand how to access the APIs → Now go to the Data Viewer Section.

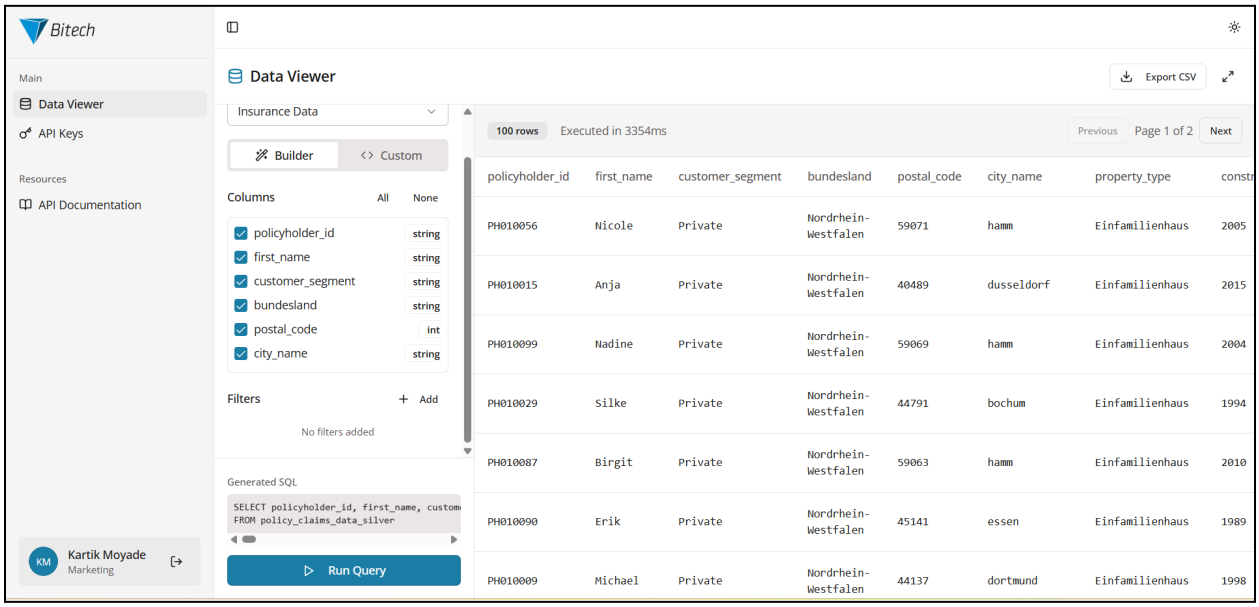


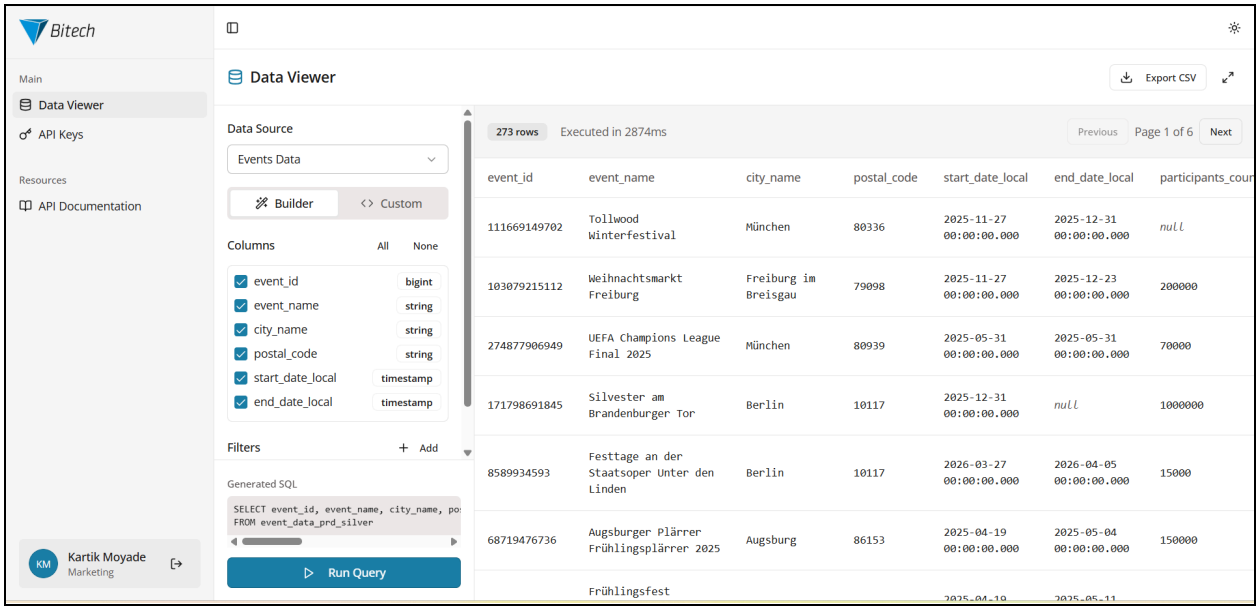
- This user has access to two data sources. The first is the Event data source which provides full Table level access.
- The second is the Insurance data source which has access restricted to specific columns. Users can query these data sources and perform any required operations based on their access permissions.





- Users can also write custom complex queries as needed.





- The Data Viewer interface demonstrates the flexibility and granularity of the PBAC platform from a user's perspective.
- As shown in the walkthrough, administrators can easily control access per role by defining which data sources are available, switching between table-level and column-level permissions, and applying row-level filters without requiring any changes to the underlying infrastructure.

The platform is also designed to seamlessly support new data sources such as mobility, environmental, or other external datasets. New sources can be onboarded by defining the required data and columns, creating the corresponding Glue ETL job, and storing the extracted data in the Bronze layer in its raw, as-is form. The data can then be transformed into the Silver layer, where it is cataloged through the Glue Crawler and Data Catalog, followed by the required access permissions in Lake Formation. This allows the data access model and level of granularity to evolve with changing business needs, providing a scalable and future-ready foundation for secure data consumption across diverse users and datasets.